

Sadržaj

Uvod	1
1. Promet električnih vozila	3
1.1. Stanice za punjenje	4
1.2. Autonomni promet	4
2. Postojeća literatura	8
2.1. Dinamički podaci simulacije	13
3. Problem raspodjele stanica	15
3.1. Postojeće tehnike	15
3.1.1. k-medijan i k-centar problem	16
3.1.2. Pokrivenost	17
3.1.3. Mješovito cjelobrojno programiranje	18
3.1.4. Teorija igara	19
3.1.5. Genetski algoritmi	20
3.1.6. Strojno učenje	21
3.2. Model okoline sustava	22
3.2.1. Model sa troškovima skretanja	23
3.2.2. Linijski graf	25
3.2.3. Graf proširen incidentnim čvorovima	26
3.3. Stanice	28
3.3.1. Alokacija vozila na stanice	28
3.3.2. Sustav raspodjele stanica	29
4. Implementacija	32
4.1. Alati	32
4.1.1. <i>Eclipse SUMO</i>	32
4.1.2. NetworkX	35

4.1.3.	OpenStreetMap	35
4.2.	Pod-mreža stanice za punjenje vozila.....	37
4.3.	Model izvršavanja evaluacije	39
4.3.1.	Generiranje prometne potražnje	40
4.3.2.	Odabir stanice za punjenje.....	42
4.4.	Graf neuronska mreža.....	43
4.4.1.	Rangiranje bridova	44
5.	Rješenja	45
5.1.	Algoritam baziran na pokrivenosti mreže	45
5.1.1.	Više-agentska kompetitivna varijacija.....	51
5.2.	Algoritam baziran na teoriji igara.....	52
5.2.1.	Više-agentska kompetitivna varijacija.....	57
5.3.	Strojno učenje	60
5.3.1.	Scenarij s jednim agentom.....	60
5.3.2.	Scenarij više agenata	62
6.	Rezultati.....	63
6.1.	Rešetka	65
6.1.1.	Najbolje rješenje	67
6.1.2.	Ponovno pokretanje	71
6.2.	Zagreb.....	74
6.2.1.	Najbolje rješenje	75
6.2.2.	Ponovno pokretanje	78
	Zaključak	81
	Literatura	83
	Sažetak.....	85
	Summary.....	86

Privitak	87
----------------	----

Uvod

Razvojem tehnologije električnih vozila sve više se istražuje mogućnost prometa dominiranog električnim vozilima i njegove koristi. Glavna korist takvog prometa zbog koje je i počeo rad na razvoju tehnologije električnih motora i vozila je korištenje električne energije umjesto fosilnih goriva, koja se može dobiti iz raznih obnovljivih izvora kojima se manje zagađuje okolina u kojoj živimo i iscrpno iskorištavaju ograničeni resursi. Također je i sama pretvorba fosilnih goriva u električnu energiju u elektranama efikasnija od direktnog gorenja u motorima s unutarnjim izgaranjem [1], te se dodatnim razvojem tehnologije električnih motora čak postiže i generalno bolja efikasnost samih električnih motora od motora sa unutarnjim izgaranjem [2]. No očekivani nedostatak prometa dominiranog električnim vozilima je povećano opterećenje elektroenergetske mreže budući da bi većina vozila ovisila o električnoj energiji koja se već koristi u infrastrukturi i o kojoj cijela postojeća infrastruktura i ovisi. Zbog toga su se razvila nova područja istraživanja kojima je cilj učiniti takav sustav električnog prometa ostvarivim u pravom svijetu, kao područje tehnološkog razvoja baterija i stanica za punjenje električnih vozila. Zasad su stanice za punjenje električnih vozila svojstveno drugačije od dosadašnjih benzinskih postaja po tome da je punjenje električnog vozila uveliko dugotrajnije te je zato potrebno više mjesta za punjenje za ostvarenje jednakog protoka vozila, te se zbog toga trenutne benzinske postaje ne mogu jednostavno zamijeniti sa stanicama za punjenje električnih vozila. Također su baterije trenutno puno manjeg efektivnog kapaciteta u suprotnosti sa konvencionalnim vozilima, te zbog toga stanice za električna vozila moraju biti na manjim udaljenostima, što zadaje dodatan problem. Zbog ta se dva razloga uz sami tehnološki napredak istražuju i naprednije metode postavljanja samih stanica za punjenje električnih vozila. U zadnjih par desetljeća također su započeta istraživanja o novoj mogućoj prednosti prometa električnih vozila, a to je autonomni promet u kojem je eliminirana greška i nepromišljenost ljudskih vozača. U idealnom potpuno autonomnom prometu moguće je optimizirati transport ljudi i materijala te bi nesreće bilo puno rjeđe zbog kontrole prometa od vanjskog sustava ili predvidljivim softverom samog vozila čime se omogućuje efektivna komunikacija između svih vozila, koja nedostaje u slučaju ljudskih vozača.

No kada razmislimo o tome u kontekstu realnog svijeta, praktički je nemoguće staviti sva vozila cijelog svijeta pod jedan sustav, nego bi bilo više različitih sustava za upravljanje pojedinim regijama, koji bi onda bili različiti i time nejednako efektivni. Ista logika može se primijeniti i na same stanice za punjenje električnih vozila, današnje benzinske postaje nisu pod vlasništvom jedne već više različitih tvrtka sa drugačijim cijenama, te zato isto možemo očekivati i za buduće stanice za električna vozila. Time dolazimo do glavne ideje kojom se ovaj rad bavi, analiza razlika između centralizirane i kompetitivne raspodjele električnih stanica, u slučaju autonomnog i sebičnog prometa. Korisnost ekonomske kompetitivnosti već je dobro istražena, kompetitivnost generalno poboljšava alokaciju resursa i smanjuje cijene, što bi bilo idealno u realnoj implementaciji raspodjele stanica za punjenje električnih vozila [5]. U radu su opisani spomenuti fundamentalni problemi i glavni problem optimizacije raspodjele stanica za punjenje električnih vozila te analizirane simulacije od par mogućih rješenja za svaki od opisanih slučajeva.

U prvom poglavlju detaljnije su opisana električna vozila, promet dominiran električnim vozilima i promet autonomnih vozila, uz opis prednosti, nedostataka i preostalih izazova. Drugo poglavlje prolazi kroz odabran skup postojeće literature koja pokušava riješiti dijelove problema optimalne raspodjele stanica za punjenje električnih vozila kroz različite pristupe, uz nabranje najčešće korištenih tehnika. U trećem poglavlju formaliziran je problem optimalne raspodjele električnih vozila i definirani model i pristup problemu u kontekstu rada, te su dodatno objašnjenje tehnike korištene u postojećoj literaturi. U četvrtom poglavlju prikazana je implementacija modela problema i korišteni alati. Peto poglavlje predstavlja rješenja razvijena u sklopu rada, te šesto poglavlje prikazuje rezultate dobivene primjenom razvijenih rješenja na dva primjera konkretnih problema.

1. Promet električnih vozila

Električna vozila bazirana su na električnom motoru koji je prvi put uspješno napravljen čak i prije danas popularnijeg motora s unutarnjim izgaranjem. Oboje su izumljeni krajem 19. stoljeća te je pri izumu motora s unutarnjim izgaranjem već bila velika zainteresiranost oko električnog motora i električnih vozila. Kroz sljedeću polovicu 20. stoljeća električna vozila bila su popularnija od konvencionalnih, zbog bolje efikasnosti, manje veličine i bolje robusnosti motora od motora sa unutarnjim izgaranjem. No kroz rapidni razvoj i poboljšanje motora s unutarnjim izgaranjem polagano su nedostaci električnih motora postajali preveliki. Glavni problem kod poboljšanja praktične korisnosti električnog motora za vozila su baterije, čiji razvoj je usko povezan sa samim razvojem motora, što u slučaju motora sa unutarnjim izgaranjem nije bilo potrebno nakon izuma tekućeg goriva. Baterije nisu samo dopuštale manji domet samih vozila nego su i ekstremno povećavale samu težinu vozila zbog lošeg omjera energije i mase baterije. Tako su konvencionalna vozila pretekla električna u popularnosti i omjeru osobnih vozila. No električni motori i dalje su mogli isporučiti veći okretni moment i snagu od motora sa unutarnjim izgaranjem, uz rjeđe kvarove i bolju izdržljivost te su se pokušali koristiti u specifičnim slučajevima kao kod tenkova ili trkaćih vozila, iako su se kod tenkova koristili uz pomoć motora sa unutarnjim izgaranjem, a kod trkaćih vozila su zadavali dodatni problem prevelike mase [3].

Kroz istraživanja o snazi utjecaja gorenja fosilnih goriva na okolinu zainteresiranost za električna vozila je opet porasla. Otkrivene su nove metode kreiranja baterija s većim kapacitetima, manjom masom i efikasnijom proizvodnjom te se sve više nade polaže u veće korištenje električnih vozila u svijetu. No još uvijek postoji glavni nedostatak zbog kojeg su električna vozila izgubila bitku protiv konvencionalnih prije stotinjak godina, a to je veliko ograničenje dometa u suprotnosti sa dosad uobičajenog. Velika većina ljudi i dalje preferira konvencionalna vozila zbog činjenice da nije potrebno više puta stati tijekom puta za punjenje energije vozila. Za olakšanje utjecaja tog problema predoče dva rješenje, poboljšanje same tehnologije baterija za brže punjenje i veći kapacitet te fokusiranje na drugačiji pristup, mijenjanje same baterije na stanicama za punjenje umjesto punjenja. Prvo rješenje je previše dugotrajno uzimajući u obzir kontinuirano oštećenje okoline, dok drugo donosi mnoge druge nedostatke, kao kvarenje baterija i potrebne kompatibilnosti baterija sa

svim električnim vozilima, no za oba slučaja je potrebna efektivna raspodjela stanica za punjenje.

1.1. Stanice za punjenje

Stanice za punjenje električnih vozila su osmišljene još i prije ideje benzinskih postaja. Budući da je prvo razvijen električni auto te je prvi bio popularan, osmišljene su i stanice za punjenje u urbane i izvan urbane svrhe. U New Yorku je oko 1900. godine napravljen takozvani „*charging hydrant*“ sustav, tj. sustav „hidranata“ za punjenje električnih vozila. Funkcionirao je na način da korisnik ubaci novce te mu „hidrant“ ispiše koliko watt sati punjenja može dobiti za tu cijenu. Naravno, i dalje je potreba za dugotrajnim punjenjem bila veliki problem za proširenje korištenja električnih vozila, te su zato osmišljene spomenute stanice za izvan urbano punjenje. Te stanice prate ideju mijenjanja baterije, čime bi pokušale riješiti problem kratkog dometa kroz često no minimalno stajanje na stanicama za punjenje tijekom puta. Također su isprobane razne danas učestale metode, kao punjenje baterije pri kočenju i hibridna vozila. Nažalost ništa od toga nije doživjelo dovoljno veliku popularnost zbog brzog razvoja vozila koja koriste motor sa unutarnjim izgaranjem [3].

1.2. Autonomni promet

Autonomni promet podrazumijeva promet od ekskluzivno autonomnih vozila, dok mješovito autonomni promet označava postojanje konvencionalnih vozila uz određen postotak autonomnih vozila. Autonomno vozilo definira se kao vozilo upravljano od strane ugrađenog računala koje ima sposobnosti samostalnog navođenja, razumijevanja okoline, donošenja odluka i potpuno samostalne operacije bez potrebe za ljudskom interakcijom. Ideja autonomnih vozila potječe još od 1920-ih godina, kada su se razvijala takozvana „fantomska vozila“ koja su upravljana preko daljinskog upravljača. Takva vrsta vozila, i dalje upravljana sa strane čovjeka, danas se naziva „automatiziranim“ vozilom, no nedavnim naglim razvojem tehnologije automobila i računarske znanosti ta ideja evoluirala u ideju potpuno samostalnog vozila, tj. „autonomnog“ vozila [4].

Samostalnost autonomnih vozila ostvaruje se kombinacijom raspodijeljenih upravljajućih sustava i senzora koji prikupljaju informacije o okolini. Zbog raznih prednosti i nedostataka svake od tehnologije senzora više njih se koristi u isto vrijeme, kao radar kratkog i dugog

dometa, *Light detection and ranging* (LIDAR) te infracrvene i konvencionalne kamere. Dobiveni podatci od senzora zatim se obrađuju u raspodijeljenim sustavima, kao sustave za lokalizaciju, tj. procjenu položaja i orijentacije, navigaciju i odlučivanje [4].

Postepenom globalizacijom automobila javlja se problem sve veće količine vozila na cesti što povlači češće nesreće, povećanje negativnog utjecaja cestovnog prometa na okoliš, pogoršanje protoka i veći pritisak na infrastrukturu [4], zbog čega su potrebna nova rješenja optimizacije navedenih područja prometa, od kojih je jedno od mogućih autonomni promet. Autonomna vozila eliminiraju mogućnost ljudske greške zbog koje može doći do nesreće, kao nedostatak koncentracije, vožnja pod utjecajem alkohola, droga ili umora te nedostatno znanje prometnih pravila, kojih bi se idealna autonomna vozila uvijek pridržavala. Razvijenom tehnologijom kontinuirane međusobne komunikacije i dijeljenja informacija između autonomnih vozila također se cestovni promet čini sigurnijim. Autonomnim vozilima može se povećati i broj putnika po vozilu i time smanjiti broj vozila na cesti, čime se unaprijeđuju prometni uvjeti. Također se kroz dijeljenje informacija može postići optimalna raspodjela vozila po prometnoj mreži, grupirati vozila u takozvane „vodove“ koja se kreću zajedno te smanjiti razmak između auta i ubrzati kretanje u gužvi. Time se također smanjuje potrošnja goriva zbog kraćih puteva, rjeđeg stajanja u redovima i manje potrebe za ubrzanjem iz stanja mirovanja. Većina razloga nepotrebne potrošnje goriva dolazi zbog ponašanja ljudskih vozača, kao vožnja nepotrebno velikim brzinama, nepravilna vožnja, konstantno kretanje i zaustavljanje te iznenadno kočenje, koji se svi eliminiraju komunikacijom autonomnih vozila [4].

No uz sve nabrojene prednosti autonomnog prometa naravno postoje i brojni nedostaci, uglavnom zbog mogućnosti krivog ili nedefiniranog ponašanja sustava autonomnog vozila. Ljudski vozač uvijek će imati prednost organske prilagodljivosti kojom se može snaći u bilo kojoj neočekivanoj situaciji, dok je ponašanje sustava autonomnog vozila nedefinirano u situacijama za koje nije unaprijed razvijen. Sustav potpuno samostalnog autonomnog vozila sastoji se od velikog broja međuvisnih komponenata koje konstanto surađuju za donošenje odluka, te uvijek postoji mogućnost kvara ili gubitka shvaćanja konteksta u kojem se vozilo nalazi koje može dovesti do katastrofalnih posljedica [4].

Posebnost dosad razvijene tehnologije autonomnih vozila je sposobnost efektivne komunikacije sa okolnim vozilima i prometnom infrastrukturom u stvarnom vremenu. Vozila se spajaju u lokalnu mrežu kroz takozvanu *Vehicular ad-hoc network* (VANET)

arhitekturu koja omogućuje opisanu komunikaciju (V2X), koja se dodatno dijeli na komunikaciju između vozila (V2V) te vozila i okolne prometne infrastrukture (V2I). Konkretna implementacija mreže koristi razne tehnologije kao Wi-Fi, LTE i *Visible light communications* (VLC), uz razvoj novih posebno za V2X kao *Dedicated short-range communications* (DSRC) i *IEEE 802.11p*, što je specijalizacija Wi-Fi tehnologije za V2X komunikaciju. Koristeći formiranu mrežu okolna vozila i infrastruktura dijele prometne informacije, od kojih su u kontekstu rada relevantne informacije o navigaciji, usmjeravanju i stanju prometne infrastrukture, tj. u ovom slučaju stanica za punjenje električnih vozila [4]. Kroz V2X komunikaciju vozila bi mogla doći do informacija o stanju stanica za punjenje, kao količina slobodnih mjesta i broj vozila koje čeka na red, te bi bilo moguće uzeti u obzir i puteve ostalih vozila pri odabiru stanice za punjenje.

Stoga se samostalna vozila dodatno razlikuju po tome da li aktivno uzimaju u obzir informacije o okolini i komuniciraju s okolnim vozilima i infrastrukturom ili samo koriste osnovne informacije da uspješno stignu do odredišta. Drugi tip vozila naziva se „ego-vozilo“, te su na taj način funkcionirala početna samostalna vozila, dok današnjim razvojem tehnologije taj tip vozila ostaje u prošlosti [4].

Autonomni promet je u sklopu rada modeliran na dva načina. Prvi koristi *StationFinder* algoritam koji je dio *Eclipse SUMO* programskog paketa (opisan u 4.1.1.2), te predstavlja autonomnost na način da ugrađeni sustav vozila optimalno zaključi kada treba ići do stanice, koju stanicu odabrati i koliko energije treba napuniti kroz informacije o energiji potrebnoj za dovršavanje puta i pretpostavci o prosječnom vremenu čekanja na stanici. Drugi implementira centralizirani sustav koji upravlja raspodjelom vozila na stanice koristeći konkretne informacije o mreži i stanicama, koristeći funkciju cilja koja uključuje dinamičko izračunato vrijeme putovanja do stanice i slobodna mjesta te red čekanja stanice (opisan u 3.3.1). Tako prvi način zapravo modelira sebični oblik autonomnog vozila, tj. spomenuto „ego-vozilo“, dok drugi zahvaća V2X komunikaciju među vozilima čime svako vozilo ima pristup konkretnim podacima o stanju prometa i stanica za punjenje te može donijeti globalno bolju odluku o odabiru stanice za punjenje.

No, osim na razini provođenja ispitivanja, tehnologija autonomnih vozila još nije na dovoljnoj razini za uspješnu implementaciju u globalnom prometu. Preostali su mnogi tehnički, socijalni i politički izazovi do usavršavanja tehnologije i prodora na tržište [4]. Sve

dok autonomna vozila ne izađu uspješno na tržište nabrojene prednosti autonomnog prometa ostaju na razini teorije i rezultata ispitivanja, te stvarne utjecaje na globalnoj razini vjerojatno nećemo vidjeti u skoro vrijeme. U svrhu predočanja ideje dugotrajnosti implementacije nove tehnologije na razini automobila može se spomenuti da je za prodor tehnologije automatskog mjenjača na tržište bilo potrebno razdoblje od pet desetljeća prije postizanja dovoljno dobre kvalitete sustava i isplativosti uporabe [4].

2. Postojeća literatura

Problem se u postojećoj literaturi uglavnom rješava koristeći algoritme grupiranja ili pokrivenost [5, 7], tehniku mješovitog cjelobrojnog linearnog programiranja (MILP) [5, 6], genetske algoritme [5, 8, 9, 10] i strojno učenje [13, 14], te koristeći tehnike iz teorije igara [11, 12].

Rad [5] formulira problem raspodjele stanica za punjenje električnih vozila kroz model sustava u obliku grafa i definirajući model mješovitog cjelobrojnog linearnog programiranja (MILP) za rješavanje problema. Model sustava formulira kao neusmjereni graf $G(N, E)$ u kojem je svaki čvor potencijalna lokacija za izgradnju stanice. Bridovi su onda udaljenosti između stanica, koja nije specificirana točnom formulom nego kao generalizirano najkraći put između dva čvora. Također definiraju attribute čvorova: kapacitet posluživanja i zahtjev potražnje F_i , koji je jednak lokalnoj potražnji. Zatim definiraju pokrivenost stanice kroz definiciju dobro planirane raspodjele, koja označava da je u bilo kojem čvora grafa najbliža stanica unutar udaljenosti D , što je maksimalna udaljenost koju potpuno napunjeno vozilo može proći. Zatim dodatno definira vrijednost c_i koja reprezentira trošak izgradnje stanice na čvoru i . Još nadograđuje formulaciju grafa pojednostavljenjem na podgraf H na način da uključuje samo bridove iz G čije su duljine manje ili jednake od D , te zatim eliminira novo nepovezane čvorove, čime se dobiva jedna povezana komponenta. Nakon toga rad dokazuje NP-težinu problema i predstavlja četiri moguća razvijena rješenja. Prvo rješenje je iterativni MILP model definiran kroz opisani graf sustava i koncept mrežnog toka, koji u ovom slučaju predstavlja tok vozila, tj. prometne potražnje. Izvori toka su definirani u čvorovima grafa, te svaki čvor ima vrijednost x_i koja označava da li je odabran, tj. da li je njemu postavljena stanica. U početnom stanju modela svi su čvorovi ne-odabrani, te je dokazano da je problem rješiv u slučaju da imamo početno stanje u kojem je barem jedan čvor odabran, tj. jedna stanica postavljena. No to vodi do računalne složenosti budući da moramo izračunati model za svaki čvor grafa. Drugo rješenje je pohlepni algoritam koji iterativno pokušava maknuti najskuplje (po trošku izgradnje c_i) čvorove iz grafa bez da rezultirajući graf bude nepovezan. Treće rješenje je nadogradnja na drugo, koja rezultira u takozvani „efektivni“ MIP model. Pomoću pretpostavke da je lokalna potražnja F_i svakog čvora veća od nule dokazuje se da se ta potražnja mora ispuniti pomoću susjednog čvora, što vodi na zaključak da se MILP prvog rješenja može riješiti fokusom na jedan čvora i izračunom modela u slučaju odabira

svakog od susjednih čvorova umjesto svih čvorova grafa. Posljednje rješenje je primjena prirodom inspiriranog metaheurističkog postupka; optimizacija kemijskim reakcijama (CRO). Nakon svega, rad je zanimljiv zbog formulacije problema na način da je svaki čvor potencijalna lokacija za stanicu te su bridovi samo udaljenosti između njih, bez direktnog modeliranja detaljne strukture prometne mreže.

Za glavni primjer korištenja mješovitog cjelobrojnog linearnog programiranja (3.1.3) pregledat ćemo [6]. Cilj rada je opet optimizacija raspodjele stanica po trošku izgradnje i održavanja, tj. trošak punjenja vozila na lokaciji, uz kasnije dodavanje devijacije napona kao dodatnu kaznu. Problem je prvo formaliziran u obliku modela mješovito cjelobrojnog nelinearnog programiranja (MINLP), koji je zatim ekstrahiran u jednostavniji linearni (MILP) ekvivalent. Početni MINLP problem sastojao se od nelinearnih i ne-konveksnih jednačja zbog čega je izvršavanje metoda za rješavanje problema bilo vrlo dugotrajno. Transformacija u MILP problem se vrši aproksimacijom par realnih varijabli u cjelobrojne, linearnu aproksimaciju jednačja kružnice podjelom na cjelobrojnu količinu dijelova i linearnom jednačjom devijacije napona. Tako formaliziran MILP model primijenjen je na 69-sklopnu radijalnu distribucijsku mrežu. Svaki od 69 sklopova potencijalno je mjesto za postavljanje stanice, tj. spajanje stanice na distribucijsku mrežu. Originalni MINLP model također je primijenjen na mrežu i uspoređeni su rezultati. Razlike između rezultata su minimalne, te su oba modela došli do jednake raspodjele stanica, dok je vrijeme izvršavanja MINLP modela bilo znatno dulje od MILP modela. Zaključuje se da je MILP model prikladna zamjena originalne formulacije problema.

U [7] autori pokušavaju riješiti problem raspodjele stanica u kontekstu jedne od gradskih četvrti Lisabona, glavnog grada Portugala, s najvećim brojem stanovnika. Prije razvijanja rješenja sakupljaju potrebne podatke kroz informirane procjene: procjenjuju očekivanu prometnu potražnju, razdvajajući je na potražnju po danu i po noći, gdje potražnja po danu korelira sa punjenjem u blizini poslovnih prostora, dok je noćna u blizini stambenih prostora. Zatim iz podatka o planiranom postavljanju određenog broja stanica u gradu procjenjuju broj stanica u fokusiranom kvartu na 43 stanice po omjeru stanovnika gradskih četvrti, te definiraju da će stanice imati od 2 do 10 mjesta za punjenje. Za moguće lokacije stanica uzimaju sva parkirališta gradske četvrti, kojih je onda 290. Zadavanjem svih potrebnih podataka kreću na pokušaj rješenja problema, definiraju pokrivenost stanice po L1 ili *Manhattan* udaljenosti, te zadaju maksimume poželjne i prihvatljive udaljenosti pokrivenosti

od 350 i 550 metara. Obuhvaćajući sve podatke i definicije formiraju model mješovitog cjelobrojnog programiranja (MIP) kroz 12 jednadžba i funkcijom cilja maksimizacije pokrivenosti potražnje, posebno za slučaj potražnje tijekom dana i tijekom noći. Definiraju četiri različita scenarija po broju ukupnih mjesta za punjenje koje je potrebno instalirati i broju noćnih punjenja, te rješavaju postavljeni model MIP za svaki scenarij. Rezultati prikazuju skoro potpunu pokrivenost četvrti, osim područja sa minimalnom lokalnom potražnjom [7]. Sve u svemu, rad koristi tehnike pokrivenosti mreže i zadavanje problema kroz model mješovitog cjelobrojnog programiranja, te je zanimljiv po temeljitim definicijama i procjenama potražnje te fokusom na broj mjesta za punjenje umjesto broju stanica.

U radu [8] javlja se ideja praćenja količine neuspješnih putovanja, tj. vozila koja ne mogu stići do niti jedne stanice sa količinom električne energije koju imaju. Cilj rada je pokušaj pronalaska optimalne raspodjele stanica za punjenje električne energije kroz višekriterijski optimizacijski problem minimizacije troška punjenja vozila na lokaciji stanice i količine neuspješnih putovanja. Generiraju se podaci potražnje kroz estimaciju iz statistike grada Singapura: količina vozila, omjer električnih vozila u prometu, modeli električnih automobila i prosječna potrošnja električnih vozila. Iz tih podataka definirano je 2.45 milijuna aktivnosti od 3,100 vozila sa omjer električnih vozila od 0.5%, simulirane 164 puta. Dobivene su informacije o neuspjelim putovanjima te su po njima rangirane lokacije stanica. Zatim je razvijen selekcijski algoritam za odabir važnijih potencijalnih lokacija i genetski algoritam za krajnje određivanje raspodjele iz smanjenog odabira. Za funkciju cilja definiran je već spomenut višekriterijski optimizacijski problem troška punjenja vozila na lokaciji stanice i broja neuspjelih putovanja do stanice. Neuspješnost putovanja određuje se postotkom napunjenosti baterije nakon puta, korišteni prag je 5% kapaciteta baterije. Rezultati provođenja simulacija jasno pokazuju kompromis između investicije i broja neuspjelih putovanja, te se dodatno ističe povećanjem omjera električnih vozila u prometu. Kriterij važnosti praćenja količine neuspjelih putovanja također se implementira u ovom radu, no u znatno manjoj količini. Podaci potražnje generiraju se slično kao u opisanom radu, konkretno početna napunjenost baterije i energija potrebna za napuniti na stanici za svako vozilo (4.3.1) te može doći do slučaja da vozilo ne stigne do stanice ili na red na stanici da uspije napuniti potrebnu količinu energije. No nisu korištene stvarne statistike, već su podaci energije i putevi proceduralno generirani da emuliraju veće gužve na gušće povezanim

dijelovima mreže, te da je time sama proceduralna generacija primjenjiva na bilo kakvu strukturu prometne mreže.

Budući da se problem optimalne raspodjele stanica za punjenje električnih vozila može formularizirati u jedno-kriterijski ili višekriterijski optimizacijski problem (3.1.5) iskorišteni su mnogi evolucijski algoritmi za pokušaj pronalaska optimalnih rješenja, te je dobar primjer toga rad [10]. U sklopu rada razvijen je novi genetski algoritam baziran na kombinaciji dva postojeća: *Chicken Swarm Optimization* (CSO) i *Teaching Learning Based Optimization* (TLBO). CSO je evolucijski algoritam koji simulira proces traženja hrane od strane kokoši, dok TLBO emulira proceduru podučavanja i učenja. Problem je modeliran minimizacijom jednostavne jedno-kriterijske funkcije cilja ukupnog troška stanice. Ukupni trošak stanice dijeli se na direktni i indirektni, gdje trošak izgradnje i održavanja stanice spada pod direktni. Indirektnim troškom model zahvaća pouzdanost elektrodistribucijske mreže pri odabiru lokacija za stanice, kroz kaznu za kršenje prosječne neisporučene energije (*Average Energy Not Served*, AENS), uz kombinaciju korisničke pogodnosti kroz dodatak troška vremena putovanja korisnika do najbliže stanice. Time je funkcija cilja zapravo zbroj uključenih troškova uz definiciju koeficijenta težine svakog od troškova na traženi rezultat. Jedinka je vektor $X_i = \{x_{i1}, \dots, x_{in}\}$ gdje su x_{ik} binarne varijable te $x_{ik} = 1$ označava odabir stanice na indeksu k . Populacija je onda matrica izgrađena iz više vektora jedinka. Hibridni algoritam većinom je baziran na TLBO, uz dodatak mehanizma evaluacije uspješnosti iz CSO algoritma. Definirane su dvije varijacije hibridnog algoritma koje su zatim uspoređene sa CSO, TLBO i još nekim state-of-the-art evolucijskim algoritmima. Analiza rezultata pokazuje da prva varijacija hibridnog algoritma postiže bolje rezultate uz slično vrijeme izvršavanja, dok druga varijacija brže konvergira uz dulje vrijeme izvršavanja. Novost koju rad uvodi osim hibridnog algoritma je obuhvaćanje performansi elektrodistribucijske mreže pod utjecajem rada odabranih stanica.

Rad [11] kontekstualizira problem u problem grupiranja (3.1.1) te ga rješava tehnikom baziranom na konceptu Nashove ravnoteže iz teorije igara (0). Novost rada je uzimanje i perspektive korisnika stanica u obzir pri optimiranju raspodjele stanica. Osim konvencionalnog kriterija troška izgradnje i održavanja stanice uključuju se i želje korisnika. Želje korisnika modeliraju se kroz kombinaciju vremena putovanja do stanice uz faktor „gužve“ (*crowdedness*) i specifičnih preferencija korisnika kao brend i lokalne atrakcije u blizini stanice. Takav model onda stvara kompetitivni element optimizacije, gdje korisnici

biranjem stanice koje preferiraju no u isto vrijeme izbjegavaju popularne stanice sa velikom gužvom [11], što je realistično zato što je dugotrajnost čekanja na stanici jedna od velikih izazova pri globalnom prihvatu električnih vozila [3]. Stvara se igra sume nula u kojoj odabir stanice sa strane jednog korisnika njemu ostvaruje dobitak dok smanjuje vrijednost stanice za ostale. Simulacija modela izvršava se na način da korisnici iterativno i istovremeno biraju stanicu koju preferiraju u svrhu postizanja dobitka čime se formiraju grupe korisnika po odabranim stanicama. Proces se nastavlja dok god korisnici odluče promijeniti izbor iz prošle iteracije, te pri konvergenciji odabira na način da niti jedan korisnik ne može dobiti veći dobitak promjenom odabira sustav dolazi u stanje Nashove ravnoteže. Sada su svi korisnici uspješno grupirani po odabranim stanicama čime je zapravo izvršeno k -grupiranje, gdje je k ukupan broj stanica. No, pravi cilj optimizacije problema raspodjele stanica je zapravo zadovoljenje prometne potražnje uz ostvarenje što boljih prometnih uvjeta, tj. u ovom radu nazvano socijalnim kriterijem. Stoga je svrha sljedećeg koraka algoritma izračun nove raspodjele stanica prateći socijalni kriterij. Za svaku grupu korisnika izračunava se centroid, tj. točka koja je sveukupno najmanje udaljena od lokacija svih korisnika grupe zajedno, čime se smanjuju udaljenosti koje će morati proći. Odabire se lokacija za stanicu najbliža izračunatom centroidu kao zamjena za trenutnu stanicu oko koje je formirana grupa. Algoritam je nazvan k -GRAPE (*Group Agent Partitioning and Placing Event*), te se uz njega koriste i određene heuristike za olakšanje složenosti problema. Rad zatim prezentira efektivnost razvijenog algoritma nad pokrivenom literaturom, uz detalj da je algoritam 17% uspješniji od tada aktualnog algoritma također baziranog na teoriji igara. Rad je generalno zanimljiva kombinacija i primjena grupiranja i Nashove ravnoteže u algoritam za rješavanje problema, te prikazuje klasični način korištenja Nashove ravnoteže u kontekstu optimizacije kombinatorno složenih problema.

Za primjer rješenja baziranog na tehnikama strojnog učenja uzet ćemo u obzir rad [13]. Rad postavlja problem raspodjele stanica u veći kontekst prostora županije, te umjesto konkretnih lokacija pokušava predvidjeti gustoću raspodjele stanica po prostoru. Glavni cilj rada je sakupiti saznanja o socijalnim, ekonomskim i demografskim faktorima kojima se može kvantificirati trenutna nepravedna raspodjela stanica za punjenje električnih vozila i kako bi se trebale raspodijeliti buduće stanice s obzirom na postojeće. Za konkretni prostor odabran je *Orange County* u Kaliforniji te su prikupljeni i prikazani socioekonomski podaci regije. Gustoća postojećih stanica za punjenje obrađena je kroz tehniku procjene gustoće jezgre

(*Kernel Density Estimation*, KDE) te je gustoća transformirana u mapu intenziteta (*heatmap*) sa tri razine intenziteta. Zatim su razvijena tri modela strojnog učenja sa ciljem zaključivanja kroz obradu podataka iz postojećih stanica i solarnih panela te spomenutih socioekonomskih faktora, konkretno model slučajne šume (*Random Forest*), *support vector machine* (SVM) i model multinomijalne logističke regresije. Modeli su primijenjeni na ciljanu regiju te je najbolji rezultat pokazao model slučajne šume. Uglavnom, rad je zanimljiv zbog analize generalizirane varijacije problema raspodjele stanica u većem prostoru, analizirajući samo gustoću umjesto detaljnih karakteristika stanica koju zatim uspješno primjenjuje za dobivanje korisnih zaključaka o nepravednoj raspodjeli stanica.

U ovom radu će problem optimalne raspodjele stanica biti formaliziran sa fokusom na bridove umjesto čvorove grafa kao potencijalne lokacije stanica (3). Stoga je zanimljiv rad [14] kao primjer korištenja strojnog učenja za rangiranje bridova mreže. Cilj tog rada se znatno razlikuje od dosadašnjih, a to je razvoj modela graf neuronske mreže (GNN) za identificiranje kritičnih i ključnih cesta grada radi poboljšanja otpornosti ili brže obnove nakon katastrofe. Koristi se novija tehnika GNN modela (4.4) uz modifikaciju fokusa modela na bridove umjesto čvorove, zamjenom matrice susjedstva čvorova sa matricom susjedstva bridova. Model koristi matricu susjedstva bridova uz matricu značajka te dodatno modificira matricu susjedstva po stupnjevima čvorova i svojstvima bridova, nad kojima zatim izvršava unaprijedni i unazadni prolaz kroz proces učenja. Model je primijenjen uz par konvencionalnih algoritama rangiranja bridova na graf kojeg čini 10000 bridova, te tri sintetički generirana slučajna grafa. Rezultati pokazuju da je razvijeni model efikasniji od ispitanih postojećih rješenja, uz umjereno smanjenje točnosti. Model je također implementiran na način da se generalno može primijeniti na bilo koji graf. Prateći ideju korištenja strojnog učenja za rangiranje bridova rješenje bazirano na strojnom učenju inspirirano opisanim modelom razvijeno je u sklopu ovog rada, no kroz drugačiji pristup (4.4.1).

2.1. Dinamički podaci simulacije

Svi navedeni radovi koriste statičke podatke o prometnoj mreži i okolini, kao strukturu mreže, ekonomske i socijalne faktore regije i slično, ne zahvaćajući dinamične podatke prometa, kao prosječnu gustoću vozila. Razvojem simulatora za dinamičku analizu prometa kroz iterativnu simulaciju moguće je zahvatiti i dinamičke podatke pri testiranju algoritama,

koji se zatim mogu zamijeniti statistikom prometne mreže iz stvarnog svijeta pri praktičnoj primjeni algoritma.

Za jednostavno objašnjenje korisnosti dinamičkih podataka možemo uzeti u obzir protok vozila po pojedinim elementima prometne mreže. Iako je logično da će raskrižja i ceste koje su gušće povezane imati veći protok vozila, mogući su razni netrivialni faktori koji utječu na razliku između tih protoka. Kroz simulaciju prometne mreže možemo dobiti konkretne podatke o razlikama protoka između dva gusto povezana raskrižja te time zahvatiti faktore kao koliko dugo treba vozilima za skretanje u raskrižju ili politiku upravljanja signalizacije semafora raskrižja koji oboje mogu imati veliki utjecaj na ukupni protok vozila kroz raskrižje. Sakupljanjem dinamičkih podataka za svaki element prometne mreže također se može doći do odluke micanja određenih elemenata iz liste potencijalnih mjesta za stanice za punjenje vozila čime se cjelokupni problem čini utjecajno lakšim za riješiti, iako se kroz previše takvog zaključivanja potencira suboptimalnost rješenja. Kroz statističke analize prometnih mreža regija u svijetu moguće je prikupiti slične dinamičke podatke prometa kao što su dobiveni iz simulacija te ih na isti način primijeniti u rezultirajućem algoritmu. Zbog navedenih razloga ovaj rad također testira i algoritme ojačane sa dinamičkim podacima praznih simulacija prometa zadane prometne mreže.

3. Problem raspodjele stanica

Problem raspodjele stanica je generalno problem određivanja lokacija objekata u kojem je cilj minimizirati cijenu (ili maksimizirati profit) pri zadovoljavanju globalne ili lokalne potražnje sustava. Generalno su zadane određene cijene za postavljanje objekata i posluživanje klijenata za svaku od potencijalnih lokacija. No problem je primjenjiv na razna specifična područja, kao lokacija bankovnog računa, naftne rafinerije, [15]

U slučaju raspodjele stanica objekti čije lokacije moramo optimalno odabrati su stanice za punjenje električnih vozila, dok je potražnja električna energija koju vozila sustava traže, uz potrebu da svako vozilo može doći do barem jedne stanice te dobiti slobodno mjesto za punjenje u razumno vrijeme.

Problem se generalno dijeli na dvije razine:

- 1) Odabir potencijalnih lokacija za postavljanje stanica
- 2) Raspodjela stanica na unaprijed određen broj lokacija među odabranim kandidatima

Većina postojeće literature rješava problem odabira potencijalnih lokacija za postavljanje stanica na način da se prometna mreža transformira u oblik gdje čvorovi predstavljaju fizički moguće lokacije postavljanja stanice.

U ovom radu su za potencijalna mjesta stanica odabrani bridovi po uzoru na postavljanje benzinskih postaja uz cestu umjesto direktno na raskrižja, te zbog tehničkog razloga da je moguće efektivnije dodati stanice u mrežu za simulacije kroz umetanje novog čvora usred brida i spajanje sa pod-mrežom stanice. Time je svaki brid mreže potencijalno mjesto za postavljanje stanice što pogoršava kombinatornu složenost problema

3.1. Postojeće tehnike

Iskorištene su razne tehnike i metode za transformaciju originalnog problema u jednostavniju formulaciju.

3.1.1. k-medijan i k-centar problem

Jedna od tehnika je grupiranje potencijalnih lokacija po određenim sličnostima sa ciljem smanjenja kombinatorne složenosti raspodjele. Ideja je da bi odabirom bilo koje lokacije u grupi bili postignuti dovoljno slični rezultati kao da je odabrana bilo koja druga od lokacija grupe. Pri grupiranju je ključna definicija parametara po kojima se određuje sličnost lokacija. Čest je slučaj da ne postoji dovoljno prirodnih faktora za efektivno određivanje sličnosti pri čemu se zadaje dodatno ograničenje da se smije odabrati samo k stanica, što je zapravo ekvivalentno grupiranju na k grupa. Time se problem generalizira u takozvani problem lokacije k -objekata, koji se zatim može razdijeliti na k -medijan ili k -centar problem [15].

Razlika između dva spomenuta problema je način definiranja funkcije cilja. Neka je X skup svih točaka sustava, f funkcija cilja, $C \subseteq X$ skup potencijalnih lokacija objekata te $\lambda: C \times X \rightarrow \mathbb{R}^+$ funkcija udaljenosti točke $x \in X$ od potencijalne lokacije $c \in C$

$$\lambda(c, x) = d(x_c, x) \quad (3.1)$$

gdje je $d: X \times X \rightarrow \mathbb{R}^+$ proizvoljno definirana funkcija udaljenosti te x_c točka potencijalne lokacije objekta u sustavu. Dodatno generaliziramo funkciju udaljenosti pri kalkulaciji udaljenosti neke točke $x \in X$ do najbližeg objekta

$$\lambda(S, x) = \min_{s \in S} \lambda(s, x) \quad (3.2)$$

gdje je $S \in C$ podskup odabranih k od svih potencijalnih lokacija objekata [15].

U k -medijan varijaciji cilj raspodjele objekata je minimizacija sume udaljenosti svih točaka sustava do njima najbližeg objekta,

$$f = \min_{S \subset C} \sum_{x \in X} \lambda(S, x) \quad (3.3)$$

gdje je $S \subset C$ skup k odabranih lokacija za postavljenje objekata [15], dok je cilj k -centar problema minimizacija maksimalne udaljenosti svake točke do njoj najbližeg objekta

$$f = \min_{S \subset C} \max_{x \in X} \lambda(S, x) \quad (3.4)$$

pri čemu funkcija udaljenosti može biti konkretno definirana na proizvoljan način [16]. Često su za funkciju udaljenosti korištene metrike Euklidske ili Manhattan udaljenosti te duljine najkraćeg puta između zadanih točaka, tj. cestovne udaljenosti u slučaju sustava prometne mreže.

3.1.2. Pokrivenost

Nadograđujući na koncept minimiziranja udaljenosti bilo koje točke do njoj najbližeg objekta dolazimo do koncepta pokrivenosti. Definiramo pokrivenost objekta kao skup svih točaka čija je udaljenost od lokacije objekta manja ili jednaka određenom radijusu:

$$Cov(s, r) = \{x \in X \mid \lambda(s, x) \leq r\} \quad (3.5)$$

gdje je $s \in C$ jedno od potencijalnih lokacija za postavljanje objekta, λ funkcija udaljenosti (izraz 3.1) i $r \in \mathbb{R}^+$ proizvoljno odabrani radijus. Zatim proširujemo funkciju na skup objekata na način da je pokrivenost cijelog skupa jednaka uniji pokrivenosti objekata u skupu

$$Cov(S, r) = \bigcup_{s \in S} Cov(s, r) \quad (3.6)$$

Za sustav kažemo da je potpuno pokriven za određeni radijus ako i samo ako je unija pokrivenosti svih objekata odabranog skupa $S \subseteq C$ jednaka cijelom sustavu

$$Cov(S, r) = X \quad (3.7)$$

Time dolazimo do formulacije nove funkcije cilja za rješenje problema lokacije objekata

$$f = \min_{r \in \mathbb{R}^+} (Cov(S, r) = X) \quad (3.8)$$

što je jednako odabiru skupa objekata na način da se minimizira radijus pokrivenosti skupa. Budući da obrađujemo problem raspodjele stanica za punjenje vozila po prometnoj mreži vraćamo se ograničenju odabira samo k objekata i koristimo cestovnu udaljenost (konkretno definiranu kasnije (1)) za definiciju udaljenosti formule pokrivenosti. Obradom problema na ovaj način dolazimo do rješenja koje nam daje podjednako raspoređene stanice po prometnoj mreži, no ne uzimaju se u obzir težine bridova i dinamički podaci simulacije. U (0) opisana je nadograđena verzija algoritma korištena u radu.

3.1.3. Mješovito cjelobrojno programiranje

Još jedna često korištena metoda je mješovito cjelobrojno programiranje (MIP). MIP je vrsta modela optimizacijskog problema definiranog kroz funkciju cilja i ograničenja koji mogu sadržavati i cjelobrojne i kontinuirane varijable. U slučaju da se koriste jedino cjelobrojne varijable (većinom binarno) problem se naziva samo cjelobrojnim programiranjem, te u slučaju da su funkcija cilja i ograničenja linearni, mješovito cjelobrojno linearno programiranje (MILP), odnosno nelinearni, mješovito cjelobrojno nelinearno programiranje (MINLP). Većina formulacija problema optimizacije raspodjele stanica za punjenje električnih vozila je ili u MILP varijaciji zbog manje složenosti (kao u [5]), ili je dio cilja pojednostaviti problem u MILP (kao u [6]). Jednostavan primjer MIP modela prikazujemo kroz minimiziranje funkcije cilja f

$$\min (f(x, y))$$

uz ograničenje

$$Ax - By > C$$

i definicijom varijabla $x \in \mathbb{R}, y \in \mathbb{Z}$, tako da se vidi da je problem mješoviti zbog postojanja realne varijable x . U slučaju da je također $x \in \mathbb{Z}$ problem bi se pretvorio u cjelobrojno programiranje, dok bi definicijom funkcije cilja f linearnom funkcijom, npr.

$$f(x, y) = x + y$$

definirali jednostavan MILP problem.

Sada ćemo definirati generalnu verziju problema raspodjele stanica u obliku MILP-a; neka su $i \in I$ klijenti koji zadaju potražnju te $j \in J$ potencijalne lokacije stanica. Neka je f_j fiksni trošak održavanja stanice na lokaciji j i c_{ij} profit zadovoljavanja potražnje klijenta i kroz stanicu j . Profit je tipično funkcija troška proizvodnje resursa na stanici, troška transporta klijenta do stanice, potražnje klijenta i prodajne cijene koju klijent plaća. Za kraj definiramo varijable $x \in \{0,1\}$, $y \in [0,1]$ na način da $x_j = 1$ označava da je stanica j izgrađena i funkcionalna, odnosno $x_j = 0$ da ne postoji na toj lokaciji, te $y_{ij} = 1$ znači da je potražnja klijenta i potpuno zadovoljena kroz stanicu j , dok $y_{ij} < 1$ označava nepotpunu zadovoljenost potražnje klijenta. Tada je MILP problem [15]

$$Z = \max \sum_{i \in I} \sum_{j \in J} c_{ij} y_{ij} - \sum_{j \in J} f_j x_j \quad (3.9)$$

$$\sum_{j \in J} y_{ij} = 1 \quad \forall i \in I \quad (3.10)$$

$$y_j \leq x_{ij} \quad \forall i \in I, j \in J \quad (3.11)$$

$$x_j \in \{0,1\}, y_{ij} \geq 0 \quad \forall i \in I, j \in J \quad (3.12)$$

Funkcija cilja (izraz 3.9) matematički modelira maksimizaciju profita uz minimizaciju troškova održavanja stanica. Uvjet (izraz 3.10) modelira potrebu da je sva potražnja zadovoljena, (izraz 3.11) garantira da su klijenti posluženi samo iz postojećih i funkcionalnih stanica te izraz (izraz 3.12) definira domene varijabla [15]. Može se uočiti da je ovo MILP problem zbog pripadnosti varijable y u domeni realnih brojeva, dok bi olakšanje $y_{ij} \in \{0,1\}$ zadani problem pretvorilo u jednostavniji problem cjelobrojnog programiranja.

3.1.4. Teorija igara

U literaturi gdje je fokus na optimiranje raspodjele stanica za punjenje električnih vozila u kontekstu sebičnih i kompetitivnih želja korisnika stanica često se za pokušaj rješenja koristi područje teorije igara. Glavni koncept koji se koristio, kao i u radovima spomenutima u pregledu postojeće literature [11, 12], je Nashova ravnoteža.

Teorija igara potječe iz ekonomske teorije te analizira matematičke modele strateških interakcija sudionika. Uvodi osnovni koncept igre sume nula od dva igrača koja definira matematički model u kojem svaki potez igrača koji pomaže njemu šteti drugom igraču [18]. Koncept se kroz vrijeme nadograđuje u igru sume nula od n igrača te J. F. Nash Jr. u članku [19] dolazi do teorijske obrade od tada takozvane Nashove ravnoteže.

Nash definira unutar igre sume nula od n igrača n -torku strategija (*n-tuple of strategies*) koja sadrži po jednu od ograničenog broja Σ strategija za svakog igrača koju taj igrač odabire za izvršavanje u sljedećem potezu igre:

$$(\sigma_1, \sigma_2, \dots, \sigma_n)$$

Pretpostavlja se da je svaki igrač kompetitivan i racionalan, tj. da će odabrati strategiju kojim maksimizira vlastiti dobitak. Za jednostavnost definiramo funkciju dobitaka koji igrači dobiju nakon izvršavanja određene n -torke strategija $D: \Sigma^n \rightarrow \mathbb{R}$. Nash zatim definira suprotstavljajuću n -torku (*countering n -tuple*) strategija na način da je n -torka suprotstavljajuća nekoj drugoj n -torki ako strategija svakog igrača u toj n -torki donosi najveći dobitak za njezinog igrača protiv $n - 1$ strategija drugih igrača u n -torki kojoj se suprotstavlja. To možemo prikazati koristeći funkciju D na način:

$(\sigma_1^*, \sigma_2^*, \dots, \sigma_n^*)$ suprotstavlja $(\sigma_1, \sigma_2, \dots, \sigma_n)$ ako

$$D(\sigma_1, \dots, \sigma_i^*, \dots, \sigma_n) \geq D(\sigma_1, \dots, \sigma_i, \dots, \sigma_n) \mid \forall \sigma_i^*, i$$

te Nash zatim definira ravnotežu kao n -torku koja se suprotstavlja sama sebi [19].

Uzimamo u obzir problem optimalne raspodjele stanica s fokusom na kompetitivni odabir stanice sa strane racionalnih korisnika kojima je cilj jedino povećati vlastiti dobitak. Kreira se sustav u kojem na odabir preferirane stanice korisnika ne utječe samo udaljenost do stanice i slične vrijednosti, nego i vrijednost čijim se dobitkom za jednog korisnika direktno smanjuje drugi, kao gužva na stanici u slučaju [11]. Zatim se iterativno izvršava odabir preferirane stanice dok se ne dođe do Nashove ravnoteže, tj. dok niti jedan korisnik ne može promijeniti svoj odabir na način da on dobije veći dobitak. U (5.2) opisana je implementacija algoritma jednako baziranog na Nashovoj ravnoteži korištena u radu.

3.1.5. Genetski algoritmi

Genetski algoritmi se generalno koriste za pokušaj rješenja problema optimalne raspodjele stanica za punjenje električnih vozila transformacijom problema u jedno-kriterijski ili višekriterijski optimizacijski problem koji se zatim rješava kroz postojeće ili nove, posebno razvijene genetske algoritme. U poglavlju 2 nabrojani su primjeri razvoja i primjene genetskih algoritama.

Pri analizi problema optimalne raspodjele stanica za punjenje električnih vozila uočavamo da na odabir raspodjele utječe velik broj faktora iz raznih područja. Postoje direktni faktori kao trošak izgradnje i održavanja stanice, lokalna cijena i opskrba električne energije i slično, te indirektni kao udaljenost i vrijeme putovanja vozila, utjecaj stanice na elektrodistribucijsku mrežu, preferencije korisnika, itd. U definiciji optimizacijskog problema imamo slobodu izbora faktora koje želimo uzeti u obzir i odlučivanje o težini

utjecaja na rješenje koju ćemo im dati. Najčešći je slučaj da se težine određuju kroz koeficijente i time se formira jedno-kriterijska funkcija cilja sa težinskim faktorima. Primjer takve funkcije sa faktorima X i Y te pripadajućim koeficijentima α i β :

$$\min/\max (\alpha * X + \beta * Y)$$

Optimizacijom jedno-kriterijskog optimizacijskog problema dobiva se jedno optimalno rješenje, no zbog slobode definiranja težina utjecaja faktora, tj. koeficijenata, rješenje je rijetko korisno te je potrebno ili unaprijed istražiti i informirano odrediti utjecaj faktora ili konfigurirati koeficijente kroz kontinuirano ispitivanje korisnosti rješenja za različite koeficijente. Pokušavajući popraviti opisani nedostatak dolazimo do koncepta višekriterijskog optimizacijskog problema čijom se optimizacijom dolazi do više rješenja, svako optimalno s obzirom na specifični kriterij. Umjesto jedne funkcije cilja definira se više funkcija cilja f_i , tako da svaka uključuje jedan ili više određenih kriterija. Optimizacijom tako zadanog problema dobiva se takozvana Pareto fronta, tj. spomenuti skup relativno optimalnih rješenja.

$$\min/\max ((f_1, \dots, f_n))$$

Genetski algoritmi su jedna od metoda optimizacije ovako zadanih optimizacijskih problema inspirirani konceptima iz prirode, kao algoritam optimizacije rojem čestica koji imitira socijalno ponašanje roja organizama.

3.1.6. Strojno učenje

Tehnike strojnog učenja korištene su za rješavanje raznih razina problema optimalne raspodjele stanice, od makroskopske [13] do mikroskopske [14] razine. Budući da strojno učenje možemo generalizirati u metodu estimacije stvarne distribucije analiziranih podataka iz ograničenog broja uzoraka postoje mnoge tehnike koje obrađuju razne oblike podataka. Zato je korišten i velik raspon različitih tehnika za pokušaj derivacije korisnih podataka u svrhu optimizaciju raspodjele stanica za punjenje, od slučajnih šuma (engl. *Random Forest*) [13] do relativno nedavno razvijenih graf neuronskih mreža (engl. *Graph Neural Network*) [14].

3.2. Model okoline sustava

Modeliramo okolinu sustava kao usmjereni graf $G = (N, E)$ gdje N predstavlja skup čvorova mreže i E predstavlja sve bridove koji ih povezuju. Budući da pokušavamo modelirati stvarne prometne mreže, bridovi su ekvivalentni cestama te čvorovi križanjima zadane stvarne prometne mreže. Također definiramo dodatna svojstva bridova za modeliranje korisnih statistika strukture stvarne prometne mreže:

- 1.) Duljina, d – modelira duljinu ceste, tj. udaljenost koju vozilo mora proći u stvarnoj prometnoj mreži da prijeđe sa jednog kraja brida do drugog
- 2.) Najveća brzina, v – modelira najveću brzinu kojom vozilo smije voziti po cesti u slučaju idealnih uvjeta, tj. prazne ceste
- 3.) Vrijeme putovanja, t – modelira približno vrijeme putovanja kroz formulu $t(e) = l(e)/v(e)$, gdje je $e \in E$

s time da se vrijeme putovanja također može računati na dinamički način tijekom simulacije, uzimajući u obzir trenutno stanje prometa na cesti.

Neka je $\Delta: N \times N \rightarrow \mathbb{R}^+$ funkcija cestovne udaljenosti, gdje je $\Delta(n_1, n_2)$ jednako udaljenosti najkraćeg puta između dva čvora n_1 i n_2 unutar grafa G . Neka je također $\Theta: N \times N \rightarrow \mathbb{R}^+$ funkcija vremena putovanja, gdje je $\Theta(n_1, n_2)$ jednako vremenu potrebno vozilu za prelazak puta od križanja i do križanja j uzimajući u obzir trenutno stanje prometne mreže. Obje vrijednosti su trivijalne za izračun kroz iterativno zbrajanje udaljenosti/vremena putovanja svake ceste najkraćeg puta između zadanih čvorova:

$$\Delta(n_1, n_2) = \sum_{p \in P} d(p) \quad (3.13)$$

$$\Theta(n_1, n_2) = \sum_{p \in P} t(p) \quad (3.14)$$

gdje je $P \subset E$ podskup bridova koji čine najkraći put od čvora n_1 do čvora n_2 , te l i t maloprije definirana dodatna svojstva duljine i vremena putovanja bridova.

Definiramo pomoćnu funkcije susjedstva čvora $\eta: N \rightarrow 2^N$ i susjedstva brida grafa $\sigma: E \rightarrow 2^E$, pri čemu za bridove $e, f \in E$ vrijedi $e = (x, y)$ i $f = (w, z)$:

$$\eta(n) = \{u \in N \mid (n, u) \in E\} \quad (3.15)$$

$$\sigma(e) = \{f \in E \mid \{x, y\} \cap \{w, z\} \neq \emptyset\} \quad (3.16)$$

Funkcija poprima vrijednost 1 u slučaju da bridovi dijele zajednički čvor. Također definiramo pomoćnu funkciju incidencije bridova $\iota: V \rightarrow 2^E$ pri čemu za svaki čvor $n \in N$ vrijedi:

$$\iota(n) = \{(x, y) \in E \mid x = n \vee y = n\} \quad (3.17)$$

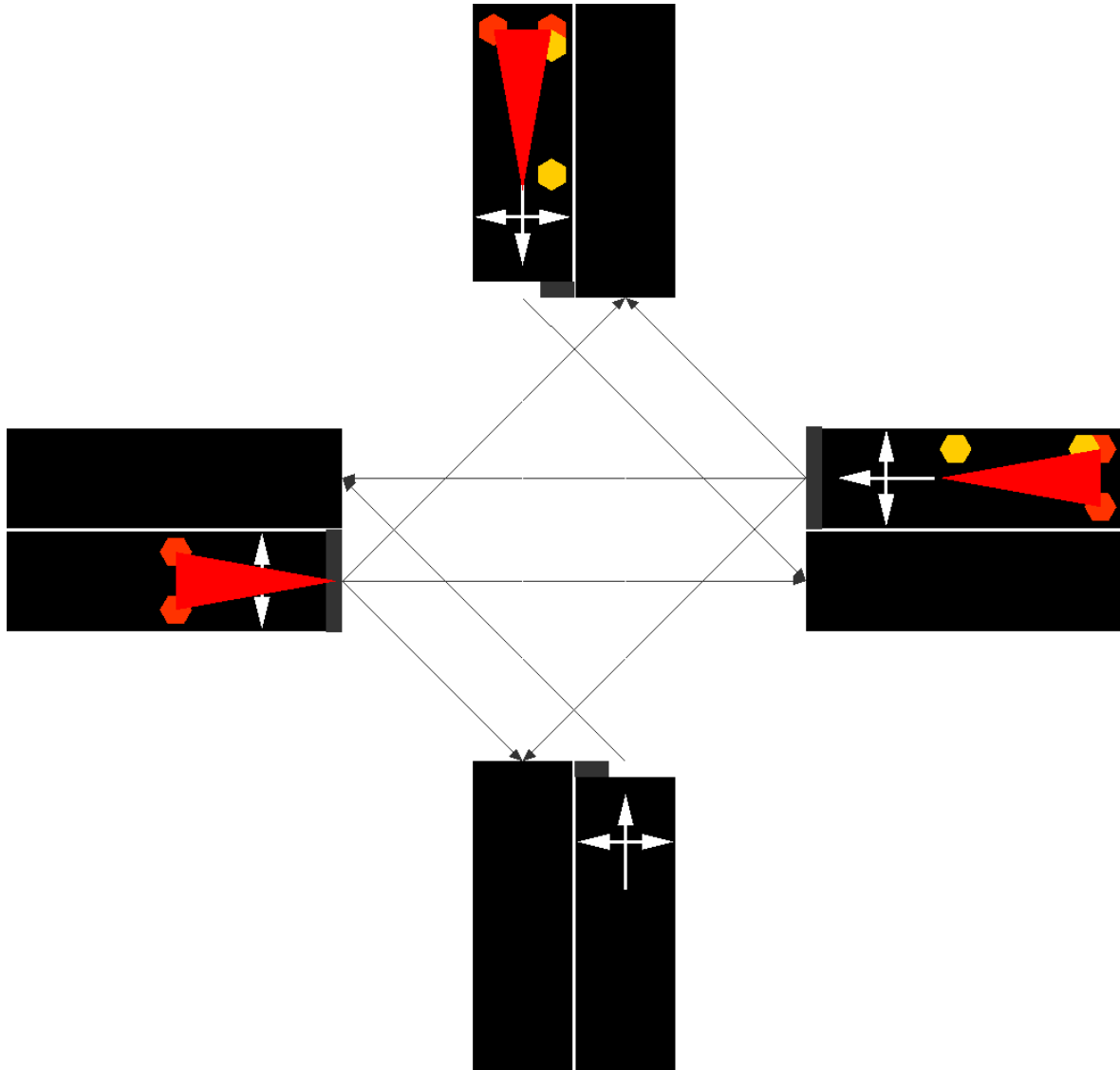
Funkcija vraća skup bridova koji povezuju čvor n s nekim drugim čvorom, tj. koji su incidentni čvoru n . Budući da je cilj zadatka odabir bridova za optimalnu raspodjelu stanica za kraj definiramo skup $C \subseteq E$ koji čine svi bridovi kandidati za postavljanje i povezivanje mreže sa stanicom za punjenje. U slučaju da rješavamo prvu razinu problema raspodjele stanica skup C će idealno biti pravi podskup čime se ne uzimaju svi bridovi kao potencijalna mjesta za stanice i time uvelike smanjuje kombinatornu složenost druge razine problema.

3.2.1. Model sa troškovima skretanja

Definiramo graf sa troškovima skretanja $G = (N, E)$ na jednak način, uključujući dodatna svojstva bridova. Dodajemo dodatna svojstva čvorovima:

- 1) Duljina skretanja, D – modelira udaljenost koju vozilo mora prijeći da dođe s trenutne ceste na sljedeću; to je funkcija dvaju brida čvora $D: N \times \varepsilon \times \varepsilon \rightarrow \mathbb{R}^+$ gdje je $\varepsilon \subset E$ podskup bridova koji su spojeni s čvorom
- 2) Najveća brzina skretanja, V – modelira najveću moguću brzinu kojom je moguće izvršiti skretanje; također funkcija $V: N \times \varepsilon \times \varepsilon \rightarrow \mathbb{R}^+$ gdje je $\varepsilon \subset E$ podskup bridova koji su spojeni sa čvorom
- 3) Vrijeme skretanje, T – modelira približno vrijeme potrebno za dovršetak skretanja, izračunava se jednako ekvivalentnom svojstvu brida $T(n, e_1, e_2) = L(n, e_1, e_2) / v(n, e_1, e_2)$ gdje je $n \in N$ i $e_1, e_2 \in E$.

U ovom slučaju se vrijeme skretanja može dinamično računati isto kao vrijeme putovanja u slučaju svojstva čvorova, također uzimajući u obzir trenutno stanje prometa na križanju. Može se uočiti da ovakvo dinamično modeliranje vremena zadržavanja vozila unutar raskrižja može imati veliki utjecaj u slučaju dugog čekanja vozila za lijevo skretanje. Opisana svojstva vizualiziraju se kao ekstenzija grafa u obliku „unutarnjih bridova“ čvorova (Slika 3.1), kojima bi se onda ta svojstva primijenila slično kao u slučaju glavnih bridova grafa.



Slika 3.1 Prikaz povezanosti prometnih traka unutarnjim bridovima u grafičkom sučelju *SUMO* simulatora. Težine unutarnjih brida definiraju troškove skretanja križanja.

Sada proširujemo formule za izračun funkcija cestovne udaljenosti, $\delta: N \times N \rightarrow \mathbb{R}$, i vremena putovanja između čvorova, $\theta: N \times N \rightarrow \mathbb{R}$:

$$\delta(n_1, n_2) = \Delta(n_1, n_2) + \sum_{i=2}^{|P|-1} D(p_i, (p_{i-1}, p_i), (p_i, p_{i+1})) \quad (3.18)$$

$$\theta(n_1, n_2) = \Theta(n_1, n_2) + \sum_{i=2}^{|P|-1} T(p_i, (p_{i-1}, p_i), (p_i, p_{i+1})) \quad (3.19)$$

gdje (p_{i-1}, p_i) predstavlja ulazni i (p_i, p_{i+1}) izlazni brid na čvor $p \in P$ podskupa $P \subset N$ koji predstavlja najkraći put između čvorova n_1 i n_2 . Time se u izračunu traženih vrijednosti uzima u obzir i novodefinirani trošak skretanja na svakom čvoru.

3.2.2. Linijski graf

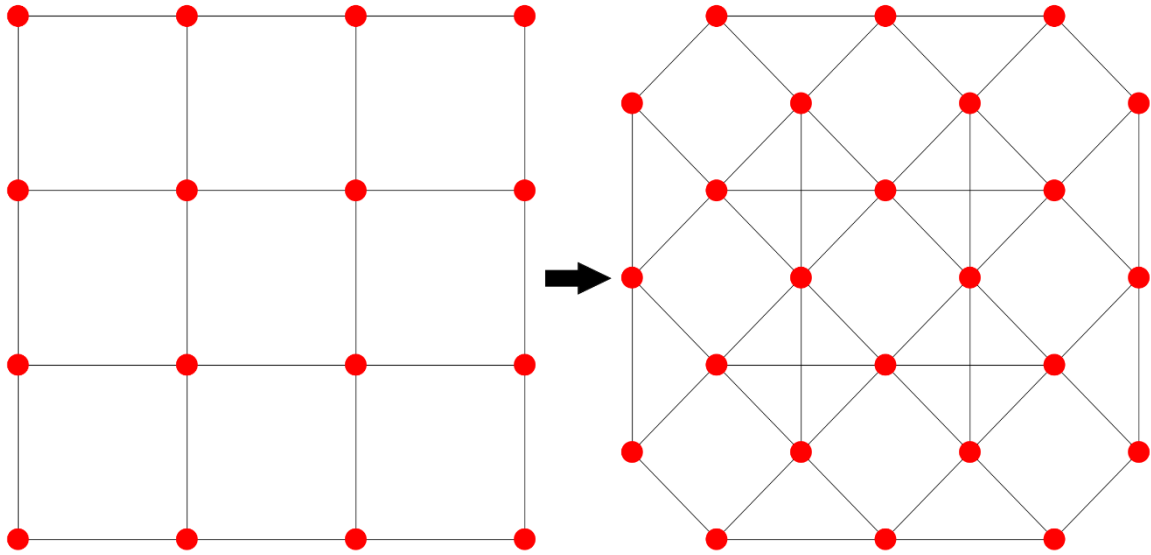
Alternativna definicija grafa sustava kojim možemo ostvariti jednaki cilj korištenja bridova za potencijalna mjesta postavljanja stanica je linijski graf. Neka je $G = (N, E)$ jednak već definiranom grafu, onda je $L(G) = (N', E')$ linijski graf gdje je

$$N' = E$$

i

$$E' = \{(e_1, e_2) | e_1, e_2 \in E, e_1 \neq e_2, e_2 \in \sigma(e_1)\}$$

pri čemu je σ funkcija susjedstva bridova (izraz 3.12). Time je linijski graf zapravo inverzni graf u kojem čvorovi i bridovi „zamjene mjesta“ (Slika 3.2). Transformacijom grafa u kojem su bridovi potencijalna mjesta za stanice u linijski graf potencijalna mjesta sada postaju čvorovi, te se na graf mogu primijeniti jednostavnije i bolje istražene varijacije raznih funkcija. No transformacijom u linijski graf svojstva čvorova i bridova bi jednako skupovima zamijenili uloge. Time se svojstvo duljine ceste apstrahira u vrijeme čekanja vozila prije ponovnog kretanja nakon dolaska do čvora. No svojstva križanja postaju jednaka težini ili duljini brida čime se znatno pojednostavljaju za modeliranje.



Slika 3.2 Prikaz transformacije grafa 4x4 rešetke u linijski graf. U ovom slučaju je broj čvorova povećan s 16 na 24 i broj bridova sa 24 na 52, čime je ovo primjer loše primjene transformacije.

Metoda linijskog grafa je korisna za razna kalkulacijska rješenja, no budući da je fokus rada na simulacije uglavnom će se koristiti klasična formulacija grafa budući da linijski graf nije kompatibilan sa simulacijskim alatima. Važno je spomenuti i svojstvo linijskog grafa da uvijek vrijedi $|N'| = |E|$ što može imati velik utjecaj na računske operacije čija kompleksnost ovisi većinom o broju čvorova. Primjer toga je *Floyd-Warshall* algoritam čija je prosječna kompleksnost $\Theta(|V|^3)$, tj. ovisi samo o broju čvorova. Stoga ako graf koji se sastoji od puno većeg broja čvorova nego bridova, $|N| \gg |E|$, transformiramo u linijski graf broj čvorova postaje puno manji od broja bridova, $|N'| = |E| \ll |N|$, te se algoritam brže izvršava. U određenim slučajevima modeliranja ruralnih područja ili neoptimalnog modeliranja prometne mreže na način da su preostali čvorovi jediničnog ranga ovo svojstvo može imati veliki utjecaj. No u isto vrijeme za guste prometne mreže najčešće vrijedi $|E| \gg |N|$, što je još jedan razlog zašto se ne koristi ova metoda u sklopu rada.

3.2.3. Graf proširen incidentnim čvorovima

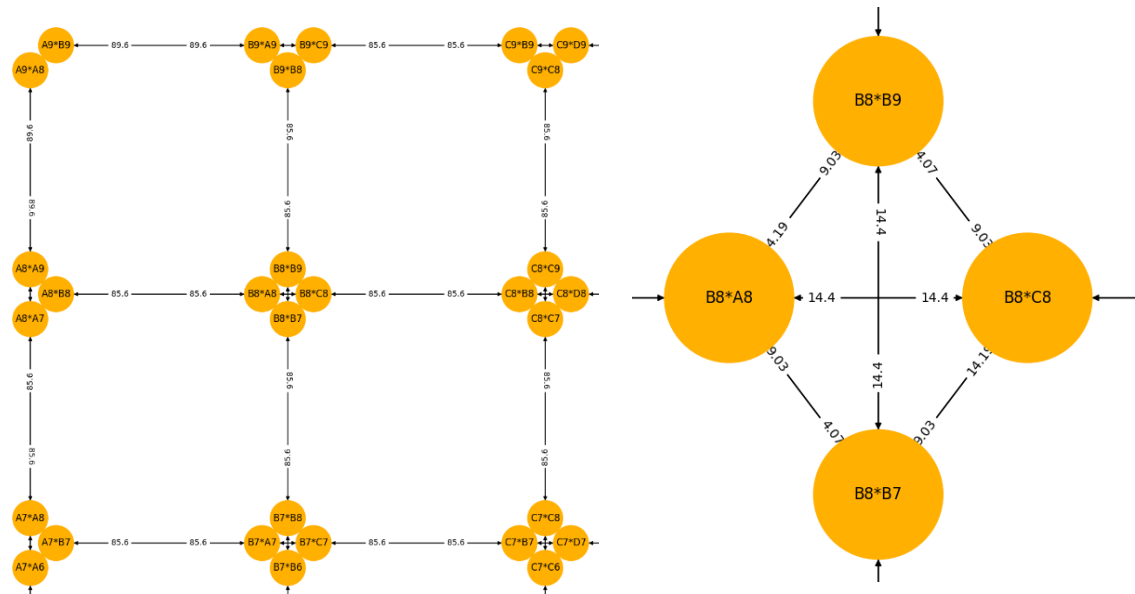
Unatoč tome definirat ćemo graf proširen incidentnim čvorovima kao hibrid linijskog i klasičnog grafa. Graf proširen incidentnim čvorovima usmjereni je graf $G = (N, E)$ dobiven transformacijom $H(G) = (N', E')$ gdje je N' novi skup čvorova te $E' = E_G \cup I$:

$$N' = \{(n, u) \mid n, u \in N, u \in \eta(n)\}$$

$$E_G = \{((n, u), (u, n)) \mid n, u \in N, \eta(n, u) = 1\}$$

$$I = \bigcup_{n \in N} \{(i_1, i_2) \mid i_1, i_2 \in \iota(n), i_1 \neq i_2\}$$

te su η i ι pomoćne funkcije susjedstva čvorova (izraz 3.15) i incidencije bridova (izraz 3.17). Transformacijom zapravo svaki čvor razdijelimo na broj čvorova jednak broju bridova incidentnih čvoru. Svaki od novih čvorova označimo (n, u) na način da je n originalni čvor, a u čvor na koji se spaja. Zatim se novostvoreni čvorovi povežu „vanjskim“ bridovima, tj. ekvivalentnima originalnim bridovima E , na način da se spoji svaki par (n, u) , (u, n) ako su čvorovi n i u bili povezani u originalnom grafu G . Na kraju povežemo graf „iznutra“, sve čvorove (n, n_i) nastale iz istog originalnog čvora, tj. $\eta(n, n_i) = 1$, povežemo novim bridovima.



Slika 3.3 Prikaz strukture grafa proširenog incidentnim čvorovima na rešetkastoj mreži. Lijeva slika prikazuje širu strukturu i „vanjske“ bridove ekvivalentne bridovima originalne mreže, te održavanje svojstva duljine ceste kao težine brida. Desna slika detaljnije prikazuje rastav čvora na incidentne čvorove te definira način njihovog označavanja te transformaciju svojstva duljine skretanja unutar križanja u težine novonastalih „unutarnjih“ bridova.

Sada se može lagano uočiti da ovakav graf ima korisno svojstvo pojednostavljenja modeliranja svojstva križanja tako da postaju jednaki težinama ili duljinama bridova (Slika 3.3), kao i u linijskom grafu. To također čini mnoge postojeće metode bazirane na težinskim grafovima primjenjivije na graf, dok u linijskom grafu ostaje problem svojstva duljine ceste unutar čvorova. No znatni nedostatak ovakve strukture grafa je da je broj čvorova i bridova

puno veći te su time kalkulacije na grafu proširenim incidentnim čvorovima utjecajno dugotrajnije. U računskim operacijama se uglavnom koristila kombinacija originalno definiranog grafa u kontekstu simulacije, te grafa proširenim incidentnim čvorovima u kontekstu računanja.

3.3. Stanice

3.3.1. Alokacija vozila na stanice

Alokacija vozila na stanice za punjenje zaseban je problem optimizacije po sebi, no za potrebe simulacije mora se odlučiti na određeno pravilo ili metodu alokacije. Budući da rad analizira mješoviti autonomni promet metode se generalno mogu podijeliti na dvije razine:

- Sebični odabir
- Centralizirani sustav alokacije vozila

Sebični odabir modelira konvencionalno upravljanje vozilom od strane ljudskog vozača ili operatera. Vozač nema informacije o samim stanicama osim njihovih lokacija, time i cestovnu udaljenost do svake stanice. Stoga vozila u sklopu politike sebičnog odabira uzimaju u obzir samo cestovnu udaljenost do svake od stanica i odlučuju po tome koja im je najbliža. Također se modelira utjecaj znanja o potrebnom vremenu putovanja do stanice kao dodatni, dinamički utjecaj na sebični odabir, koji modelira autonomno vozilo koje sadrži aktualne informacije o prometu no i dalje odlučuje sebično.

Nasuprot tome politika centraliziranog sustava alokacije vozila na stanice modelira potpunu sustavnu kontrolu nad prometom autonomnih vozila. Vozilo se pri detekciji potrebe za punjenjem javlja sustavu koji zatim odlučuje o odabiru stanice uzimajući u obzir i faktore cestovne udaljenosti te dinamički izračunatog vremena putovanja, uz znanje o trenutnim stanjima na stanicama. Time sustav preferira jednoliku raspodjelu vozila po svim stanicama, čak i u slučajevima kada bi put vozila bio duži zbog obilaska do stanice ako je najbliža stanica zauzeta. Hibridna politika miješanja politike sebičnog odabira i politike centraliziranog sustava nije analizirana u ovom radu.

Dosad su postignuti razni uspjesi u analizi i konvencionalnog i autonomnog prometa te se još duboko istražuju, te isto vrijedi za slučaj mješovito autonomnog prometa. No uglavnom su samo analizirane simulacije gdje vozila sebično odabiru stanicu za punjenje nakon što

centralizirani sustav optimalno postavi stanice za punjene [9, 11], zbog čega se u radu analizira i opisani sustav centralizirane raspodjele vozila nakon odlučivanja o lokacijama stanica.

3.3.2. Sustav raspodjele stanica

U dosadašnjoj literaturi optimalne raspodjele stanica većinom se uzima u obzir model jednog sustava koji analizom prometne mreže i relevantnih podataka samo odlučuje o mjestima raspodjele stanica, kao [5, 7, 10]. To je vrsta jedno-agentskog sustava u kojem agent rješava problem tretirajući ga kao statični optimizacijski problem.

Sada dolazimo do ideje suprotne jedno-agentskom statičnom sustavu, a to je više-agentski dinamični sustav. Postojeća literatura već se bavi idejom optimizacije raspodjele stanica u dinamičnom sustavu, većinom definirajući dinamični sustav tretirajući korisnike stanica kao zasebne dinamične elemente, kao u [9, 11]. To je onda kompetitivan sustav na razini korisnika stanica, gdje je svaki od korisnika definiran funkcijom korisnosti koja određuje želju korisnika za služenjem određene stanice ovisno o definiranim faktorima stanice. Najčešći kriteriji određivanja korisnosti su udaljenost do stanice i broj korisnika koji se trenutno služi stanicom, no ponekad je uključen i kriterij individualne sklonosti stanici, kao brend stanice ili lokalne atrakcije u blizini stanice u [11]. No rijetki su radovi u kojima je sustav kompetitivan ne samo na razini korisnika nego i na razini samog sustava raspodjele stanica.

Kada razmislimo o implementaciji sustava u kontekstu stvarnog svijeta, praktički je nemoguće staviti sav prostor i sva vozila cijelog planeta pod jedan sustav. Umjesto toga bi svaki sustav imao zadanu regiju za raspodjelu stanica. Ako uzmemo u obzir idealni slučaj optimalnog algoritma raspodjele stanica, svaki sustav bi optimalno postavio stanice samo unutar vlastitog područja, čime je logično zaključiti da će biti slučaj kada će rasporedi stanica u prostoru više susjednih regija zajedno biti neoptimalno raspoređeni. Šira optimalnost sustava raspodjele za određenu regiju može se onda ostvariti dodavanjem kooperativnog elementa algoritmu koji uzima u obzir i sustave okolnih regija te njihovu raspodjelu stanica. Postavljanjem dva takva sustava u dvije susjedne regije zapravo nastaje dvo-agentski kooperativni sustav, te se također dijelovi njima zadanih regija mogu poklapati.

No čemu onda dodati element kompetitivnosti? Ako ponovno pogledamo na kontekst stvarnog svijeta možemo vidjeti da su danas korištene stanice za punjenje vozila, tj. benzinske postaje, pod vlasništvom više različitih kompanija. Zatim ako također uzmemo u obzir cijenu punjenja dolazimo do već postojećeg ekonomskog kompetitivnog elementa raspodjele stanica. Baš zbog tog ekonomskog elementa je prirodno došlo do kompetitivnosti postavljanja benzinskih stanica, te su koristi uvođenja ekonomska kompetitivnosti u raspodjeli dobara već duboko istražene: zaključak je da kompetitivnost poboljšava alokaciju i dovodi do smanjenja cijena resursa [17], što bi bilo idealno i u slučaju raspodjele stanica za punjenje električnih vozila. Osim toga je iz optimizacijske perspektive već poznato da se prilikom optimiziranja višekriterijskog problema umjesto jednog rješenja stvara *Pareto* fronta više rješenja gdje svako rješenje preferira bolju vrijednost pojedinih kriterija. Time bi se kroz dodatan element kompetitivnosti sukoba agenata moglo filtrirati rješenja u kojima je cijena punjenja velika, budući da sama cijena nije kriterij uključen u globalnu funkciju cilja. Time dolazimo do glavne ideje kooperativno kompetitivnog sustava više agenata kojima je cilj u isto vrijeme globalno optimalno raspodijeliti stanice te također izvući što više vlastitog profita što očekujemo da rezultira manjim cijenama punjenja.

Definiramo više-agentski dinamični sustav $S = (A, V, G)$ koji se sastoji od skupa više agenata, $A = \{a_1, a_2, \dots, a_m\}, m > 1$, skupa vozila koji su korisnici stanica V i graf G koji je okolina cijelog sustava. Agenti imaju zasebne funkcije cilja f_a , te je definirana globalna funkcija cilja f_g koja uključuje sveukupni odabir stanica svih agenata i prikazuje uspješnost nastale raspodjele. Pod-funkcija cilja f_a^{seb} agenta a onda definira sebično i kompetitivno ponašanje agenta, dok pod-funkcija cilja f_a^{soc} definira socijalno i kooperativno ponašanje agenta. Cijela funkcija cilja f_a je onda

$$f_a = f_a^{seb} * \alpha + f_a^{soc} * \beta$$

gdje su α i β koeficijenti koji potiču sebično i socijalno ponašanje, redom. U slučaju da je $\alpha = 0$ dobivamo potpuno kooperativni slučaj, te ako je također $f_a^{soc} = f_g$ dobivamo slučaj sličan originalnom sustavu jednog agenta jer se optimizira samo globalna funkcija cilja, no kroz kooperativnost i podjelu odabira na više agenata. Odnosno, ako je $\beta = 0$ dolazimo do potpuno kompetitivnog slučaja igre sume nula budući da se posluživanjem vozila na stanici jednog agenta direktno uzima dolazak i posluživanje tog vozila na stanici drugog agenta.

Glavni cilj optimizacije problema raspodjela stanica za punjenje ipak je globalni uspjeh, tj. globalna funkcija cilja f_g . Zbog toga definiramo socijalnu pod-funkciju cilja agenta f_a^{soc} da je jednaka globalnoj funkciji cilja f_g

$$f_a^{soc} = f_g$$

Time na kraju u funkciju cilja agenta uključujemo originalnu funkciju cilja koju optimiziramo uz dodatni element kompetitivnosti sebične pod-funkcije cilja. No budući da je sekundarni globalni cilj smanjenje cijena kroz kompetitivni sukob agenata sami kriterij cijene punjenja neće biti uključen u globalnu funkciju cilja te će biti uključen u sebičnu funkciju agenata

$$f_a^{soc} = \dots + 0 * profit + \dots$$

$$f_a^{seb} = \dots + \alpha_{profit} * profit + \dots$$

gdje je α_{profit} koeficijent utjecaja kriterija cijene na sebičnu pod-funkciju cilja agenta, te je analogni koeficijent u socijalnoj funkciji β_{profit} jednak nuli. Time će cjelokupni sustav optimalno postaviti stanicu u zadanoj regiji te se očekuje da će također doći do rješenja u kojem su cijene minimalne.

4. Implementacija

U svrhu izvršavanja zadatka usporedne analize simulacija raspodjele stanica za punjenje električnih vozila u slučaju raspodjele kroz centralizirani sustav i kompetitivne raspodjele više agenata u mješovitom autonomnom prometu razvijeno je implementacijsko rješenje za potrebnu okolinu. Odabrani su alati za simulaciju i simulacijsko okruženje te su razvijeni alati za mogućnost njegove modifikacije. Alatima se podešavaju postavke prometa simulacije te se modificira sama prometna mreža dodavanjem i micanjem stanica za punjenje koje su ostvarene kroz specifično definiranu podmrežu. Zatim su implementirani algoritmi raspodjele te je razvijeno okruženje za izvršavanje implementiranih algoritama, uz alate za analizu, prikaz i pohranu rezultata. Potom je implementirana kompatibilnost za korištenje prometnih mreža generiranih od stvarnih podataka, konkretno podataka iz *OpenStreetMap* baze podataka.

Ovo poglavlje nabraja i opisuje vanjske alate korištene u razvijenoj implementaciji rješenja te dodatno razvijene alate i modifikacije razvijene u sklopu radu, uz objašnjenja odabira izvršenih načina implementacija.

4.1. Alati

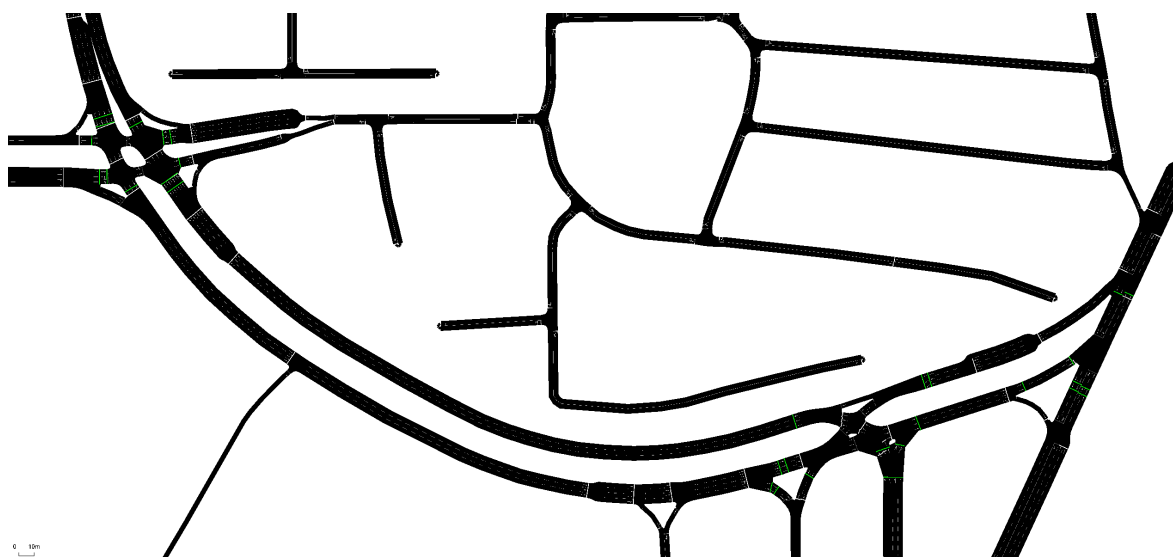
4.1.1. *Eclipse SUMO*

Eclipse SUMO je simulator i programski paket otvorenog koda za simulaciju prometa. Omogućuje modeliranje prometne okoline i prometa na četiri razine [23]:

- Makroskopski model – simulira prosječne karakteristike prometnog toka, poput gustoće prometa
- Mikroskopski model – omogućuje modeliranje svakog individualnog vozila i njegovo ponašanje
- Mezoskopski model – mješavina makroskopskog i mikroskopskog modela
- Submikroskopski model – modeliranje svakog vozila uz detalje njegovog funkcioniranja, npr. mijenjanje brzina

U ovom radu korišten je mikroskopski model zbog potrebe za definiranjem ponašanja svakog pojedinačnog vozila koje može biti konvencionalno sa vozačem ili autonomno. Model također omogućuje definiranje načina alokacije vozila na stanice, što je potrebno za definiranje algoritma alokacije vozila pomoću centraliziranog, sveznajućeg sustava.

Jedan od alata, *netconvert*, također je korišten za pojednostavljenje i standardizaciju prometnih mreža korištenih u simulacijama. U slučaju ručnog definiranja mreža mogu se definirati čvorovi, bridovi i pozicije čvorova te zatim kreirati funkcionalnu prometnu mrežu pomoću tog alata, što dopušta veliku slobodu definiranja okoline bez dodatnih komplikacija pri simulaciji. U slučaju uvoza vanjskih podataka, kao iz *OpenStreetMap* baze podataka, moguće je pojednostaviti podatke u mrežu čvorova i bridova te pomoću alata generirati funkcionalnu mrežu za simulaciju, čime mičemo dodatne elemente kao autobuse i željezničke pruge u svrhu bržih simulacija te zato što nemaju toliki utjecaj na razliku između različitih raspodjela stanica. Također se uvozom podataka direktno kroz alat mogu sačuvati geometrijski podaci prometnih pravaca, kao zakrivljenost cesta, iako je to uglavnom samo vizualni element jer se udaljenosti ne mijenjaju.



Slika 4.1 Prikaz dijela jednostavne prometne mreže generirane iz podataka grada Zagreba preuzetih sa OpenStreetMap baze podataka

Elementi mreže su unutar *SUMO* simulacije podijeljeni na sljedeći način: bridovi (*edge*) su cestovni putovi (*road*) koji sadrže više prometnih traka (*lane*) te se spajaju u čvorovima koji reprezentiraju križanja (*junction*). Između povezanih bridova definirane su veze (*connections*) koje definiraju način na koji vozila mogu prijeći iz jednog brida u drugi dok

putuju. Svaki čvor sadrži unutarnje bridove (*internal edge*) koji su geometrijska reprezentacija veze između bridova istog križanja, te definiraju putanje skretanja vozila unutar križanja tijekom prijelaza sa trenutnog brida na sljedeći [23].

4.1.1.1 TracI

TracI („*Traffic Control Interface*“) je sučelje za *Eclipse SUMO* simulator koje omogućuje direktno upravljanje simulacijom, na već spomenutim razinama kontrole. Također omogućava direktni dohvat, obradu i izdvajanje detaljnijih podataka u svakom koraku simulacije [23].

U ovom radu je i pri simuliranju bez potrebe vizualizacije korištena brža verzija sučelja, *libsumo*, oboje u *Python* jeziku. Razlika između te dvije verzije sučelja je da *TracI* koristi klijent-poslužitelj arhitekturu interakcije sa simulacijom kroz TCP protokol, što je nepotrebno sporo zbog slanja i procesiranja poruka [24]. U implementaciji se kroz sučelje ostvaruje definiranje ponašanja vozila i algoritma alokacije po stanicama tijekom svakog koraka simulacije.

4.1.1.2 Stationfinder

Stationfinder je komponenta mikroskopske simulacije instance vozila koja simulira automatsko praćenje stanja baterije električnog vozila te preusmjeravanje na najbližu i prosječno najmanje zauzetu stanicu. Tijekom simulacije komponenta prati stanje baterije i u slučaju pada energije ispod *needToChargeLevel* vrijednosti preusmjeruje vozilo na stanicu za punjenje. No u slučaju da je uključen *checkEnergyForRoute* parametar komponenta prije odluke izračunava energiju potrebnu za dovršetak puta te ne šalje vozilo na punjenje u slučaju da ima više nego dovoljno energije za put. Pri odabiru stanice u odabir uključuje samo stanice unutar *radius* metara, te u slučaju da stanica nije pronađena nastavlja voziti. Komponenta također obrađuje situaciju gdje vozilo ostane bez električne energije detekcijom pada energije vozila ispod *emptyThreshold* vrijednosti i procesira vozilo na način definiran u *rescueAction*, čime može maknuti vozilo iz simulacije, pomaknuti ga na najbližu stanicu ili pustiti da nastavlja s vožnjom. Stanicu bira po funkciji cilja koja može uključivati informacije o [25]:

- Stanici: broj mjesta za punjenje, broj slobodnih mjesta (apsolutno ili relativno), očekivano vrijeme čekanja, očekivano vrijeme punjenja

- Prometnoj mreži: udaljenost do stanice, udaljenost od stanice do odredišta
- Približnom vremenu putovanja: do stanice, od stanice do odredišta

te su u implementaciji rada ostavljeni standardni parametri funkcije cilja, što znači da se podjednako koriste informacije o očekivanom vremenu putovanja do stanice i od stanice do odredišta te očekivano vrijeme punjenja i čekanja na stanici [25]. Komponenta je originalno u radu korištena za simulaciju ponašanja „ego-vozila“, tj. autonomnog vozila koje ne komunicira sa okolinom, no kasnije je zamijenjena ručnom implementacijom koja je opisana u (3.3.1).

4.1.2. NetworkX

NetworkX je paket za programski jezik *Python* za stvaranje, obradu i proučavanje struktura, dinamike i funkcija kompleksnih mreža. Omogućuje kreiranje različitih mrežnih struktura i modela uz pristup rasponu korisnih funkcija za analizu kreiranih mreža, sa minimalnim utjecajem na performanse. Također omogućuje dodavanje meta-podataka u mrežu u bilo kojem obliku [26].

Pomoću ovog paketa kreirana je kopija učitane prometne mreže kompatibilne sa *SUMO* simulacijama u svrhu detaljnije analize mreže i izračunavanje podataka iz mreže bez potrebe za pokretanjem simulacije. Konkretno se koristi u algoritamskom rješenju raspodjele stanica za izračunavanje udaljenosti, vremena putovanja i pokrivenosti mreže. Pri kreaciji kopije mreže učitavaju se i unutarnji bridovi križanja zadane *SUMO* mreže tako da su udaljenosti i vrijeme putovanja unutar križanja uključeni u analizi mreže.

Također je korištena inačica ovog paketa, *OSMnx*, koja spaja *NetworkX* paket sa uvozom podataka prometnih mreža iz *OpenStreetMap* baze podataka. Nadodaje sposobnost ispravljanja preuzetih podataka i biranja između željenog tipa prometa, npr. automobilski ili biciklistički, koje zatim direktno pretvara u potrebnu strukturu i omogućava korištenje *NetworkX* funkcija na učitanim podacima [27].

4.1.3. OpenStreetMap

OpenStreetMap projekt je kolektiv znanja koji otvoreno pruža geografske podatke prikupljene kroz doprinose javnosti. Korisnici prikupljaju, uređuju i dijele geografske podatke koristeći GPS uređaje, zračne i satelitske slike te druge dostupne izvore. Napravljen

je po uzoru na *Wikipedia* projekt, sa ciljem sakupljanja te slobodnog pristupa i korištenja geografskih podataka na isti način kao u slučaju *Wikipedije*, zbog čega podaci nisu ograničeni strogim autorskim pravima [28]. U sklopu projekta implementirano je i online sučelje za pregled, pristup i izmjenu podataka kolektiva [28]. Također su razvijena razna sučelja za postojeće aplikacije za uvoz i procesiranje podataka projekta unutar aplikacija, kao u slučaju *Eclipse SUMO* programskog paketa koji se koristio u sklopu rada. Preuzeti su i procesirani podaci manjeg dijela Zagreba za ispitivanje efektivnosti razvijenih rješenja na prometnoj mreži generiranoj iz stvarnog svijeta (Slika 4.2).

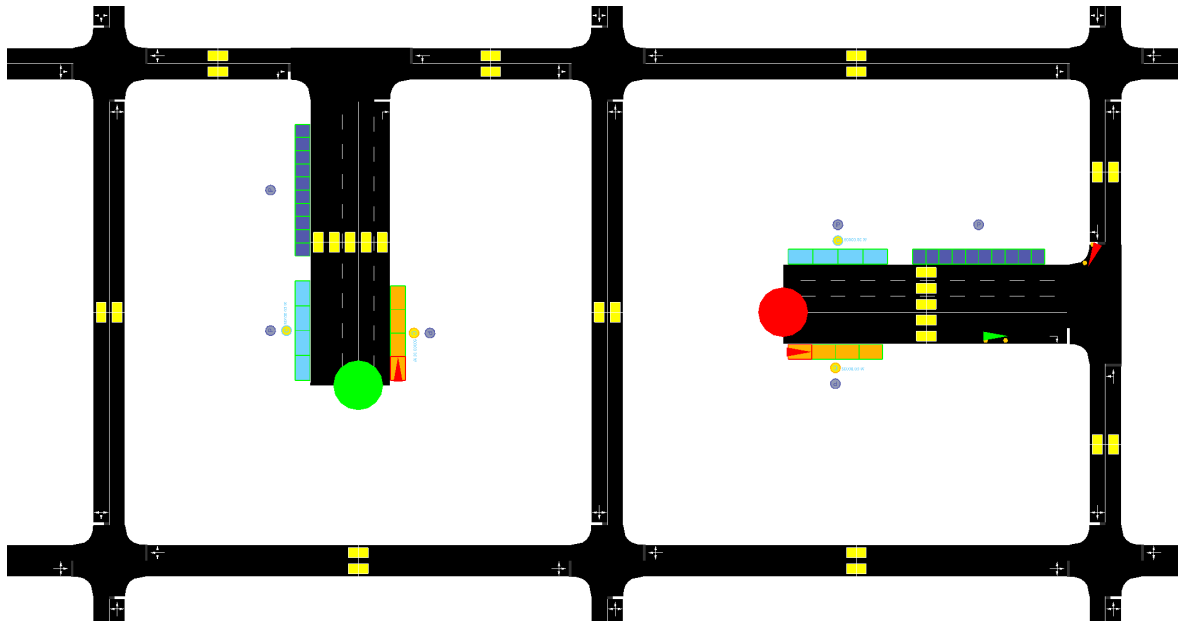
Važno je napomenuti da projekt sadrži velik broj različitih vrsta podataka, te prometna mreža generirana u sklopu rada primjenjuje uglavnom samo cestovne podatke i podatke prometne infrastrukture, dok ignorira ostale. No neki od tih podataka bi teoretski imali utjecaj na simulaciju prometa u mreži, kao autobusne stanice, biciklističke trake i željezničke pruge u slučaju simuliranja i ostalog prometa uz cestovni. No budući da raspon rada nije toliko detaljan te se fokusiran na samo cestovni promet, ti podaci su odbačeni pri procesiranju podataka.



Slika 4.2 Vizualizacija prostora unutar *OpenStreetMap* sučelja iz kojeg su preuzeti podaci za generiranje prometne mreže prikazane u (Slika 4.1) [29]

4.2. Pod-mreža stanice za punjenje vozila

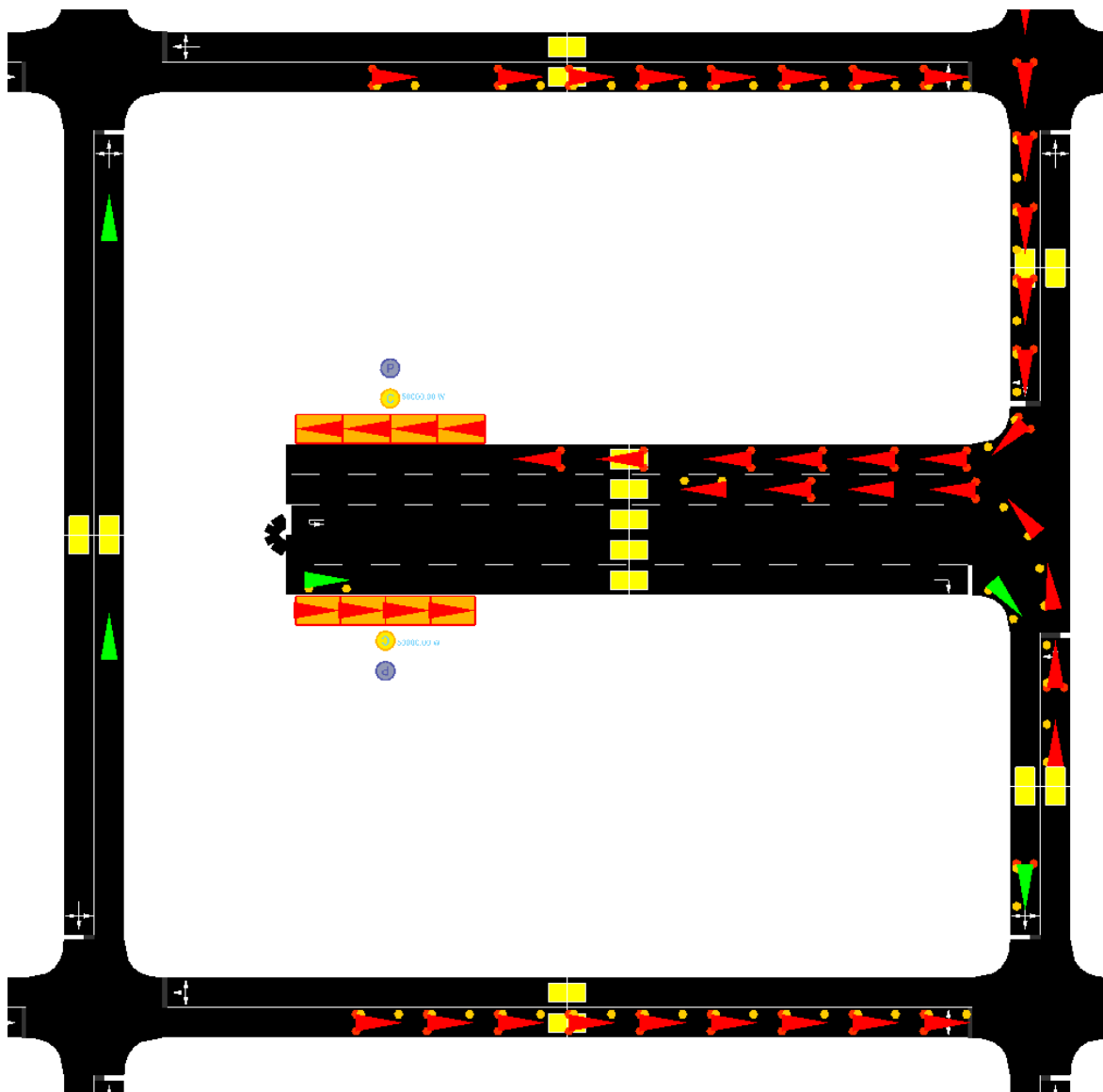
Za ostvarivanje zadatka rada potrebna je modifikacija mreže na način da se na proizvoljnim mjestima može nadodati stanica za punjenje električnih vozila. Budući da su za potencijalne lokacije stanica odabrani bridovi prometne mreže (poglavlje 3), stanica se mora moći spojiti na zadani brid. Stanica je definirana kao standardizirana pod-mreža koja koristi kombinaciju elemenata mjesta za punjenje i parkirnih mjesta, uz opcionalni parking za vozila koja čekaju. Pod-mreža stanice spaja se na brid postojeće mreže na način da se kreira novi čvor u sredini brida (interno nazvan *pcsEntry*) koji se zatim spaja s oba čvora originalnog brida. Zatim se novonastali čvor spaja s još jednim novokreiranim čvorom (nazvanim *pcsEnd*), pomaknutim na stranu okomito od smjera originalnog brida, u novi brid koji će se referencirati kao glavni brid stanice. Definira se samo desno skretanje za izlazak iz glavnog brida stanice natrag u prometnu mrežu, jer u suprotnom može doći do potpunog zastoja (*deadlock*) pri ogromnoj potražnji. Zatim se kreiraju po dva parkinga (*parkingArea*) i dvije punionice (*chargingStation*) povezane svaka sa svojim parkingom na svakoj strani brida stanice (Slika 4.3). Kapacitet stanice i parkinga za čekanje može se proizvoljno definirati te su mjesta podijeljena na dvije strane na način da ulazna punionica dobiva veći broj mjesta u slučaju neparnog kapaciteta. U bridu stanice definirane su tri ulazne prometne trake te dvije izlazne za efektivni protok vozila tijekom simulacije.



Slika 4.3 Prikaz dviju stanica suparničkih agenata spojenih na zadane bridove. Može se uočiti da stanice imaju 8 mjesta za punjenje uz 10 dodatnih mjesta za čekanje, te se po jedno vozilo trenutno

puni na svakoj stanici. Narančasta boja punionice označava da je barem jedno vozilo trenutno koristi, odnosno plava boja da se ne koristi, no budući da je stanica u kontekstu SUMO simulacije razdvojena na dvije zasebne punionice samo je punionica na kojoj je vozilo obojana narančasto. Krug na kraju stanice je u boji agenta kojem stanica pripada te samo služi za vizualizaciju, unutar SUMO simulacije ne postoji podjela stanica na agente.

U slučaju da se u stanici ne nalazi parking za čekanje, vozila čekaju na ulaznim trakama brida stanice, točno ispred parkinga za punjenje (Slika 4.4). Prilikom oslobođenja mjesta odabire se vozilo koje čeka najbliže stanici da sljedeće zauzme novooslobođeno mjesto. Vozila biraju između slobodnih mjesta na način da preferiraju drugu punionicu, čime manje smetaju nadolazećem prometu. Ideja je također bila da vozila zauzimaju i sama mjesta punionice od zadnjeg prema prvom, no budući da je to definirano u izvornom kodu SUMO programskog paketa takav poredak na kraju nije uspješno implementiran.



Slika 4.4 Prikaz stanice bez slobodnih mjesta za punjenje i red vozila koja čekaju sljedeće slobodno mjesto pri ogromnoj potražnji stanice. Crvena boja vozila označava da vozilo ide na punjenje, dok su zelena vozila dovoljno napunjena i nastavljaju kretanje do sljedećeg definiranog odredišta. U ovom slučaju je jasno vidjeti kako bi došlo do potpunog zastoja ako napunjeno vozilo želi izaći iz stanice lijevom skretanjem.

4.3. Model izvršavanja evaluacije

Evaluacija uspješnosti raspodjele stanica izvršava se koristeći već opisani *Eclipse SUMO* simulator (4.1.1). Za izvršavanje instance simulacije potrebno je unaprijed definirati tri glavna elementa [23]:

- podaci mreže (npr. ceste i raskrižja)
- dodatna prometna infrastruktura (npr. semafori, parking, stanice za punjenje)
- prometna potražnja

U simulaciji se koristi mreža dobivena iz grafa okoline definiranog u (3.2.1) te se dodatna infrastruktura parkinga i stanica za punjenje dodaje na način objašnjen u prošlom poglavlju (4.2). U ovom poglavlju još će se opisati definiranje prometne potražnje po evaluaciji i ponašanje vozila, uz referencu na korištene parametre prikazane u (Tablica 4.1).

Tablica 4.1 Parametri evaluacije

Parametar	Objašnjenje	Vrijednosti
EV_PEN	Prometna zastupljenost električnih vozila	[0, 1]
VEH_COUNT	Broj vozila u simulaciji	$\mathbb{N}$
MAX_DIST	Minimalna udaljenost putovanja	[0, promjer mreže]
MIN_DIST	Maksimalna udaljenost putovanja	[0, promjer mreže]
DEST_DIST	Distribucija broja odredišta	$([0,1], \dots, [0,1])$
Stanica		
CAPACITY	Broj mjesta za punjenje	$\mathbb{N}$
WAIT_QUEUE	Maksimalna duljina reda čekanja	$\mathbb{N}$
Preusmjeravanje		
CENTRALIZED	Koristi li se centralizirani sustav alokacije vozila na stanice?	$\{False, True\}$
STATIONFINDER	Koristi li se <i>Stationfinder</i> komponenta?	$\{False, True\}$
Vozilo		
VEH_TYPE	Tip vozila	električno, konvencionalno
SET_CHARGE	Početno stanje napunjenosti	[0, 1]
NTC	Prag pada napunjenosti	$\mathbb{R}^+$
MIN_CHARGE	Minimalna količina energije za napuniti	$\mathbb{R}^+$
TRIP	Putovanje (odredišta)	$O \subseteq E$

4.3.1. Generiranje prometne potražnje

Prometna potražnja se za *SUMO* simulaciju može definirati na tri načina: putovanja (*trips*), tokovi (*flows*) ili rute (*routes*). Osnovne informacije trebale bi uključivati vrijeme polaska, točke polazišta i odredišta te tip vozila [23]. Osim osnovnih informacija, putovanje je

definirano točkama kojima vozilo mora proći te se konkretni put generira pri početku simulacije, dok ruta unaprijed definira točni put kojim će vozilo ići, no može se dinamično mijenjati tijekom simulacije. Tokovi definiraju tok istih vozila koji imaju jednako definirana putovanja ili rute. Budući da se simulacija evaluacije fokusira na potražnju vozila za posluživanjem stanicama za punjenje, putevi kojima se kreću moraju biti dovoljno dugački da se potreba za punjenjem može prirodno pojaviti tijekom bilo kojeg dijela puta, no ne prekratki da ne dođe do slučaja gdje se vozilo ne treba puniti.

Tip vozila (*VEH_TYPE*) definiran je generalizacijom tipa vozila od kojeg će instanca vozila preuzeti razne parametre, npr. automobil s motorom s unutarnjim izgaranjem, električni automobil, bus, bicikl, itd. Cilj rada je modeliranje miješanog prometa automobila s motorom s unutarnjim izgaranjem i električnih automobila, te ćemo definirati po tip vozila sa svaki od njih: „konvencionalno“ i „električno“ vozilo. Definiramo parametar prometne zastupljenosti električnih vozila (*EV_PEN*) koji predstavlja stopu zastupljenosti električnih vozila u prometu. Dodatno je potrebno definirati početne energetske podatke za svako vozilo za kontrolu determinizma simulacije za metode iterativne optimizacije i uspoređivanje različitih raspodjela stanica. Početne energetske podatke vozila definiramo podacima o početnom stanju napunjenosti baterije električnog vozila (*SET_CHARGE*), pragu pada napunjenosti baterije kada će vozilo odlučiti potražiti stanicu za punjenje (*NTC*) i minimalnoj količini energije koje će vozilo napuniti na stanici (*MIN_CHARGE*). Vozilo ne smije započeti put sa praznom baterijom zbog potrebe za detekcijom vozila koja prirodno istroše bateriju u slučaju da ne stignu na željenu stanicu. No u suprotnom slučaju ako vozilo kreće s punom baterijom moguće je da neće biti potrebe za punjenjem ili, u slučaju da je punjenje definirano kao dio puta (kao u algoritmu 5.2), može doći do situacije gdje vozilo puni samo količinu energije potrošenu za dolazak do stanice umjesto zadane količine. Budući da je u određenim metodama potrebno puno puta izvršavati simulaciju želimo što više skratiti trajanje simulacije bez utjecaja na efektivnost evaluacije, jer bi u suprotnom vozila većinu simulacije provela na stanicama što ne pridonosi analizi efektivnosti raspodjele. Prag napunjenosti baterije (*NTC*) služi za prirodno određivanje kada će vozilo dobiti želju za punjenje i sprječava slučaj da sva vozila traže stanicu na početku simulacije, tj. u isto vrijeme. Minimalna količina energije koju će vozilo napuniti (*MIN_CHARGE*) služi da vozilo ne bude gotovo sa punjenjem odmah nakon postizanja napunjenosti baterije iznad

praga napunjenosti (*NTC*), a u isto vrijeme da se ne mora puniti skroz do pune baterije što može biti pogotovo dugotrajno.

Uzimanjem svih traženih svojstava ponašanja vozila u obzir definiramo put vozila pomoću putovanja (*TRIP*), tj. zadajući odredišta koje mora proći. Generiramo odredišta putovanja uz parametre maksimalne (*MAX_DIST*) i minimalne duljine puta (*MIN_DIST*), te parametar distribucije broja odredišta (*DEST_DIST*). Time imamo kontrolu nad duljinom puta svih vozila, čime možemo indirektno kontrolirati prosječnim trajanjem izvršavanja simulacije i utjecaja prometa ostalih auta na rješenje, budući da je dulji put jednak dugotrajnijem sudjelovanju vozila u prometu. Definiramo tip vozila (*VEH_TYPE*) podjelom na konvencionalna i električna koristeći uniformnu slučajnu distribuciju i parametar zastupljenosti električnih vozila u prometu. Zatim posebno definiramo početne energetske podatke za svako električno vozilo, koristeći uniformnu slučajnu distribuciju uz parametre. Opisani parametri vozila prikazani su u Tablica 4.1, pod „Vozilo“.

4.3.2. Odabir stanice za punjenje

Definirana svojstva vozila se dodatno dinamično mijenjaju tijekom izvršavanja simulacije. Pri odluci vozila o potrebi za punjenjem vozilo za svaku stanicu izračunava korisnost odabira te stanice koristeći parametre funkcije korisnosti sa podacima o udaljenosti i vremena putovanja do stanice i cijeni, te u slučaju centraliziranog upravljanja (Tablica 4.1 pod „Preusmjeravanje“, algoritam pod 3.3.1) također količinu vozila koja koriste ili čekaju na korištenje stanice. Nakon odabira, vozilo ažurira put dodavanjem stanice kao dodatno odredište putovanja prije sljedećeg odredišta, te generira novi put. Također se točno prije početka punjenja vozila na stanici izračunava aproksimacija potrebne energije za dovršavanje puta

$$\text{potrebna energija} = \text{prosječna potrošnja} * \text{duljina preostalog puta}$$

gdje se prosječna potrošnja i duljina preostalog puta dinamički dobivaju iz simulacije. Zatim se količina energije koju vozilo želi napuniti postavlja na maksimalnu vrijednost između aproksimacije i minimalne količine energije (parametar *MIN_CHARGE*) koje će vozilo napuniti

$$\text{količina punjenja} = \max(\text{potrebna energija}, \text{minimalna količina})$$

4.4. Graf neuronska mreža

Za pokušaj razvijanja rješenja problema baziranog na strojnom učenju korišteni su modeli graf neuronske mreže. Prije izuma graf neuronskih mreža grafovi su se procesirali kroz transformacije u jednostavnije strukture, kao skupove vektora [20]. No takvom transformacijom gube se važne topološke informacije te su efektivne graf neuronske mreže predstavljene u [20] i [21] kao generalizacija rekurzivnih neuronskih mreža koja može procesirati različite strukture grafa, kao cikličke, usmjerene i neusmjerene grafove [20].

Glavni način korištenja rekurzivnih neuronskih mreža na grafovima je stvaranje skupa stanja gdje svako stanje predstavlja jedan čvor. Zatim su stanja rekurzivno ažurirana ovisno o topološkim informacijama grafa dok se ne dođe do ravnoteže svih stanja. No ograničenje korištenja rekurzivnih neuronskih mreža na taj način je da zbog rekurzije nije bilo moguće obraditi cikličke i neusmjerene grafove [20].

Intuitivna ideja na kojoj su graf neuronske mreže bazirane je predstavljanje čvorova grafa objektima i njihove odnose bridovima. Ulazni podaci su u obliku matrice susjedstva čvorova uz matrice svojstva čvorova i bridova. Za svaki čvor n stvara se vektor stanja $x_n \in \mathbb{R}^s$ koji predstavlja objekt čvora n . Zatim se provodi takozvana funkcija lokalne tranzicije f_w kojom se stanje transformira s obzirom na ovisnost čvora n sa njegovim susjedima. Budući da čvorovi i bridovi grafa sadrže posebno definirana svojstva oni moraju biti uključeni u funkciju lokalne tranzicije. Također definiramo o_n kao izlaz, tj. odluka o stanju x_n i funkciju lokalnog izlaza g_w koja opisuje kako se izlaz izračunava [20].

$$x_n = f_w(l_n, l_{l(n)}, \eta(n), l_{\eta(n)})$$

$$o_n = g_w(x_n, l_n)$$

gdje su l_n svojstva čvora n , $l_{l(n)}$ svojstva incidentnih bridova (izraz **Error! Reference source not found.**), $\eta(n)$ skup čvorova susjeda čvora n (izraz 3.11) i $l_{\eta(n)}$ svojstva susjednih čvorova. Parametri w funkcija tranzicije i izlaza određuju se procesom učenja na temelju skupa podataka, pri čemu se minimizira određena funkcija gubitka. Prilikom prolaza unaprijed iterativno se ažuriraju stanja svih čvorova ulaznog grafa primjenom funkcije f_w sve dok se ne postigne stabilno stanje, tj. konvergencija. Nakon konvergencije, izlazi se računaju primjenom funkcije g_w za svaki čvor.

4.4.1. Rangiranje bridova

Budući da je problem rada definiran raspodjelom stanica za punjenje na bridove umjesto čvorova, potrebno je modificirati GNN definiciju na način da generira izlaz ovisno o bridu umjesto čvoru. U [14] to je napravljeno korištenjem matrice susjedstva bridova umjesto matrice susjedstva čvorova, što znači da funkcije lokalne tranzicije i izlaza izvršavaju iste operacije, samo što su stanja bazirana na bridovima, dok čvorovi služe za prijenos poruka.

U sklopu rada takav pristup nije uzet, već su napravljena dva hibridna modela čija su stanja temeljena na čvorovima, no rangiranje ovisi o bridu na način da se analiziraju stanja oba čvora koje brid povezuje, njihova svojstva i svojstva samog brida. Funkcija tranzicije se sastoji od par dijelova: prvi dio kodira duljine bridova kroz geometrijske pozicije čvorova koje povezuju, drugi zajedno kodira svojstva bridova sa izračunatim duljinama, treći provodi klasičnu lokalnu tranziciju, ažurirajući stanja čvorova kroz susjedne čvorove i bridove koji ih povezuju, te zadnji sloj ocjenjuje bridove po stanjima čvorova koje brid povezuje, njihovim svojstvima i duljini izračunatoj iz geometrijskih pozicija čvorova. Time se održava ažuriranje stanja čvorova za što su GNN i njegovi slojevi napravljeni, no također se na izlazu dobiva rangiranje bridova kombinacijom stanja čvorova brida i samih svojstva brida.

Prva varijacija modela koristi se u slučaju sustava sa jednim agentom, dok druga služi za kompetitivnu okolinu, zbog čega dodaje element procjene cijene. Ugrađen je odvojen dio koji kroz učenje optimizira Beta distribuciju iz koje se pri prolazu unaprijed uzorkuje cijena.

5. Rješenja

Budući da je problem formuliran na način da su lokacije stanica jednake bridovima prometne mreže, $C \subseteq E$, definirati ćemo proširenja određenih funkcija na domene bridova. Funkciju udaljenosti λ (izraz 3.1) proširujemo na bridove $e_1 = (x, y), e_2 = (w, z)$ na način da se udaljenost računa od krajnjeg čvora y prvog brida do početnog čvora w drugog brida.

$$\lambda(e_1, e_2) = d(y, w) \quad (5.1)$$

Jednako definiramo funkciju udaljenosti brida do najbliže stanice kao proširenje ekvivalentne funkcije čvorova (izraz 3.2).

$$\lambda(S, e) = \min_{s \in S} \lambda(s, e) \quad (5.2)$$

gdje je $e \in E$ i $S \subseteq C$, te budući da $C \subseteq E$ vrijedi $s \in E$.

5.1. Algoritam baziran na pokrivenosti mreže

Za algoritamski pokušaj rješenja zadanog problema i usporedbu rezultata sa postojećim metodama te kasnijim metodama strojnog učenja razvijen je algoritam sa fokusom na pokrivenost mreže. Glavna ideja je da želimo zahvatiti svaku točku grafa sa stanicama koje postavljamo. To znači da će vozilo na bilo kojoj točki u grafu biti maksimalno udaljeno od najbliže stanice za određenu udaljenost r . To je ekvivalentno potpunoj pokrivenosti sustava definiranoj jednačom (izraz 3.6). Stoga će cilj ovog algoritma zapravo biti (izraz 3.7), tj., u kontekstu problema, minimizacija radijusa potpune pokrivenosti prometne mreže.

Razvijeno je više varijacija algoritma koji se razlikuju po funkciji težine pokrivanja određenih čvorova ili bridova, te po načinu na koji odlučuje gdje će započeti pretraživanje sljedeće lokacije stanice. Funkcije su slične heuristikama lokalnog i globalnog pretraživanja, redom, što će se bolje vidjeti kasnije. Varijacije funkcije težine pokrivenosti dodatno se razlikuju po tome da li se fokusiraju na informacije čvorova ili bridova, te su varijacije sljedeće:

- čvorovi; količina pokrivenih čvorova

- čvorovi; stupnjevi pokrivenih čvorova
- bridovi; stupnjevi čvorova pokrivenih bridova
- bridovi; zbroj atributa duljine pokrivenih bridova
- bridovi; zbroj protoka pokrivenih bridova

Protok brida predstavlja količinu prometa koja prolazi bridom u promatranom vremenskom razdoblju. Podaci o protoku bridova dobivaju se dinamički iz simulacije prethodne iteracije, odnosno iz prazne simulacije u slučaju prve iteracije. Varijacije su razvijene redom kojim su navedene, te je posljednja odabrana zbog najbolje kompatibilnosti s iterativnim izvođenjem algoritma. Zatim su varijacije po odabiru početne točke traženja sljedeće stanice:

- najviše udaljena točka od već odabranih
- najmanje udaljena točka od već odabranih

Što su zapravo funkcije

$$\max_{x \in X} \min_{s \in S} \lambda(s, x) \quad (5.3)$$

i

$$\min_{x \in X} \sum_{s \in S} \lambda(s, x) \quad (5.4)$$

redom, gdje je $S \subset C$ podskup odabranih lokacija za stanice od skupa kandidata, te λ funkcija udaljenosti točke (izraz 3.1) od stanice te za konkretnu funkciju udaljenosti koristimo cestovnu udaljenost (izraz 3.18). Prva funkcija maksimizira udaljenost točke od njoj najbliže stanice, dok druga minimizira zbroj udaljenosti točke od svake već odabrane stanice.

Prije samih implementacija definirati će se pomoćne funkcije. Implementirana je pomoćna funkcija dohvaćanja skupa svih čvorova unutar pokrivenosti zadane stanice *getNodesInRadius*(*G, s, radius, f_udaljenost*), uz varijacije ovisno o korištenoj metodi izračuna udaljenosti (Kôd 5.1).

```
def getNodesInRadius(G, s, radius, f_udaljenost):
    pokriveni = set()
    gomila = TupleMaxHeap();
    gomila.push((radius, s));
    while len(gomila) > 0:
        d_preostala, x = gomila.pop()
```

```

    if x not in pokriveni:
        pokriveni.add(x)
        poveznice = G.out_edges(x)
        for p in poveznice:
            y = p[1]
            if y not in pokriveni:
                udaljenost = f_udaljenost(G, x, y)
                if udaljenost <= d_preostala:
                    d = d_preostala - udaljenost
                    gomila.push((d, y))

return pokriveni

```

Kôd 5.1 Funkcija izračuna skupa pokrivenosti čvora unutar zadanog radijusa

No budući da formulacija problema zadaje bridove kao potencijalne lokacije za postavljanje stanica implementirana je pomoćna funkcija *discretizeGraph*(G, E_c) koja ubacuje nove čvorove usred zadanih bridova kandidata E_c grafa G , te pomoćna funkcija *calcCandidates*(G) koja u isto vrijeme vrši detekciju bridova kandidata i izvršava maloprije definiranu pomoćnu funkciju umetanja čvorova na bridove kandidate (Kôd 5.2). Time dobivamo skup novonastalih čvorova koji postaju skup potencijalnih lokacija postavljanja stanica C , tj. skup kandidata.

```

def discretizeGraph(G, E_c):
    # Usmjereni bridovi u neusmjerene
    bridovi = set()
    for brid in G.edges():
        if brid not in E_c:
            continue;
        if brid[0] <= brid [1]:
            bridovi.add(brid);
        else:
            bridovi.add(brid[::-1]);
    for brid in bridovi:
        insertNode(G, brid[0], brid[1])
def calcCandidates(G):
    kandidati = []
    originalni_X = set(G.nodes())
    # Dodaj čvorove na usred bridova
    discretizeGraph(G, G.edges())
    kandidati = set(G.nodes()) - originalni_X

```

```
return kandidati
```

Kôd 5.2 Funkcije transformacije bridova grafa u čvorove potencijalnih mjesta za stanice (kandidate)

gdje je $insertNode(G, x_a, x_b)$ implementacija jednostavne pomoćne funkcije ubacivanja čvora usred dva zadana čvora $x_a, x_b \in X$.

Za početak izvršavanja algoritma potrebno je izabrati lokaciju prve stanice, za što su određene tri heuristike:

- odabir točke s periferije grafa
- odabir točke oko centra grafa, koristeći centralnost bliskosti (*closeness centrality*)
- odabir točke oko bridova sa najvećom potražnjom

te se zatim izvršava funkcija lokalnog pretraživanja *localSearch* za izračun najbolje lokacije u okolini odabrane (Kôd 5.3) i dobivena točka postaje lokacija prve stanice.

Zatim ulazimo u petlju odabira lokacija ostalih stanica. Za početak izvršavanja svakog od odabira potrebno je prvo odabrati početnu točku u globalnom prostoru, slično kao i na početku algoritma. No ovaj put se koristi spomenuta heuristika odlučivanja početne točke pretraživanja. Cilj heuristike je sličan globalnoj pretrazi, jer trebamo pronaći početnu točku pretraživanjem cijelog grafa, tj. globalnog prostora. U slučaju prve varijacije odabiremo točku koja je najviše udaljena od svih već odabranih lokacija stanica, čime pokušavamo postepeno pokriti mrežu izvana prema unutra. U drugoj varijaciji odabiremo točku koja je najbliža svim stanicama, čime mrežu pokušavamo pokriti proširujući se iz početne stanice. Važno je napomenuti da se čvorovi i potencijalne lokacije već pokrivene dosad odabranim lokacijama ne uključuju u prostor pretraživanja ovog koraka, što znači da druga varijacija za početnu točku pretraživanja zapravo odabire točku odmah pokraj pokrivene regije, tj. udaljenu za vrijednost radijusa od okolnih stanica. No stanica na toj točki bi onda dijelom pokrivala područje koje je već pokriveno okolnim postojećim stanicama, zbog čega se izvršava postupak lokalne pretrage. U tom slučaju bi idealni rezultat lokalne pretrage bila točka udaljena za duplu vrijednost radijusa od svake okolne stanice, čime bi pokrila samo nepokriveni prostor. Lokalna pretraga izvršava se u obje varijacije; analiziraju se okolne točke po heuristici težine pokrivenosti spomenutoj na početku potpoglavlja. Cilj heuristike je onda pronalazak najboljeg čvora u okolini početne točke pretraživanja (Kôd 5.3).

```
def localSearch(G, x, C, radius, heuristika, f_udaljenost) -> str:
```

```

# Dohvati sve potencijalne lokacije u radijusu oko x
okolina = getNodesInRadius(G, x, radius, f_udaljenost)
kandidati = C & okolina
# Izračunaj vrijednost/težinu pokrivenosti svakog zahvaćenog
kandidata
najbolji = set(); najbolji_v = 0;
for i in range(len(kandidati)):
    okolina = getNodesInRadius(G, kandidati[i], radius)
    v = heuristika(okolina)
    if v > najbolji_v:
        najbolji = {i}; najbolji_v = v;
    elif v == najbolji_v:
        najbolji.append(i)
# Ako je više najboljih opcija odaberi po blizini početnoj točki
if len(najbolji) > 1:
    najblizi = -1; najblizi_v = radius;
    for i in najbolji:
        c = kandidati[i]
        udaljenost = f_udaljenost(G, c, x)
        if udaljenost < najblizi_v:
            najblizi = c
            najblizi_v = udaljenost
    return najblizi;
else:
    return kandidati[najbolji.pop()];

```

Kôd 5.3 Funkcija lokalne pretrage koristeći heuristiku težine pokrivenosti

Cjelokupni algoritam prikazan je u nastavku (Kôd 5.4).

```

def coverageBased(G, C, k, radius, h_prva_stanica, h_global, h_local,
f_udaljenost):
    X = G.nodes()
    S = []
    # Prva stanica, odaberi heuristikom
    S_prva = h_prva_stanica(G, C)
    S_prva, pokrivenost = h_local(G, S_prva, C, X, radius)
    S.append(S_prva)
    C = C - pokrivenost
    X = X - pokrivenost
    if len(C) == 0: return S

```

```

d_do_stanice = [None]; d_do_stanice[0] = [];
for x in X_preostali:
    udaljenost = f_udaljenost(G, S[0], x)
    d_do_stanice[0].append((x, udaljenost));
for i in range(1, k):
    S_tocka = h_global(G, X, d_do_stanice)
    S_tocka, pokrivenost = h_local(G, S_tocka, C, X, radius)
    S.append(S_tocka)
    C = C - pokrivenost
    X = X - pokrivenost
    if S[i] == None or len(C) == 0:
        break;
    d_do_stanice.append([]);
    for x in X:
        udaljenost = f_udaljenost(G, S[i], x)
        d_do_stanice[i].append((x, udaljenost));
return S

```

Kôd 5.4 Cjelokupni kod algoritma baziranog na pokrivenosti mreže

Te se za minimizaciju radijusa pokrivenosti algoritam koristi u sklopu binarnog pretraživanja (Kôd 5.5).

```

def coverageBinarySearch(G, C, k, h_distribucija, h, epsilon):
    max_radius = nx.diameter(G)
    a = 0; b = max_radius; radius = 0; S = [];
    while (b - a > epsilon):
        radius = (b + a) / 2
        S = h_distribucija(G, C, k, radius, *(h))
        izvedivo = len(S) < k or isGraphCovered(G, S, radius);
        if izvedivo: b = radius;
        else: a = radius;
    return (radius, S)

```

Kôd 5.5 Funkcija binarne pretrage minimalnog radijusa za algoritam baziran na pokrivenosti mreže

Izvršavanjem cjelokupnog algoritma u sklopu binarnog pretraživanja dobivamo raspodjelu k stanica na način da je mreža potpuno pokrivena, tj. svaki čvor mreže, za minimalni radijus.

5.1.1. Više-agentska kompetitivna varijacija

Budući da je cilj rada razvoj mogućih rješenja za optimiranje raspodjele stanica za punjenje električnih vozila opisani algoritam nadograđen je u višeagentsku kompetitivnu varijaciju. Za realizaciju kompeticije potreban je dodatan element cijena koje agenti odlučuju i mijenjaju uz odabir lokacija stanica. Pitanje je na koji način bi se koristio koncept pokrivenosti za odlučivanje o cijeni stanica agenta.

Agenti biraju stanice kroz isti algoritam kao u osnovnoj varijaciji, jedino se odabir izvršava po redu; prvo bira prvi agent, zatim drugi, i tako do zadnjeg. Time zapravo prednost uvijek ima prvi agent te se mogu implementirati mehanizmi protiv prednosti, kao izmjena redoslijeda odabira u svakoj iteraciji, no u razvijenom rješenju je poredak uvijek isti, uz ideju da agenti koji nemaju prednost imaju mogućnost smanjenja cijena za izjednačavanje količine privučenih vozila. Također se pazi na to da agenti ne odabiru iste stanice, svaki agent uzima u obzir samo stanice koje još nisu odabrane od strane prošlih agenata u trenutnoj iteraciji.

U kompetitivnom okruženju pokrivenost pojedinog agenta nije određena samo radijusom, već i prisutnošću stanica drugih agenata. Neka je A skup svih agenata sustava i S_a skup stanica koje pripadaju agentu $a \in A$, tj. lokacije stanica koje je taj agent odabrao. Definiramo podjelu bridova prometne mreže po agentima na način da je brid dodijeljen agentu čija je stanica njemu najbliža.

$$P_a = \{e \in E \mid \lambda(S_a, e) \leq \lambda(S_b, e), \forall b \in A\}$$

gdje je λ funkcija udaljenosti brida e do najbliže stanice (izraz 5.2) podskupa $S \in C$ odabranih stanica od svih potencijalnih lokacija stanica $C \subseteq E$.

Definiramo potražnju stanice $s \in S$ kao zbroj umnožaka protoka brida i inverzne udaljenosti brida od te stanice za svaki brid koji ta stanica pokriva.

$$potražnja_s = \sum_{e \in Cov(s,r)} protok_e * \exp(-0.01 * \lambda(s, e))$$

gdje je r proizvoljni radijus, $Cov(s, r)$ funkcija pokrivenosti (izraz) i $\exp$ eksponencijalna funkcija s bazom e (Eulerov broj). Korišteno je eksponencijalno opadanje kod udaljenosti za glatko opadanje utjecaja udaljenosti i realističnije modeliranje definirane potražnje. Podaci protoka brida također se dobivaju dinamički iz prethodne simulacije, kao u osnovnoj

varijaciji algoritma. Zatim definiramo ukupnu potražnju koju agent uspijeva pokriti u jednoj iteraciji sustava.

$$potražnja_a = \sum_{s \in S_a} potražnja_s$$

Agenti cijene određuju ovisno o uspješnosti pokrivanja mreže u odnosu na druge agente. Budući da je važnije pokriti što veću potražnju umjesto samo broja čvorova ili bridova, u izračunu se koristi opisana ukupna potražnja agenta. Svaki agent $a \in A$ započinje sa unaprijed definiranom početnom cijenom $cijena_a$. Nakon odabira normaliziraju se potražnje agenata koristeći prosječne apsolutne udaljenosti podataka od srednje vrijednosti (*mean absolute deviation*, MAD).

$$norm_cijena_a = \frac{potražnja_a - \mu_{potražnja}}{\mu_{potražnja}}$$

gdje je $\mu_{potražnja}$ srednja vrijednost potražnja svih agenata ove iteracije. Te se nove cijene svakog agenta računaju pomoću normalizirane potražnje agenta i faktora momenta α .

$$nova_cijena_a = cijena_a + (\alpha * norm_cijena_a)$$

Uz napomenu da su unaprijed definirane minimalna i maksimalna cijena.

5.2. Algoritam baziran na teoriji igara

Za usporedbu sa ostalim rješenjima razvijen je i algoritam baziran na teoriji igara, po k -*GRAPE* algoritmu iz [11]. Glavna ideja je da svako vozilo odabere stanicu iz zadanog skupa od K stanica na kojoj preferira punjenje ovisno o njihovom putu i trenutnoj „gužvi“ na stanici, tj. koliko je vozila odabralo tu stanicu u prošloj iteraciji. Taj postupak se iterativno ponavlja dok se ne postigne Nashova ravnoteža za zadane stanice, što znači da je odabir preferiranih stanica u stanju ravnoteže gdje niti jedno vozilo ne mijenja svoj odabir preferirane stanice u sljedećoj iteraciji. Izvan ove funkcije ravnoteže provodi se algoritam koji mijenja raspodjelu stanica ovisno o grupama vozila formiranim kroz odabir preferiranih stanica [11].

Najutjecajnija razlika algoritma sa k -*GRAPE* algoritmom je način na koji su definirana vozila. U slučaju k -*GRAPE* algoritma vozila imaju definirane početne lokacije sa kojih se kreću kretati, dok su u kontekstu ovog rada definirani njihovim odredištima iz kojih su

generirani njihovi putevi. Stoga se u *k-GRAPPE* algoritmu računanje troška korištenja stanice za vozilo generalno računa kroz udaljenost njegove početne lokacije od zadane stanice, uz faktor gužve

$$\min_{c \in C} (d(x_c, x_v) + \text{guzva}(c))$$

gdje je $C \subseteq X$ skup svih potencijalnih stanica, x_c točka lokacije potencijalne stanice $c \in C$, $d: X \times X \rightarrow \mathbb{R}^+$ proizvoljno definirana funkcija udaljenosti te $\text{guzva}(c)$ broj vozila koje trenutno preferira stanicu c . Zato se zbog načina definicije vozila u ovom radu ta metoda mora proširiti na svako od odredišta koje je zadano vozilu, jer vozilo može odlučiti napraviti zaobilazak do stanice za punjenje nakon bilo kojeg odredišta. Zato u izračunu troška punjenja na stanici uzimamo u obzir svako odredište kao moguću početnu točku i odabiremo odredište koje daje najmanji mogući trošak (Kôd 5.6):

$$\min_{c \in C} \min_{o \in O_v} (d(x_c, o) + \text{guzva}(c))$$

gdje je $O_v \subseteq E$ skup bridova odredišta vozila v .

```
def trosakOdabiraStanice(vozilo, put, stanica):
    # Provjeri ako je već izračunato
    global matrica_trosaka
    if matrica_trosaka [vozilo][stanica][0] >= 0.0:
        return matrica_trosaka [vozilo][stanica]
    else:
        # Koeficijenti
        udaljenost_c = 0.0; vrijeme_c = 1.0;
        min_trosak = np.inf; indeks_ubacivanja = -1;
        # Izračunaj trošak za svako odredište
        for i in range(1, len(put)):
            trosak = 0.0;
            # Kreiraj novi put ubacivanjem stanice
            put_do_stanice = put.insert(stanica, i)
            # Izračunaj trošak
            udaljenost = edgePathLength(put_do_stanice,
            weight="udaljenost")
            trosak += udaljenost * udaljenost_c
            vrijeme_puta = edgePathLength(put, weight="vrijeme")
            trosak += vrijeme_puta * vrijeme_c
            # Ažuriraj ako je manji trosak
```

```

        if trosak < min_trosak:
            indeks_ubacivanja = i
            min_trosak = trosak
# Ažuriraj matricu
matrica_trosaka[vozilo][stanica] = (indeks_ubacivanja, min_trosak)
return indeks_ubacivanja, min_trosak

```

Kôd 5.6 Algoritam izračuna troška punjenja na stanici za zadano vozilo i stanicu

Opisani algoritam zatim koristimo pri izračunu Nashove ravnoteže preferencija vozila (Kôd 5.7) i sljedeće raspodjele stanica (Kôd 5.8).

```

def findEquilibrium(EVs, putevi, S):
    guzva = [0] * len(S)
    proslost = deque(); max_ciklus = 3;
    for i in range(10000):
        update = {};
        # Vozila odabiru preferirane stanice
        for vozilo in EVs:
            odabrana_stanica, novi_put =
trosakOdabiraStanice(vozilo, putevi[vozilo], S, guzva);
            if (vozilo not in odabrane_stanice) or
(odabrane_stanice [vozilo] != odabrana_stanica):
                update[vozilo] = odabrana_stanica;
        # Provjeri ravnotežu
        if len(update) == 0: break;
        # Ažuriraj guzvu
        for vozilo in update:
            if vozilo in odabrane_stanice:
                guzve[odabrane_stanice[vozilo]] -= 1;
                odabrane_stanice[vozilo] = update[vozilo];
                guzva[update[vozilo]] += 1;
        # Provjeri ciklus
        h = hash(tuple(odabrane_stanice.values()))
        ciklus = False;
        for j in range(len(proslost)):
            if int(h) == int(proslost[-j-1]):
                ciklus = True; break;
        if ciklus: break;
        proslost.append(h)

```

```

        if len(proslost) > max_ciklus: proslost.popleft();
    return odabrane_stanice, guzva

```

Kôd 5.7 Algoritam za pronalazak Nashove ravnoteže

```

def sljedecaRaspodjela(C, putevi, K, stanice_vozila):
    S = [None] * K
    prosle_S = list(stanice_korisnici.keys())
    # Za svaku grupu korisnika
    while any(s is None for s in S):
        dretve = []
        for i in range(len(stanice_korisnici)):
            if S[i] is not None: continue;
            vozila = stanice_korisnici[prosle_S[i]]
            if len(vozila) == 0:
                # Odaberi slučajno -> eksploracija
                S[i] = random.choice(C)
                continue
            d = dretva(C, putevi, vozila, S, i)
            dretve.append(d); d.start();
        for d in dretve: d.join();
        # Provjeri ako je stanica odabrana dva puta
        odabrane = set()
        for i in range(len(S)):
            if S[i] in odabrane:
                S[i] = None;
            elif S[i] in C:
                C.remove(S[i])
                odabrane.add(S[i])
    return S

```

Kôd 5.8 Algoritam izračuna sljedeće optimalne raspodjele stanica za zadane grupe preferencije vozila

gdje je $dretva(C, putevi, vozila, S, i)$ pomoćna funkcija izračuna optimalne lokacije stanice od svih potencijalnih lokacija C za zadanu grupu vozila $vozila$ (Kôd 5.9).

```

def dretva(C, putevi, vozila, S, indeks):
    najbolji_c = None; min_trosak = np.inf;
    for c in C:
        trosak_c = 0
        # Izračunaj

```

```

        for vozilo in vozila:
            indeks_ubacivanja, trosak =
            trosakOdabiraStanice(vozilo, putevi[vozilo], c)
            trosak_c += trosak
        # Ažuriraj najbolje
        if trosak_c < min_trosak:
            najbolji_c = c; min_trosak = trosak_c;
    S[indeks] = najbolji_c

```

Kôd 5.9 Kôd algoritma za pronalazak optimalne lokacije stanice za zadana vozila, koji se izvršava u neovisnim dretvama

Pomoćna funkcija pokreće se u odvojenim dretvama na način da svaka dretva izračunava optimalnu stanicu na indeksu *indeks* u listi raspodjele stanica. Na kraju obje funkcije pronalaska ravnoteže i izračuna sljedeće raspodjele koristimo u glavnoj petlji algoritma (Kôd 5.10).

```

najbolje = None
S = raspodjeliNasumično(C)
for i in range(broj_iteracija):
    # Izračunaj ravnotežu i evaluiraj raspodjelu
    odabrane_stanice, guzva = findEquilibrium(EVs, putevi, S)
    rezultati = evaluateAssignedSolution(odabrane_stanice)
    stanice_vozila = {}
    for s in S:
        stanice_vozila[s] = set()
    for vozilo, s in odabrane_stanice.items():
        stanice_vozila[s].add(vozilo)
    # Ažuriraj najbolje
    if najbolje is None: najbolje = rezultati;
    else:
        ocjena = usporedi(rezultati, najbolje)
        if ocjena[0] > ocjena[1]: najbolje = rezultati;
    # Izračunaj novu raspodjelu
    S = sljedecaRaspodjela(C, putevi, K, stanice_vozila)

```

Kôd 5.10 Glavna petlja algoritma baziranog na teoriji igara

gdje je *evaluateAssignedSolution(odabrane_stanice)* funkcija evaluacije rješenja sa vozilima unaprijed raspoređenima po stanicama, uključujući indeks odredišta nakon kojega odlaze na stanicu.

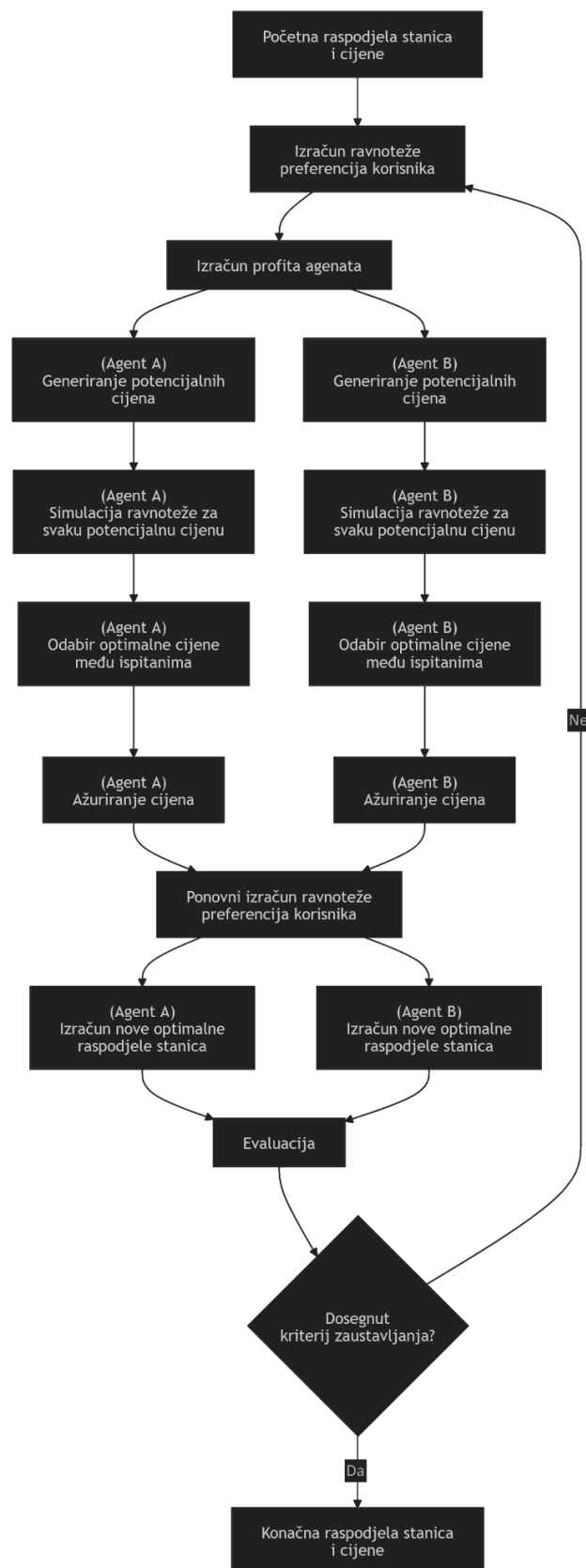
5.2.1. Više-agentska kompetitivna varijacija

Za realizaciju kompeticije u opisanom algoritmu potreban je opet dodatan element cijena koje agenti odlučuju i mijenjaju uz odabir lokacija stanica. Odlučivanje o cijeni bi poželjno također bilo bazirano na konceptima iz teorije igara, stoga je u tu svrhu opet odabran izračun Nashove ravnoteže.

Algoritam također kreće sa slučajno odabranim stanicama te se na početku iteracije računa Nashova ravnoteža preferencija vozila (Kôd 5.7) čime se vozila grupiraju po preferiranim stanicama. Unaprijed su definirane maksimalna i minimalna vrijednost cijena te početne cijene, te se izračun Nashove ravnoteže izvršava koristeći te cijene. Zatim se započinje glavna petlja i pokreće se korak izračuna novih cijena, na način da svaki agent ima skup potencijalnih cijena oko trenutne cijene za koje želi vidjeti da li mu daju bolji rezultat, tj. profit. Potencijalne cijene računaju se kroz tri vrste koraka: mali korak (od 0.05 do 0.1), srednji korak (0.1 do 0.15) i veliki korak (0.15 do 0.2). Konkretno vrijednosti svakog koraka dobivaju se uniformnom distribucijom, te su potencijalne cijene jednake $cijena \pm korak$ za svaki korak. To znači da svaki agent generira skup od šest potencijalnih cijena. Za svaku od potencijalnih cijena ponovno se formiraju grupe vozila izračunom Nashove ravnoteže, privremeno postavljajući cijene stanica pripadajućeg agenta na ispitivanu cijenu, uz to da cijene ostalih agenata nisu mijenjane.

U sklopu varijacije algoritma nadograđen je i odabir preferencija vozila dodavanjem cijene stanice kao faktor u odabiru. Stoga su skupovi grupa uzimajući u obzir potencijalne cijene agenata drugačiji od skupa grupa s početka iteracije te algoritam za svaki od njih, uključujući početni skup, računa profit svakog agenta za zadani skup. Profit agenta izračunava se zbrajanjem količine energije koju vozilo planira napuniti za svako od vozila u grupama stanica koje pripadaju agentu. Količina energije koje vozilo planira napuniti izračunata je iz duljine planiranog puta vozila, što znači da za izračun nije potrebno pokretanje simulacije, no također je dobiveni profit samo estimacija. Zbog relativne dugotrajnosti izračuna Nashove ravnoteže svako od izračuna izvršava se na zasebnoj dretvi, u isto vrijeme. Nakon završetka rada svih dretvi i dohvaćanja rezultata grupiranja i profita svaki agent analizira postignute profite te usporedbom razlika vlastitog i protivničkih profita sa dosad najboljim slučajem odlučuje da li će promijeniti cijene vlastitih stanica na jednu od novih vrijednosti. Zatim se iteracija nastavlja jednako kao u osnovnoj verziji algoritma; vozila se ponovno

grupiraju po preferiranim stanicama kroz izračun Nashove ravnoteže kao i na početku iteracije, no ovaj put se koriste novopostavljene cijene. Zatim se provodi algoritam odabira novih stanica ekvivalentan osnovnom (Kôd 5.9), čime agenti odabiru novu raspodjelu stanica. Iteracija završava evaluacijom novih raspodjela stanica i cijena agenata, koristeći nadograđenu funkciju iz osnovnog algoritma *evaluateAssignedSolution(odabrane_stanice, cijene)*.



Slika 5.1 Dijagram toka više-agentske kompetitivne varijacije algoritma baziranog na teoriji igara za slučaj dva agenta.

5.3. Strojno učenje

U implementaciji rješenja koristeći metode strojnog učenja korištene su graf neuronske mreže za rangiranje bridova objašnjene u potpoglavlju 4.4.1. Razvijene su dvije varijacije rješenja, jedna za slučaj jedno-agentskog, globalnog sustava raspodjele stanica, te druga za slučaj više-agentskog kooperativno-kompetitivnog sustava. Modeli su ućeni kroz nadzirano ućenje koje koristi jedno-kriterijsku optimizaciju linearne funkcije težina (3.1.5) za određivanje nagrada.

Prije početka treniranja za svako vozilo generirana su njegova odredišta koja mora proći po redu i podaci o punjenju, po postupku opisanom u potpoglavlju 4.3.1. Ti podaci su unaprijed generirani za konzistentnost i determinizam tijekom treninga jer bi u suprotnom rezultati evaluacije bili pod utjecajem slučajnosti i model bi ućio po slučajnim vrijednostima. To naravno daje veliko ogranićenje rezultirajućem modelu jer nije primjenjiv na nove situacije, no generiranje podataka na ovaj naćin moće se poistovjetiti sakupljanju podataka analizirane prometne mreće u stvarnom svijetu, npr. postavljanjem više vozila da se kreće guććim cestama.

5.3.1. Scenarij s jednim agentom

Tablica 5.1 Globalni kriteriji funkcije cilja, naćin optimizacije i početne vrijednosti težina

Kriterij	Smjer optimizacije	Poćetna težina
Radijus globalne pokrivenosti	MIN	1.0
Ukupna poslućena energija	MAX	0.0
Trajanje simulacije	MIN	0.2
Prosjećno trajanje puta	MIN	0.25
Prosjećna duljina puta	MIN	0.25
Prosjećno vrijeme čekanja vozila	MIN	0.15
Prosjećno vrijeme stajanja vozila	MIN	0.15

Funkcija cilja jednog agenta uključuje globalne kriterije nabrojene u (Tablica 5.1). Radijus globalne pokrivenosti određen je binarnim pretraživanjem radijusa pokrivenosti za odabrane stanice, opisanim u potpoglavlju 0. Agent je učen metodom podržanog učenja (*Reinforcement learning*, RL) na način da je u svakoj iteraciji izračunata nagrada za odabrane stanice te iteracije kroz evaluaciju te je iz nagrade izračunat gubitak iteracije, uz dodavanje bonusa entropije za poticanje eksploracije. Svaku iteraciju izvršava se evaluacija koja generira rezultate za svaki od kriterija. Koriste se pomoćne varijable prosjeka i varijance vrijednosti rezultata cijelog treniranja koje se kontinuirano ažuriraju kroz proces treniranja. Za ažuriranje korištena je hibridna metoda od *Exponential Moving Average* (EMA) i Welfordovog algoritma

$$\mu_i = \mu_{i-1} + \alpha(x_i - \mu_{i-1})$$

$$v_i = (1 - \alpha)v_{i-1} + \alpha(x_i - \mu_{i-1})(x_i - \mu_i)$$

gdje je x_i vrijednost rezultata trenutne iteracije za određeni kriterij, μ_i, v_i prosječna vrijednost i varijanca vrijednosti rezultata za isti kriterij, ažurirani u trenutnoj iteraciji, te α koeficijent momenta. Hibridna metoda kombinira stabilnost i jednostavnost EMA uz smanjenje pristranosti kroz Welfordov algoritam, uključujući i trenutnu prosječnu vrijednost i prosječnu vrijednost prošle iteracije. Nagrada za specifični kriterij rezultata se zatim računa kroz z-vrijednost, određujući koliko je trenutna vrijednost daleko od prosjeka treninga, uz trenutnu standardnu devijaciju treninga

$$z = \frac{x - \mu}{\sigma}$$

gdje je standardna devijacija $\sigma = \sqrt{v}$. Budući da vrijednosti rezultata različitih kriterija pokrivaju različite domene normaliziramo svaku od nagrada kroz tangens hiperbolni

$$r = \tanh(z)$$

čime dobivamo normaliziranu nagradu $r \in (-1,1)$ za svaki od kriterija. Dodatno u slučaju minimizacije invertiramo vrijednost nagrade $r = -r$. Zatim svaku od nagrada zbrojimo u totalnu nagradu, množenjem svake sa pripadnim faktorom težine (Tablica 5.1). Budući da rezultirajuća ukupna nagrada opet nije normalizirana, računamo prednost uspoređujući nagradu sa tekućom nagradom, tj. prosječnom nagradom treninga, te zatim ažuriramo i samu tekuću nagradu. Na kraju se računa gubitak produktom log-vjerojatnosti odabranih stanica i prednosti, uz dodatak već spomenute entropije za poticanje eksploracije.

5.3.2. Scenarij više agenata

Tablica 5.2 Kompetitivni kriteriji agenata, način optimizacije i početne vrijednosti težina

Kriterij	Smjer optimizacije	Početna težina
Radijus pokrivenosti stanica agenta	MIN	1.0
Poslužena energija na stanicama agenta	MAX	0.0
Profit	MAX	2.0

U ovom scenariju agenti su učeni kroz više-agentsko podržano učenje (MARL). Postupak je sličan kao u scenariju jednog agenta uz nadogradnju funkcije cilja kompetitivnim kriterijima (Tablica 5.2). Radijus pokrivenosti stanica agenta također je određen binarnim pretraživanjem radijusa pokrivenosti po (0), no koristeći samo stanice koje je taj agent odabrao. Profit je izračunat produktom poslužene energije na stanicama agenta i cijenom koju je agent odabrao, što znači da već uzima u obzir posluženu energiju agenta te je zato težina poslužene energije agenta jednaka nuli. Zbrajanjem težina globalnih i kompetitivnih faktora dobivamo vrijednost 5 sa omjerom 2:3 za ukupni utjecaj globalnih i kompetitivnih faktora po redu, što je određeno sa ciljem isticanja kompetitivnosti za ovaj scenarij.

Za svaki od kompetitivnih kriterija prati se posebni prosjek i varijanca rezultata, te su nagrade razdvojene na globalnu i nagrade po agentu. Globalna nagrada računa se po globalnim kriterijima (Tablica 5.1), jer je cilj algoritma i dalje globalna optimizacija odabira stanica, dok se nagrade po agentu računaju po kompetitivnim kriterijima. Finalna nagrada agenta je zbroj globalne nagrade iteracije i vlastite nagrade agenta, iz čega se zatim računa prednost agenta usporedbom sa tekućom nagradom agenta.

Dodatan element scenarija je treniranje odabira vrijednosti cijene. Odabir cijene modeliran je kroz uzorkovanje iz Beta distribucije (4.4.1) te se gubitak po cijeni računa odvojeno od gubitka odabira stanica. Agent trenira α i β parametre Beta distribucije te se u svakoj iteraciji iz trenutnih parametara uzorkuje normalizirana cijena koja se zatim primjenjuje na potpunu definiranu domenu cijena, u radu konkretno $[0.1, 1.0]$. Pri računanju ukupnog gubitka iteracije za agenta zbrajaju se gubici odabira stanica i cijene, uz to da se gubitak cijene množi sa faktorom težine da ima dovoljno sličan utjecaj na ukupni gubitak kao i gubitak odabira stanica.

6. Rezultati

Algoritmi su ispitani na dvije mreže; prvo na mreži u obliku 10x10 rešetke (Slika 6.1) generirane pomoću *Eclipse SUMO* alata te zatim na mreži generiranoj iz stvarne prometne mreže manjeg dijela grada Zagreba (Slika 4.1). Obje metode alokacije vozila po stanicama (sebična i centralizirana, opisane u poglavlju 3.3.1) ispitane su i uspoređene zajedno. Uspješnost algoritama uspoređena je koristeći jedno-kriterijsku težinsku funkciju cilja definiranu za algoritme strojnog učenja (5.3), koristeći definirane početne težine (Tablica 5.1 za globalnu usporedbu, Tablica 5.2 za usporedbu isključivo kompetitivnih algoritama). Ispitat ćemo algoritme na dva načina, koristeći najbolje rješenje nađeno kroz treniranje i ponovnim pokretanjem završnih stanja algoritama. Za prvi način, odabrane stanice, cijene i rezultati najboljeg rješenja nađenog kroz treniranje spremljeni su završetkom treniranja pa se prilikom usporedbe samo dohvate. No, budući da se algoritmi izvršavaju kroz više iteracija, nakon završetka treniranja algoritmi će doći do konvergencije odabira. Zato ćemo ih usporediti i na spomenuti drugi način u kojem koristimo rezultate te posljednje iteracije do kojih su algoritmi konvergirali, tj. ponovno pokrećemo algoritam koristeći njegovo naučeno, finalno stanje. Takav način usporedbe važan je u slučaju modela strojnog učenja, jer model treniranjem modificira vlastite parametre težina i cilj je usporediti finalno stanje modela sa rezultatima ostalih algoritama. Stoga se ponovno pokretanje algoritama izvršava specifično za svaki algoritam:

- 1.) Za algoritam baziran na pokrivenosti dohvaćaju se cijene iz podataka posljednje iteracije treniranja dok se za raspodjelu stanica prvo izvršava prazna simulacija koja generira podatke protoka vozila za svaki brid te se pokreće algoritam na tim podacima.
- 2.) Za algoritam baziran na teoriji igara dohvaćaju se cijene i raspodjela stanica iz stanja posljednje iteracije treniranja.
- 3.) Za model strojnog učenja učitavaju se naučeni parametri modela, izvršava se prazna simulacija i podaci zapisuju u mrežu pomoću koje se zatim izvršava unaprijedni prolaz za dohvaćanje raspodjele stanica i vrijednosti cijena.

U opisu se može uočiti da se za algoritme pokrivenosti i teorije igara uglavnom samo dohvate konvergirane vrijednosti i algoritam primjeni na mrežu nadopunjenu podacima prazne simulacije, dok se za model strojnog učenja konkretno koriste njegove naučene težine.

Budući da određene kriterije minimiziramo uspjeh se ne može izračunati samo zbrojem vrijednosti rezultata za svaki kriterij. Stoga se uspjeh računa na relativan način: uspoređuju se rezultati više algoritama tako da se izračunaju normalizirane vrijednosti rezultata, ovisno o maksimalnoj vrijednosti rezultata za pojedini kriterij, te ako se kriterij minimizira normalizirana vrijednost se invertira.

$$v_{i_{norm}}^{(c)} = v_i^c / \max v^c$$

$$u_i^{(c)} = \begin{cases} 1 - v_{i_{norm}}^{(c)}, & \min \\ v_{i_{norm}}^{(c)}, & \max \end{cases}$$

gdje je v_i^c vrijednost rezultata u i -tom algoritmu za kriterij c , $\max(x)$ funkcija maksimalne vrijednosti u skupu te $\max$ i $\min$ oznake da maksimiziramo odnosno minimiziramo kriterij c . Zatim se sve tako izračunate vrijednosti rezultata množe s težinom kriterija (Tablica 5.1, pod „Početna težina“) i zbrajaju. Također će se usporediti dugotrajnost izvršavanja algoritma i par konkretnih kriterija: trajanje simulacije, prosječna dugotrajnost puta i ukupni radijus pokrivenosti te radijus pokrivenosti stanica agenta i cijene agenta u slučaju kompetitivnosti.

Algoritmi su imenovani na sljedeći način:

- *Cover* – algoritam baziran na pokrivenosti, osnovna varijacija
- *Cover2*, *Cover3* – algoritam baziran na pokrivenosti, više-agentska kompetitivna varijacija sa 2, odnosno 3 agenta
- *Game* – algoritam baziran na teoriji igara, osnovna varijacija
- *Game2*, *Game3* - algoritam baziran na teoriji igara, više-agentska kompetitivna varijacija sa 2, odnosno 3 agenta
- *GNN* – algoritam baziran na modelu graf neuronske mreže, jedno-agentska varijacija
- *MARL2*, *MARL3* – kompetitivni algoritam baziran na više-agentskom sustavu modela graf neuronskih mreža sa 2, odnosno 3 agenta

Pri izvršavanju algoritama korištene su vrijednosti parametara prikazane u (Tablica 6.1), dok su podaci potražnje unaprijed generirani specifično mreži koristeći iste parametre. Parametri

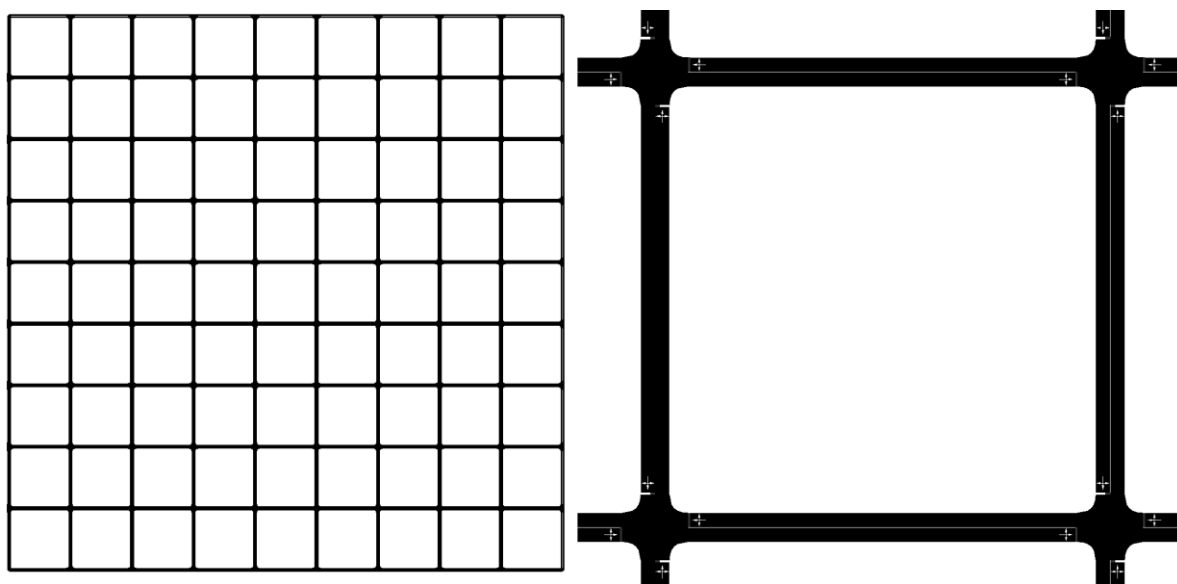
su dodatno objašnjeni u poglavlju 4.3, a postupak generiranja podataka o potražnji u potpoglavlju 4.3.1.

Tablica 6.1 Prikaz korištenih globalnih parametara

Parametar	Objašnjenje	Vrijednost
EV_PEN	Prometna zastupljenost električnih vozila	0.8
VEH_COUNT	Broj vozila u simulaciji	100
MAX_DIST	Minimalna udaljenost putovanja	2 * promjer mreže
MIN_DIST	Maksimalna udaljenost putovanja	0.5 * promjer mreže
DEST_DIST	Distribucija broja odredišta	(0, 0.3, 0.5, 0.2)
CAPACITY	Broj mjesta za punjenje	8
WAIT_QUEUE	Maksimalna duljina reda čeka	10
STATIONFINDER	Koristi li se <i>Stationfinder</i> komponenta?	<i>False</i>

6.1. Rešetka

Generirana mreža rešetke sastoji se od 100 čvorova i 360 bridova te ima promjer od 1550 metara. Broj izvršenih iteracija svakog algoritma prikazan je u (Tablica 6.2), a korišteni podaci potražnje u (Tablica 6.3).



Slika 6.1 Prikaz prometne mreže rešetke. Lijeva slika prikazuje cjelokupnu prometnu mrežu. Druga slika prikazuje detalj jednog bloka prometne mreže.

Tablica 6.2 Broj iteracija treniranja svakog od algoritama u slučaju rešetkaste mreže

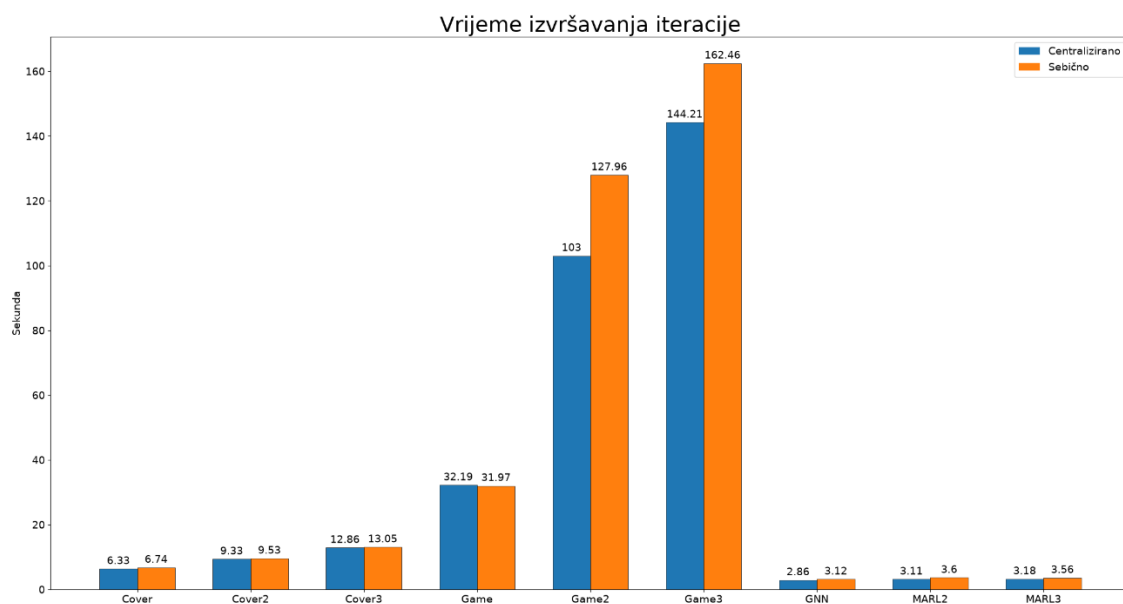
Algoritam pokrivenosti	Algoritam teorije igara	Model graf neuronske mreže
1000	100	5000

Tablica 6.3 Podaci potražnje korišteni u slučaju rešetkaste mreže

Broj električnih vozila	84 vozila
Prosječna udaljenost putovanja	2304.60 metara
Prosječni broj odredišta	2.86 odredišta

Ponajprije ćemo prikazati vremena izvršavanja jedne iteracije svakog od algoritama (Slika 6.2). Usporedba vremena izvršavanja jedne iteracije algoritama prikazana je grupiranim stupčastim dijagramom gdje svaka grupa stupaca prikazuje vrijednosti jednog algoritma pri izvršavanju sa metodom sebične (narančasta boja), odnosno centralizirane (plava boja) alokacije vozila po stanicama.

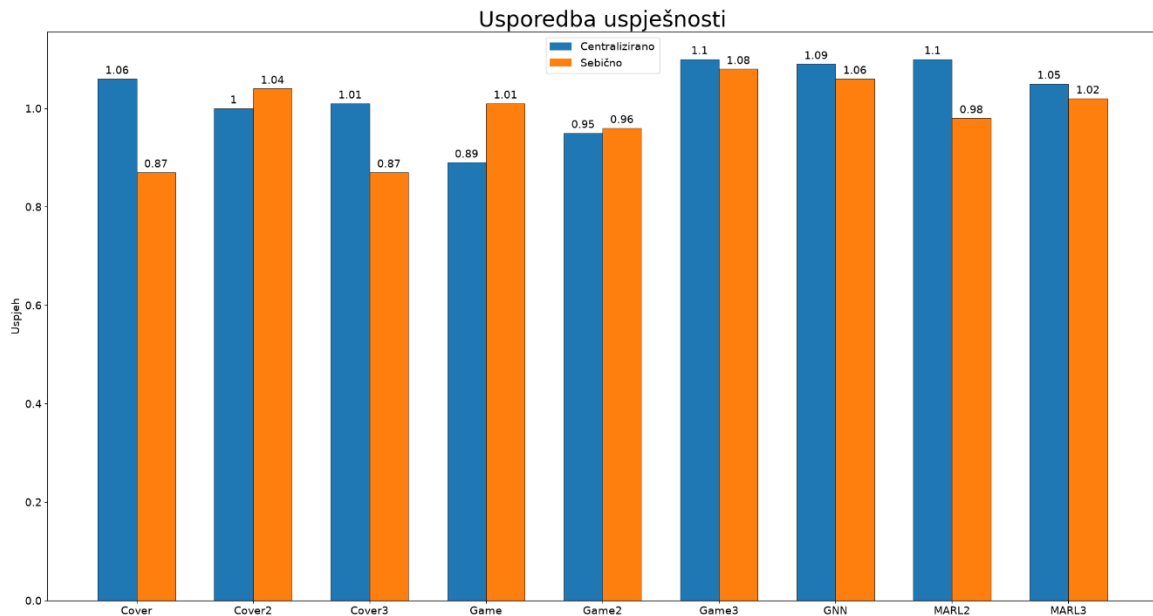
Očigledno je da se algoritmi bazirani na teoriji igara izvršavaju znatno dulje od ostalih zbog potrebe za izračunom Nashove ravnoteže i analiziranjem svih potencijalnih lokacija stanica pri određivanju sljedeće raspodjele. Dodatno se više-agentske varijante još dulje izvršavaju zbog računanja Nashove ravnoteže za svaku od potencijalnih novih cijena. No to se izvodi višedretveno u isto vrijeme što bi trebalo praktički biti vremenski jednako računanju jedne Nashove ravnoteže, no budući da su se svi algoritmi izvršavali na računalu istovremeno ta prednost vjerojatno nije došla do toliko isticanja. Također se može uočiti da je potrebno dulje vrijeme za izvršavanje algoritma pri primjeni sebične alokacije vozila na stanice.



Slika 6.2 Usporedba vremena izvršavanja po iteraciji za sve algoritama na prometnoj mreži rešetke

6.1.1. Najbolje rješenje

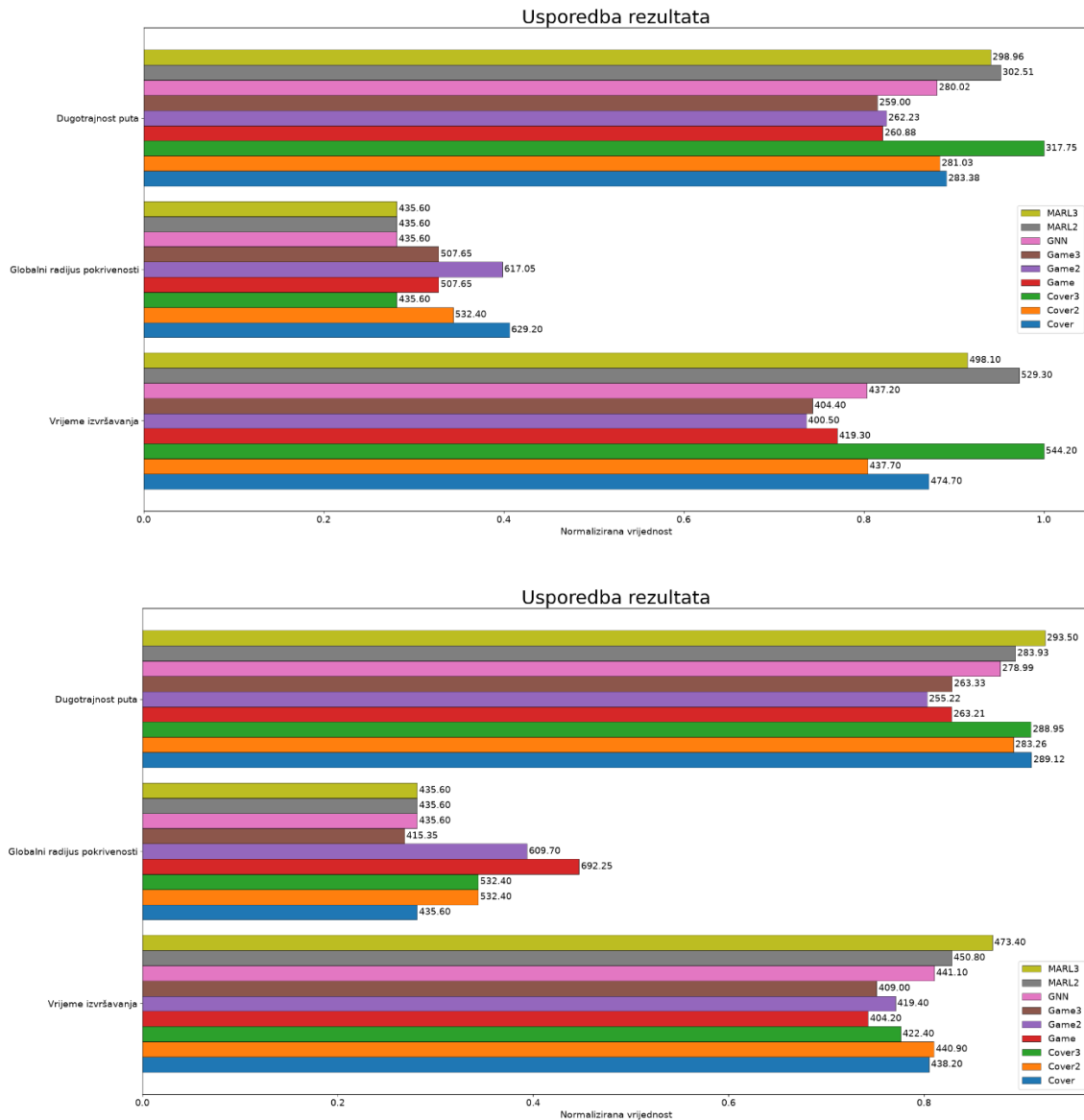
Analiziramo najbolje rezultate uspješnosti svih varijacija algoritama na izvršavanju u rešetkastoj mreži (Slika 6.3). Dijagram prikazuje maloprije objašnjenje normalizirane vrijednosti uspjeha te možemo vidjeti da algoritmi izvršeni u centraliziranoj alokaciji vozila po stanicama (plava boja) postižu uglavnom malo bolje rezultate od sebične alokacije (narančasta boja). Generalno se čini da algoritmi bazirani na pokrivenosti i algoritmi graf neuronske mreže preferiraju centraliziranu alokaciju, dok algoritam baziran na teoriji igara preferira sebičnu. Također se može uočiti da algoritmi graf neuronske mreže postižu najbolji uspjeh, no treba se uzeti u obzir veliki broj iteracija treniranja graf neuronskih mreža, čime imaju slobodu ispitivanja više mogućih raspodjela. Najgore rezultate postižu algoritmi pokrivenosti u sebičnoj alokaciji vozila.



Slika 6.3 Usporedba vrijednosti uspjeha najboljih rješenja svih algoritama na prometnoj mreži rešetke

Zatim dolazimo do konkretnih podataka. Analiziramo dijagrame rezultata trajanja izvršavanja simulacije, globalnog radijusa pokrivenosti te dugotrajnosti putovanja za slučajeve sebične i centralizirane alokacije (Slika 6.4). U ovom su slučaju rezultati grupirani po tri navedena kriterija, dok svaka boja predstavlja pripadni algoritam. Podaci su također prikazani u normaliziranom obliku, no na stupcima su zapisane apsolutne vrijednosti. Također je važno napomenuti da je radijus pokrivenosti normaliziran promjerom mreže. Budući da sve navedene kriterije minimiziramo dijagram se može proučavati na način da su kraći stupci ekvivalentni boljem uspjehu. Odmah se može uočiti dominacija algoritama teorije igara u kreiranju najkraćih putovanja i time najkraćih simulacija, iako postižu općenito goru pokrivenost mreže u odnosu na algoritme graf neuronske mreže, koji postižu zanimljivo konstantnu pokrivenost mreže. Do toga dolazi zbog glavnog načina optimizacije svakog od ta dva algoritma: u algoritmu teorije igara svako vozilo sebično bira stanicu najbližu svojem putovanju, pa algoritam zapravo optimizira problem na način da grupira vozila po njihovim putovanjima i odabire lokaciju stanice što bliže njima, čime fokusira skraćivanje putovanja. Suprotno tome algoritam graf neuronske mreže optimizira globalnu uspješnost, od kojih je važan kriterij pokrivenost mreže, uz kompetitivne kriterije, te žrtvuje optimalno kratka putovanja za bolju pokrivenost. Time korištenje algoritma teorije igara vodi do prenaučivosti na konkretno korištene podatke potražnje pri odabiru raspodjele stanica, dok model graf neuronske mreže bolje generalizira. Algoritmi pokrivenosti

generalno postižu nekonzistentni uspjeh s obzirom na vrstu alokacije vozila na stanice, što se može vidjeti i u razlici visine stupaca na dijagramu uspješnosti (Slika 6.3). No na podacima ovog dijagrama čini se da preferiraju sebičnu alokaciju umjesto centralizirane kao na dijagramu uspjeha.



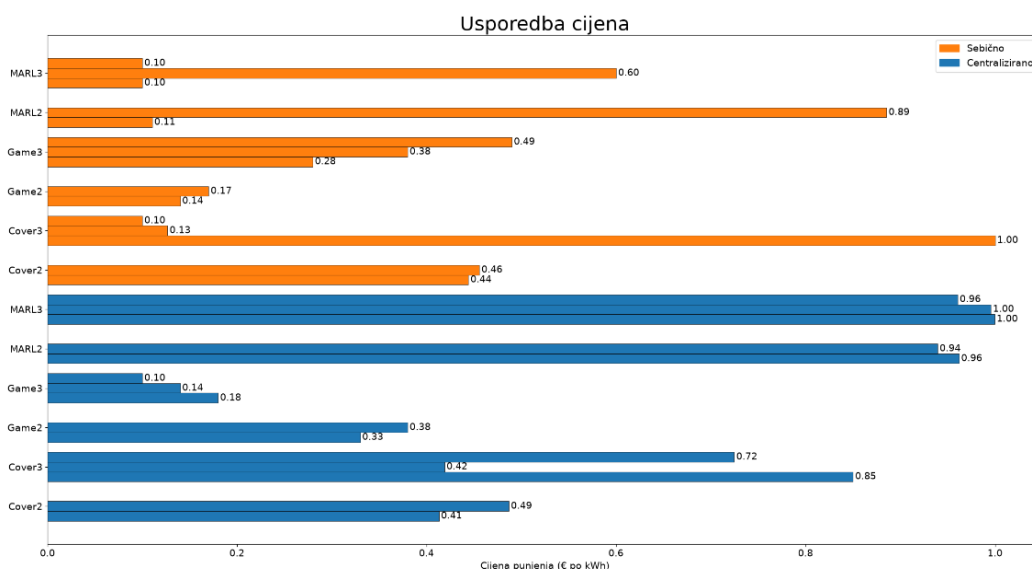
Slika 6.4 Prikaz vrijednosti rezultata globalnih kriterija najboljih rješenja algoritama na rešetkastoj mreži. Prva slika prikazuje primjenu sebične alokacije vozila po stanicama te druga slika prikazuje primjenu centralizirane alokacije vozila po stanicama.

Slijedi glavni fokus rada; dijagrami rezultata kompetitivnih kriterija, konkretno cijene (Slika 6.5), količine napunjene energije (Slika 6.6), tj. poslužene potražnje po agentu i pokrivenost mreže od strane stanica agenta (Slika 6.7). Ovo su također stupčasti dijagrami, grupirani po

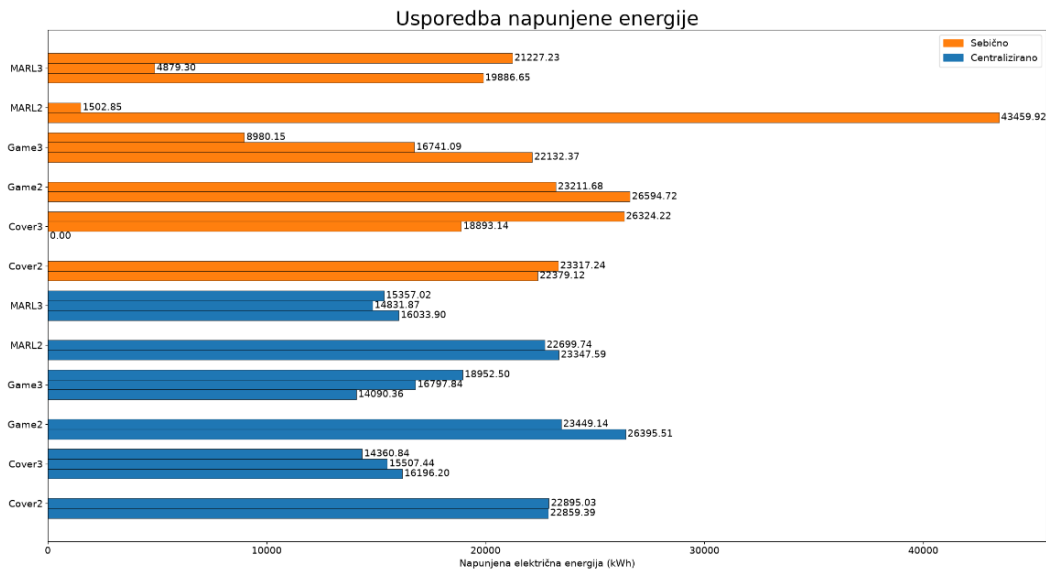
algoritmu, te svaki stupac unutar grupe predstavlja vrijednost odabranu ili postignutu od strane jednog od agenata. Grupe su također obojane u dvije boje ovisno o metodi alokacije vozila na stanice: narančasta boja za sebičnu alokaciju i plava boja za centraliziranu. Odmah se može uočiti zanimljivo ponašanje. Vidimo tri situacije do kojih je moglo doći:

- 1) Cijene nemaju dovoljan utjecaj za isticanje kompetitivnosti te su na kraju cijene svih agenata u visokim vrijednostima i agenti uglavnom podjednako dijele potražnju (npr. MARL3 algoritam sa centraliziranom alokacijom)
- 2) Dolazi do očekivanog utjecaja kompetitivnosti kroz cijene te su cijene svih agenata u niskim vrijednostima i agenti uglavnom podjednako dijele potražnju (npr. Game2 i Cover2 za sebičnu alokaciju, Game3 za centraliziranu)
- 3) Dolazi do ekstremne razlike u cijenama zbog jakog utjecaja kompetitivnosti kroz cijene te agent sa puno nižom cijenom dobiva puno veći omjer potražnje (npr. MARL3 za sebičnu alokaciju)

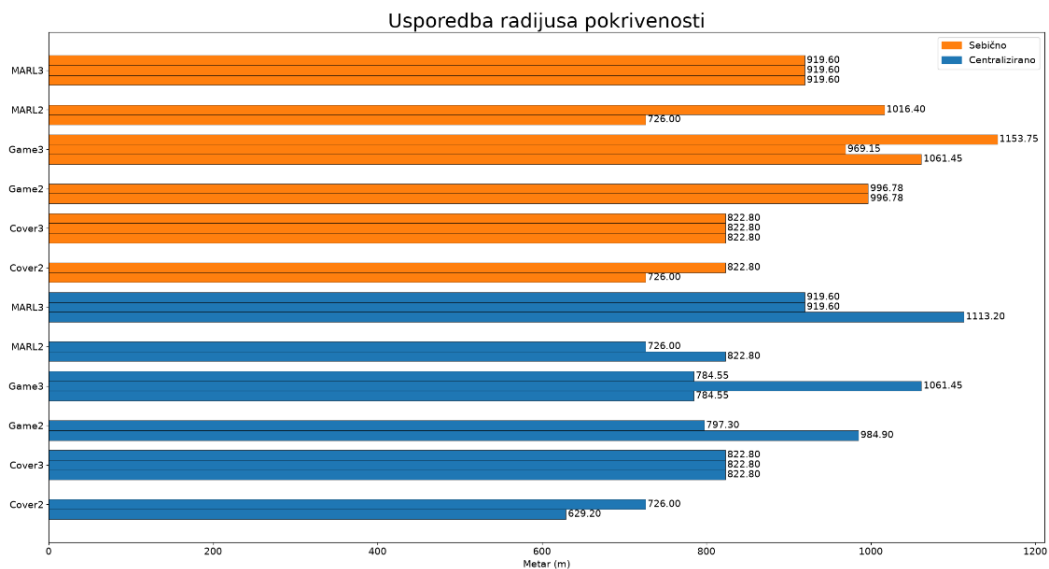
Također se pojavila situacija u slučaju Cover2 algoritma uz centraliziranu alokaciju gdje se potražnja podjednako dijeli između agenata iako postoji razlika u cijenama. Do toga dolazi i zbog bolje pokrivenosti stanica agenta sa većom cijenom, što vidimo u trećem dijagramu. Sve u svemu, može se uočiti da kompetitivnost pri sebičnoj alokaciji vozila na stanice dolazi do većeg izražaja, što je i za očekivati.



Slika 6.5 Prikaz vrijednosti kompetitivnog kriterija cijena najboljih rezultata algoritama na rešetkastoj mreži.



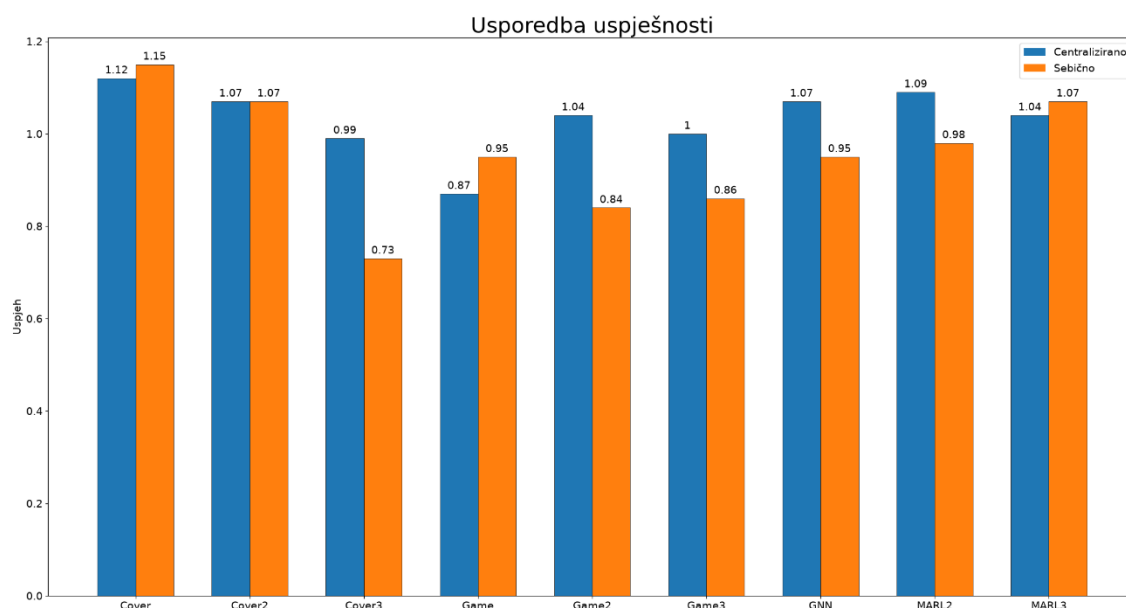
Slika 6.6 Prikaz vrijednosti kompetitivnog kriterija napunjenosti energije najboljih rezultata algoritama na rešetkastoj mreži.



Slika 6.7 Prikaz vrijednosti kompetitivnog kriterija radijusa pokrivenosti mreže najboljih rezultata algoritama na rešetkastoj mreži.

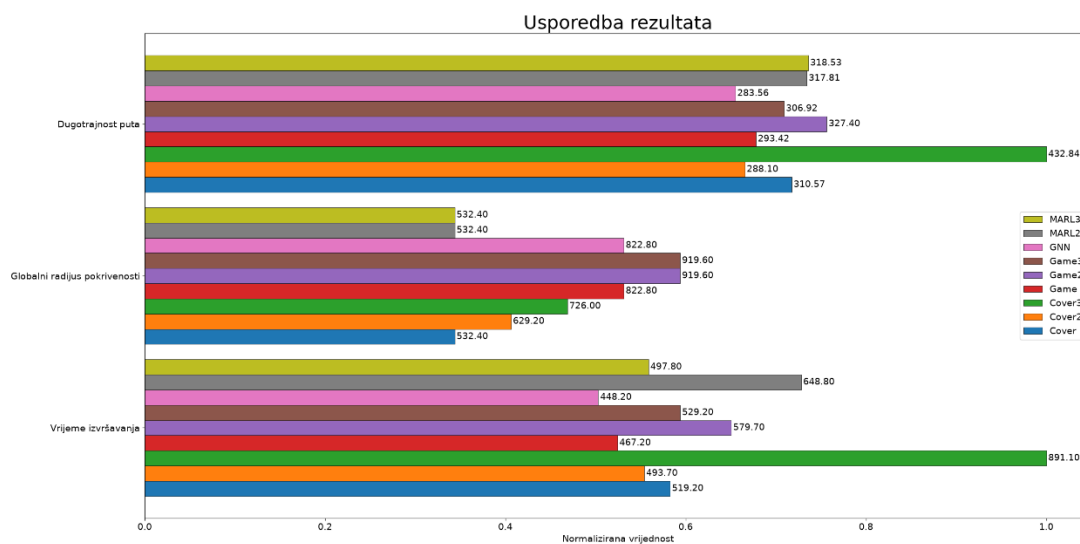
6.1.2. Ponovno pokretanje

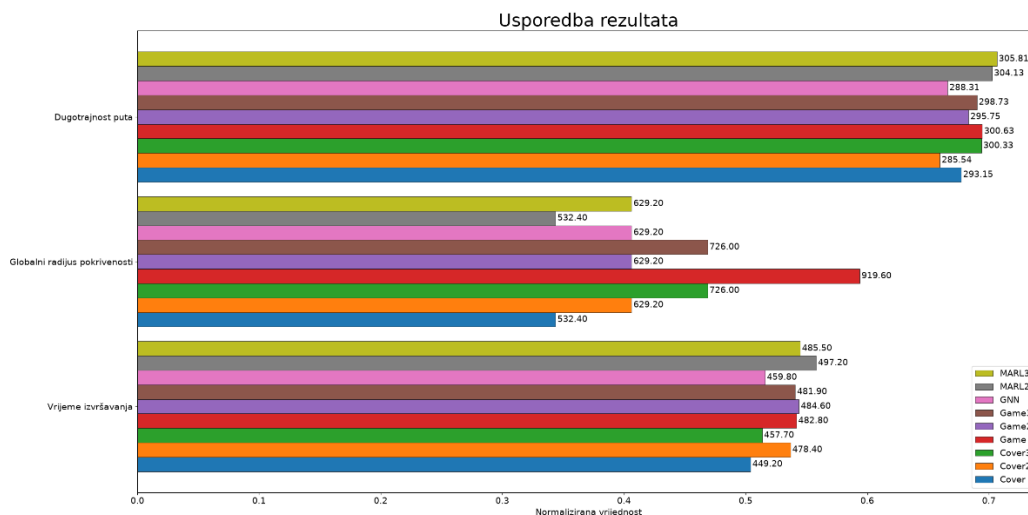
Pri analizi rezultata ponovnim pokretanjem (Slika 6.8). vidimo veće razlike između uspješnosti algoritama. U ovom slučaju algoritmi teorije igara postižu najmanji uspjeh, dok osnovni algoritam pokrivenosti postiže najveći uspjeh u grupi. Također se opet uočava dominacija algoritama pri primjeni centralizirane alokacije vozila.



Slika 6.8 Usporedba vrijednosti uspjeha svih algoritama na prometnoj mreži rešetke pri ponovnom pokretanju

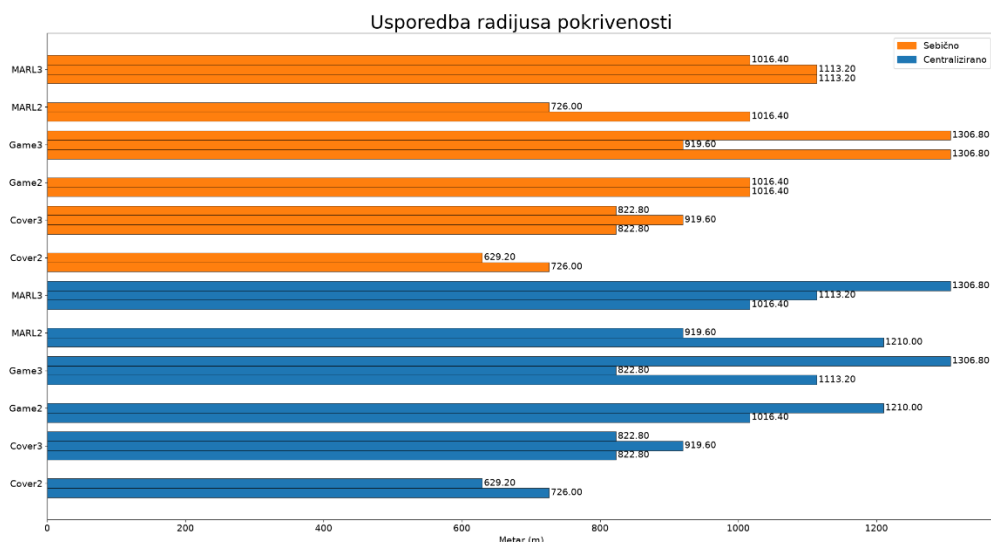
U dijagramu rezultata globalnih kriterija iz ponovnog pokretanja (Slika 6.9) vidimo općenito pogoršane rezultate u odnosu na najbolja rješenja (Slika 6.4). No zanimljivo je da su uzorci razlika relativnih vrijednosti između algoritama otprilike jednaki.





Slika 6.9 Prikaz vrijednosti rezultata globalnih kriterija ponovnog pokretanja algoritama na rešetkastoj mreži. Prva slika prikazuje primjenu sebične alokacije vozila po stanicama, dok druga slika prikazuje primjenu centralizirane alokacije vozila po stanicama

U slučaju rezultata kompetitivnih kriterija prikazujemo samo pokrivenost (Slika 6.10) jer kod cijena i podijele potražnje dolazi do istih već opisanih situacija. Iz dijagrama se može vidjeti da su uzorci pojavljivanja specifične situacije od opisanih opet slični kao kod dijagrama najboljih rješenja (Slika 6.5, Slika 6.6, Slika 6.7). No općenito su vrijednosti radijusa pokrivenosti puno veće, što prikazuje slabiju pokrivenost mreže.



Slika 6.10 Prikaz radijusa pokrivenosti mreže po agentu pri ponovnom pokretanju algoritama na rešetkastoj mreži

6.2. Zagreb

Mreža manjeg dijela grada Zagreba sastoji se od 333 čvorova i 584 bridova te ima promjer od 2879 metara. Broj izvršenih iteracija svakog algoritma prikazan je u (Tablica 6.4), a korišteni podaci potražnje u (Tablica 6.5). Broj iteracija smanjen je u slučaju algoritama teorije igara i modela graf neuronske mreže zbog utjecajno veće dugotrajnosti treniranja. Sama mreža puno je kompleksnija od rešetkaste jer je većina cesta jednosmjerno te sadrži dodatni element semafora na određenim raskrižjima. Budući da je prometna mreža „odsječena“ iz cjelokupne prometne mreže grada Zagreba također nije potpuno povezana, što znači da su za putovanja vozila i potencijalne lokacije stanica odabrani samo bridovi iz najveće komponente povezanosti mreže. Na kompleksnost utječu i mane SUMO alata za generiranje prometne mreže iz *OpenStreetMaps* podataka, kao velik broj vrlo kratkih i naizgled suvišnih bridova te čudne kompozicije većih križanja.

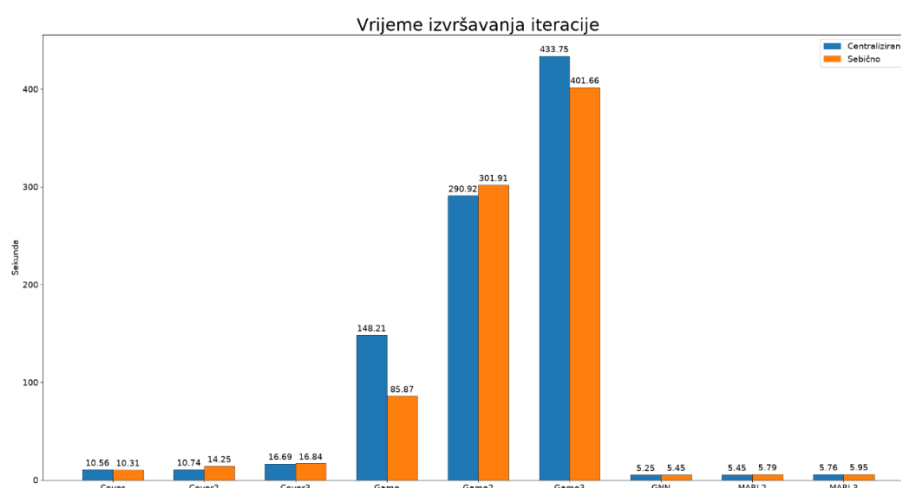
Tablica 6.4 Broj iteracija treniranja svakog od algoritama u slučaju prometne mreže dijela grada Zagreba

Algoritam pokrivenosti	Algoritam teorije igara	Model graf neuronske mreže
1000	50	2000

Tablica 6.5 Podaci potražnje korišteni u slučaju prometne mreže dijela grada Zagreba

Broj električnih vozila	80 vozila
Prosječna udaljenost putovanja	2948.78 metara
Prosječni broj odredišta	2.86 odredišta

Opet ćemo prvo prikazati vremena izvršavanja jedne iteracije za svakog od algoritama (Slika 6.11). Usporedbom ovog dijagrama sa prošlim (Slika 6.2) vidimo da je, kao i što je očekivano, potrebno dulje vrijeme za izvršavanje algoritama na većoj mreži. To vrijeme dodatno je ekstremno produljeno u slučaju algoritma teorije igara koji u svakoj iteraciji mora uračunati svaku od potencijalnih lokacija stanica. No uočavamo da je sada situacija obrnuta s obzirom na način alokacije vozila na stanice. U ovom slučaju je izvršavanje algoritama pri primjeni centralizirane alokacije dugotrajnije.

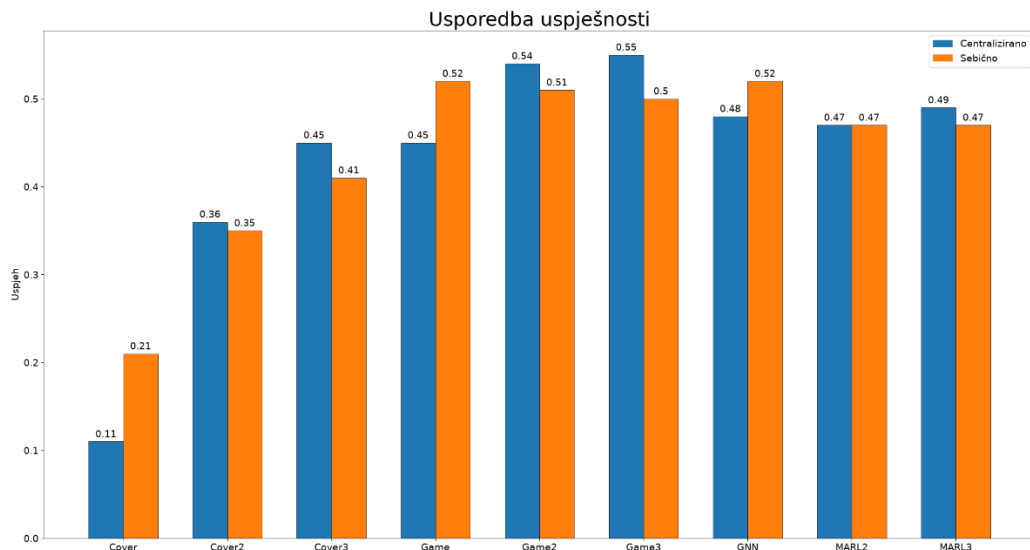


Slika 6.11 Usporedba vremena izvršavanja po iteraciji za sve algoritama na prometnoj mreži dijela grada Zagreba

6.2.1. Najbolje rješenje

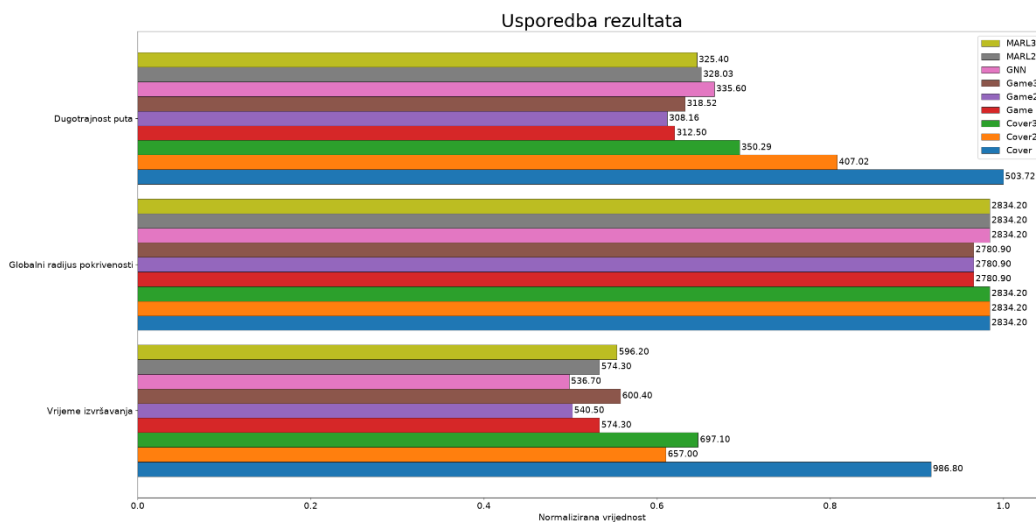
Na dijagramu uspješnosti za mrežu dijela grada Zagreba (Slika 6.12) može se odmah uočiti neuspjeh algoritama baziranih na pokrivenosti. Budući da je rešetkasta mreža jednostavna i potpuno povezana ostvarenje bolje pokrivenosti je lakše. No u slučaju realističnije mreže

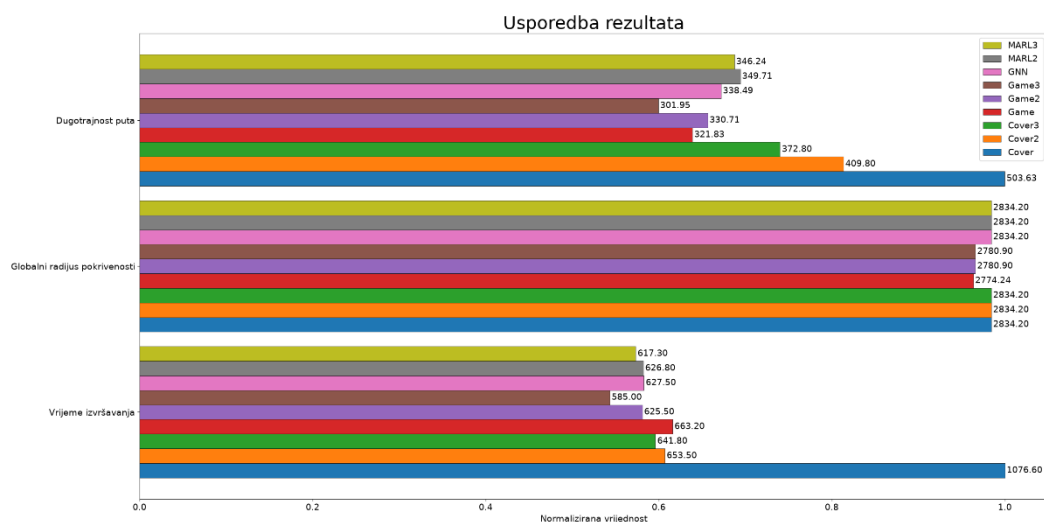
algoritam se potpuno raspada. U ovom slučaju pobjeđuje algoritam baziran na teoriju igara, te opet primjena centralizirane raspodjele dominira.



Slika 6.12 Usporedba vrijednosti uspjeha najboljih rješenja algoritama na prometnoj mreži dijela grada Zagreba

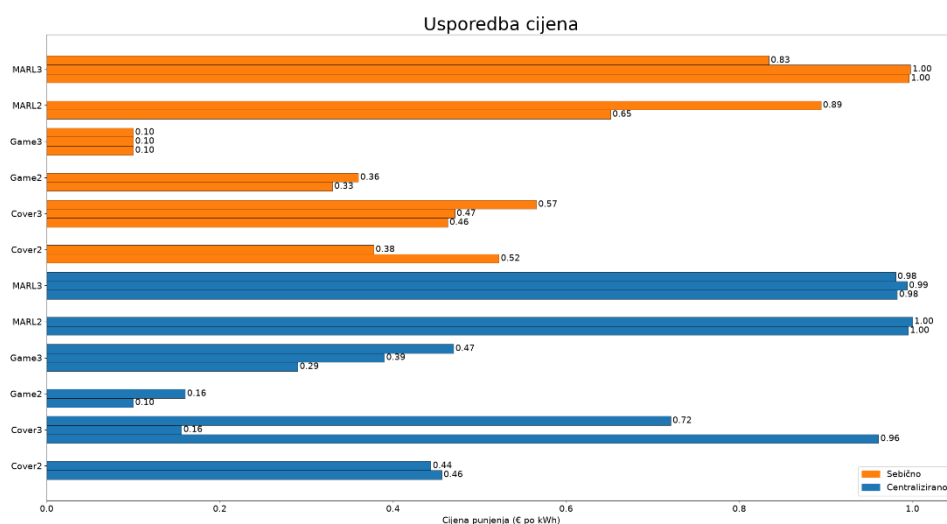
Analizom rezultata globalnih kriterija (Slika 6.13) odmah uviđamo spomenuto teško ostvarivanje dobre pokrivenosti nad mrežom, što se prikazuje visokim vrijednostima radijusa pokrivenosti za sve algoritme.



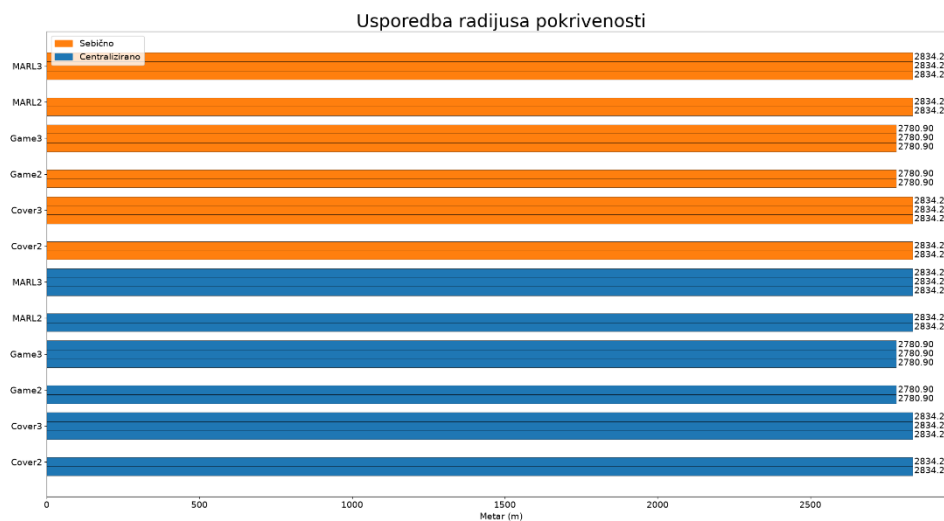


Slika 6.13 Prikaz vrijednosti rezultata globalnih kriterija najboljih rješenja algoritama na mreži dijela grada Zagreba. Prva slika prikazuje primjenu sebične alokacije vozila po stanicama, dok druga slika prikazuje primjenu centralizirane alokacije vozila po stanicama.

Cijene (Slika 6.14) opet prate slične uzorke od tri specifične situacije, ali češće dolazi do većih cijena i čak maksimalnih vrijednosti cijena. Zanimljivo je da je u slučaju Game3 algoritma čak došlo do postavljanja cijena na minimalne vrijednosti. Općenito je raspodjela cijena gora i kaotična nasuprot rešetkaste mreže (**Error! Reference source not found.**). Pokrivenost (Slika 6.15) je loša slično kao i u globalnoj perspektivi.



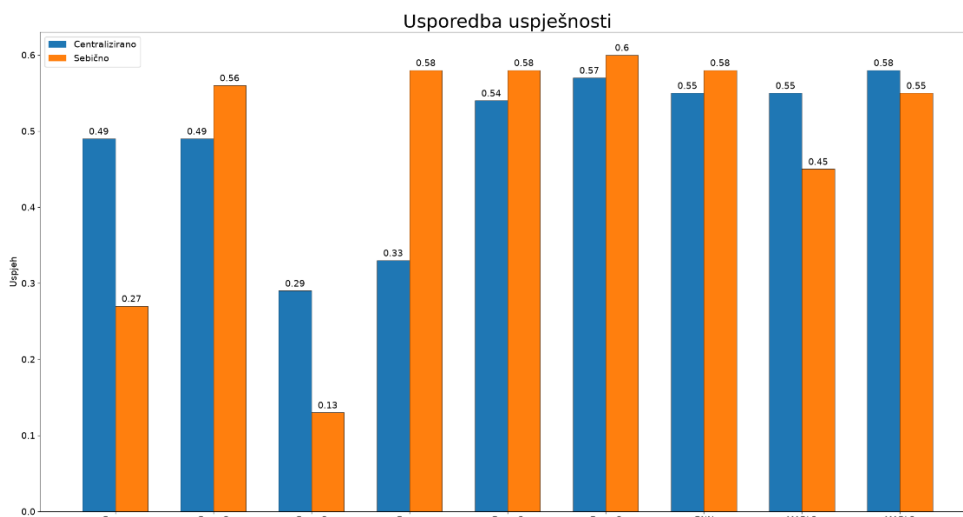
Slika 6.14 Prikaz vrijednosti kompetitivnog kriterija cijene najboljih rješenja algoritama na mreži dijela grada Zagreba



Slika 6.15 Prikaz vrijednosti kompetitivnog kriterija radijusa pokrivenosti najboljih rješenja algoritama na mreži dijela grada Zagreba

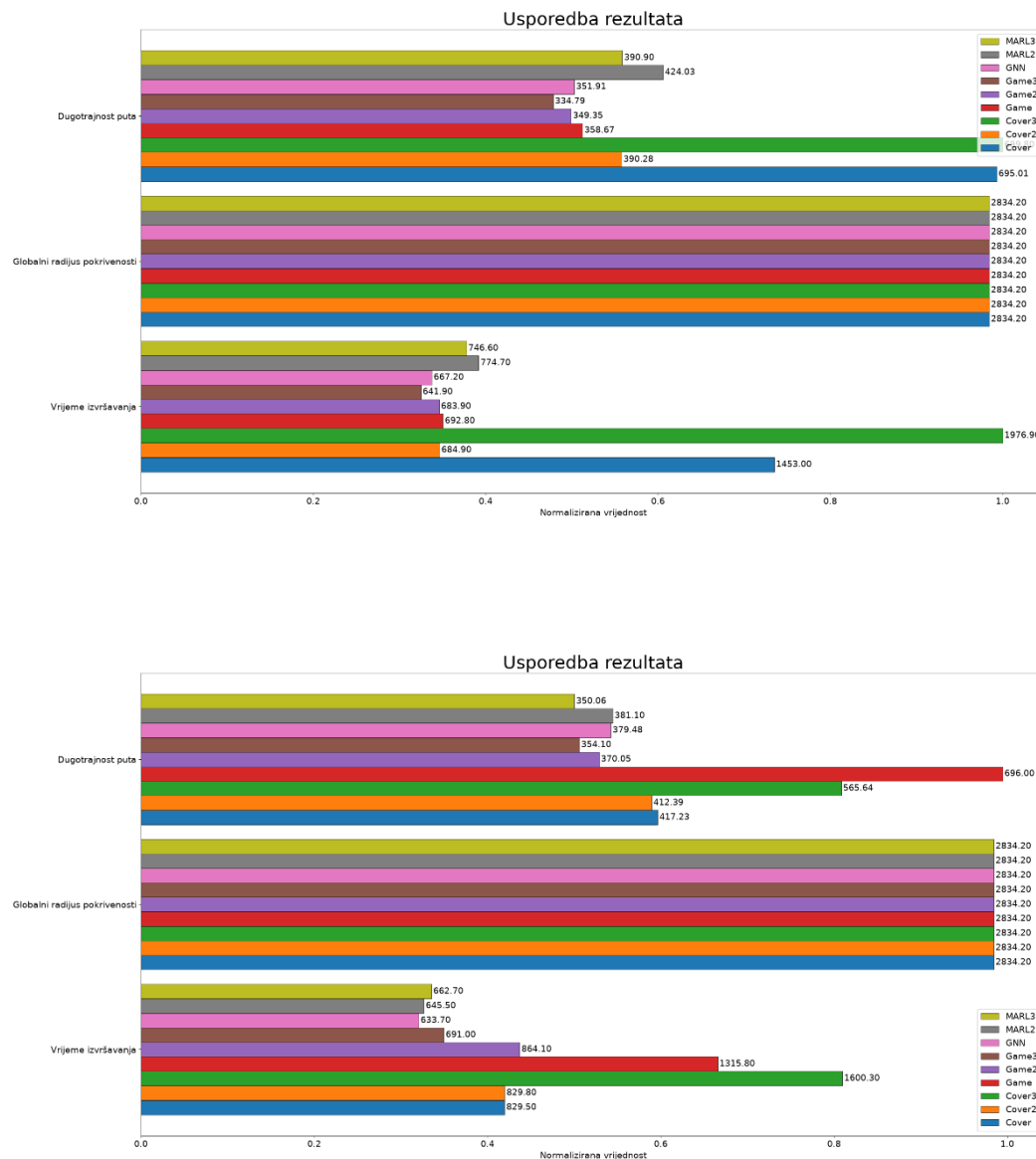
6.2.2. Ponovno pokretanje

Raspad algoritama pokrivenosti na mreži dijela grada Zagreba opet se jasno očituje iz dijagrama uspješnosti ponovnog pokretanja algoritama (Slika 6.16). Algoritmi teorije igara opet pobjeđuju, no ovaj put dominira primjena sebične alokacije vozila na stanice.



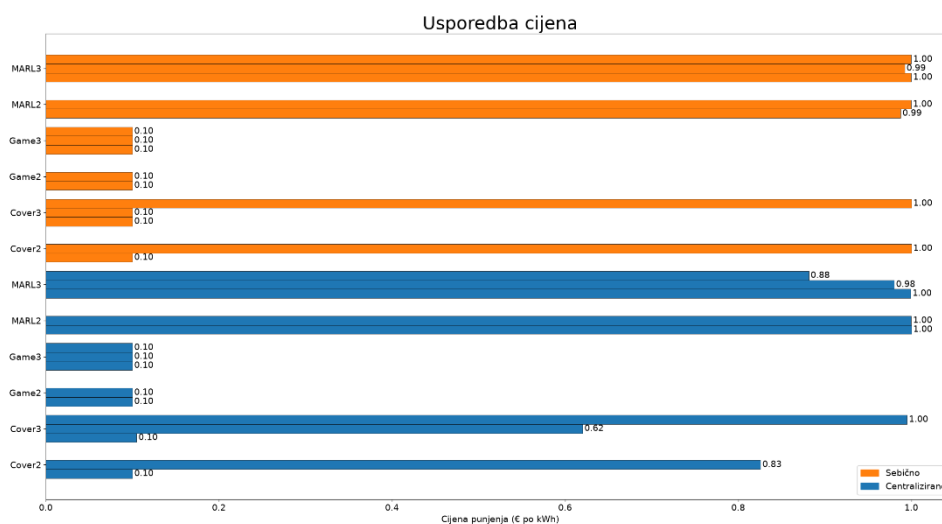
Slika 6.16 Usporedba vrijednosti uspjeha svih algoritama pri ponovnom pokretanju na prometnoj mreži dijela grada Zagreba

U dijagramu usporedbi rezultata pri ponovnom pokretanju algoritama na mreži dijela grada Zagreba (Slika 6.17) opet se jasno vidi teško ostvarivanje dobre pokrivenosti. No također vidimo specifično loše rezultate za algoritme pokrivenosti u svim područjima. Algoritam pokrivenosti jedino postiže relativno dobre rezultate u slučaju sustava dva agenta, zbog nepoznatog razloga. Možda sama struktura prometne mreže može pokazivati na optimalni broj agenta za optimalnu raspodjelu stanica.



Slika 6.17 Prikaz vrijednosti rezultata globalnih kriterija ponovnog pokretanja algoritama na mreži dijela grada Zagreba. Prva slika prikazuje primjenu sebične alokacije vozila po stanicama, dok druga slika prikazuje primjenu centralizirane alokacije vozila po stanicama.

Pri analizi cijena algoritama pri ponovnom pokretanju (Slika 6.18) mogu se vidjeti neočekivano loši rezultati. Cijene svih agenata su ili na maksimalnoj ili minimalnoj vrijednosti, uz miješanje situacija kada svi agenti postavljaju iste cijene i kada su razlike između cijena ekstremne. Do toga dolazi zbog spomenute loše pokrivenosti, što znači da je vozilima uglavnom blizu samo mali broj stanica te time udaljenost ima puno jači utjecaj na alokaciju vozila po stanicama od cijene. Ostali dijagrami nisu prikazani zbog ekvivalentnih rezultata kao i prije.



Slika 6.18 Prikaz vrijednosti cijena pri ponovnom pokretanju algoritama na mreži dijela grada Zagreba

Zaključak

Cilj rada bila je obrada problema optimalne raspodjele stanica za punjenje električnih vozila u kontekstu mješovitog autonomnog prometa i ispitivanje korisnosti dodavanja elementa kompetitivnosti u pokušaj rješavanja problema. Na početku rada opisana su električna vozila i autonomni promet, uz dodatan naglasak na stanice za punjenje električnih vozila.

Potom je analizirana postojeća literatura koja pridonosi različite tehnike i pristupe rješavanja problema koji su zatim dodatno komentirani u kontekstu ovog rada. U sljedećem poglavlju dodatno su opisane spomenute tehnike i pristupi, uz formalizaciju i prikaz postupa primjene na rješavanje problema optimalne raspodjele stanica. Sam problem optimalne raspodjele stanica za punjenje električnih vozila konkretno je formaliziran u kontekstu rada i objašnjen je model prometne mreže i stanica za punjenje koji se analizira u radu.

Zatim je opisana implementacija: korišteni alati, implementacija modela i funkcija te ponašanja vozila. Prikazan je oblik implementacije stanice za punjenje električnih vozila unutar *SUMO* simulacije te način dodavanja definirane stanice na bilo koji od bridova mreže. Za kraj poglavlja detaljnije je opisan graf neuronske mreže koji je jedna od temeljnih tehnika razvijenih rješenja rada.

Slijedi prikaz tri razvijena algoritma i njihovih kompetitivnih varijanta. Svaki od algoritama je detaljno objašnjen uz pseudokod i način na koji je nadograđen u kompetitivnu varijantu, uključujući način dodavanja elementa odlučivanja vrijednosti cijene punjenja vozila na stanici. Sva razvijena rješenja primijenjena su na dvije odabrane mreže: prva je sintetički generirana po strukturi pravilne rešetke, dok je druga generirana iz stvarnih podataka odabranog dijela grada Zagreba.

Algoritmi su analizirani kroz najbolja postignuta rješenja tijekom treniranja i kroz naučena stanja, ispitana ponovnim pokretanjem algoritama u naučenim stanjima. Rezultati najbolje postignutih rješenja pokazuju moguću efektivnost algoritama, no pri ponovnom pokretanju na generiranoj mreži dijela grada Zagreba rezultati nisu poželjni. No dobiveni rezultati pokazuju na moguću korisnost kompetitivnosti pri optimizaciji raspodjele stanica, iako uglavnom samo u kontroliranim uvjetima. U slučaju rešetkaste mreže prirodno se pojavljuje poželjna situacija utjecaja kompetitivnosti na smanjenje cijena svih agenata sustava, no pri

ponovnom pokretanju algoritama naučenih na generiranu prometnu mrežu manjeg dijela grada Zagreba sve cijene su postavljene na ekstremne vrijednosti.

No razlog dobivanja loših rezultata na prometnoj mreži dijela grada Zagreba dijelom je i zbog loše interne strukture mreže dobivene generiranjem iz preuzetih *OpenStreetMap* podataka. Za buduće, kvalitetnije ispitivanje efektivnosti razvijenih algoritama na stvarnim prometnim mrežama bilo bi korisno razviti bolju implementaciju generiranja mreže iz tih podataka, sa ciljem da je generirana mreža puno sličnija originalnoj mreži i da je puno manje kompleksnosti, tj. da ne sadrži suvišne kratke bridove te da su kompozicije raskrižja pravilno generirane. Također bi dodatno ispitivanje algoritama na većim mrežama i kroz veći broj iteracija pridonijelo boljem shvaćanju njihovih koristi i nedostataka. No, čak i u rezultatima prikazanima u ovom radu, svaki je pokazao dovoljnu korisnost u nekoj od simuliranih situacija. Stoga bi razvoj novog algoritma koji koristi kombinaciju koncepata navedenih u radu mogao rezultirati vrlo efektivnim i prilagodljivim algoritmom. Izuzev samih algoritama, element kompetitivnosti pri rješavanju problema pokazao se uspješnim u jednostavnijoj mreži te bi bilo korisno primijeniti ga i u ostalim područjima i pristupima rješavanju problema. To pogotovo vrijedi pri modeliranju realističnih situacija, jer ekonomsko stanje trenutno postojećih stanica za punjenje vozila navodi do zaključka da će i buduće stanice za punjenje električnih vozila biti pod vlasništvom više vlasnika, te time u kompetitivnoj okolini.

Literatura

- [1] Albatayneh, Aiman, et al. *Comparison of the overall energy efficiency for internal combustion engine vehicles and electric vehicle*, Environmental and Climate Technologies, 24,1 (2020.), str. 669-680
- [2] Poornesh, K. , Nivya, K.P., Sireesha, K. *A Comparative study on Electric Vehicle and Internal Combustion Engine Vehicles*, 2020 International Conference on Smart Electronics and Communication (ICOSEC), Trichy, India, (2020.), str. 1179-1183.
- [3] Høyer, K. G. *The history of alternative fuels in transportation: The case of electric and hybrid cars*, Utilities Policy, 16,2 (2008.), str. 63-71.
- [4] Hussain, R., Zeadally, S. *Autonomous Cars: Research Results, Issues, and Future Challenges*, IEEE Communications Surveys & Tutorials, 21,2 (2019.), str. 1275-1313
- [5] Lam, A.Y.S., Leung, Y., Chu, X., *Electric Vehicle Charging Station Placement: Formulation, Complexity, and Solutions*, IEEE Transactions on Smart Grid, 5,6 (2014.), str. 2846-2856.
- [6] Veisi, M., Naderian, H., Karimi, M. *Optimal charging station placement of electric vehicles in the smart distribution network based on the mixed integer linear programming*, International Journal of Electrical Power & Energy Systems, 168 (2025.), str. 110675
- [7] Frade, I., Ribeiro, A., Gonçalves, G., Antunes, A. P., *Optimal location of charging stations for electric vehicles in a neighborhood in Lisbon, Portugal*, Transportation Research Record: Journal of the Transportation Research Board, 2252,1, str. 91-98.
- [8] Zhou, G., Zhu, Z., Luo, S., *Location optimization of electric vehicle charging stations: Based on cost model and genetic algorithm*, Energy, 247 (2022.), str. 123437
- [9] Hidalgo, P. A. L., Ostendorp, M., Lienkamp, M., *Optimizing the charging station placement by considering the user's charging behavior*, IEEE International Energy Conference ENERGYCON, Leuven, (2016.), str. 1-7.
- [10] Deb, S., Gao, X.-Z., Tammi, K., Kalita, K., Mahanta, P. *A novel chicken swarm and teaching learning based algorithm for electric vehicle charging station placement problem*, Energy, 220 (2021.), str. 119645.
- [11] Bae, S., Jang, I., Gros, S., Kulcsar, B., Hellgren, J. *A Game Approach for Charging Station Placement Based on User Preferences and Crowdedness*, IEEE Transactions on Intelligent Transportation Systems, 23,4 (2022.), str. 3654-3669.
- [12] Xiong, Y., Gan, J., An, B., Miao, C., Bazzan, A. L. C., *Optimal Electric Vehicle Fast Charging Station Placement Based on Game Theoretical Framework*. IEEE Transactions and Intelligent Transportation Systems, 19,8 (2018.), str. 2493.-2504.
- [13] Roy, A., Law, M., *Examining spatial disparities in electric vehicle charging station placements using machine learning*, Sustainable Cities and Society, 83, 103978 (2022.)

- [14] Jana, D., Malama S., Narasimhan S., Taciroglu, E. *Edge-based graph neural network for ranking critical road segments in a network*, PLOS ONE, 12,18 (2023.), str. 1-36.
- [15] Cornuéjols G., Nemhauser G. L., Wolsey L. A. *The uncapacitated facility location problem*, Cornell University, School of Operations Research and Industrial Engineering, Technical Report 605 (1983.)
- [16] Handler, G. Y., Rozman, M. *The continuous m-center problem on a network*, Networks, 15,2 (1985.), str. 191–204
- [17] Dufwenberg, M., Gneezy, U. *Price competition and market concentration: an experimental study*. International Journal of Industrial Organization, 18,1 (2000.), str 7-22.
- [18] von Neumann, J., Morgenstern, O. *Theory of Games and Economic Behaviour*. Princeton: Princeton University Press, 1944., Chapter III.
- [19] Nash, J. F., *Equilibrium Points in N-Person Games*, Proceedings of the National Academy of Sciences, 36,1 (1950.), str. 48-49
- [20] Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., Monfardini, G., *The Graph Neural Network Model*, IEEE Transactions on Neural Networks, 20,1 (2009.), str. 61-80.
- [21] Di Massa, V., Monfardini, G., Sarti, L., Scarselli, F., Maggini, M., Gori, M., *A Comparison between Recursive Neural Networks and Graph Neural Networks*, Proceedings of the International Joint Conference on Neural Networks, Vancouver, (2006.), str. 778-785.
- [22] Kipf, T. N., Welling, M. *Semi-Supervised Classification with Graph Convolutional Networks*, International Conference on Learning Representations (2017.)
- [23] Lopez, P. A. et al. *Microscopic Traffic Simulation using SUMO*, 21st International Conference on Intelligent Transportation Systems (ITSC), Maui (2018.), str. 2575-2582
- [24] German Aerospace Center (DLR) et al., *TraCI*, (2026., veljača). Poveznica: <https://sumo.dlr.de/docs/TraCI/index.html>; pristupljeno 7. lipnja 2026.
- [25] German Aerospace Center (DLR) et al., *Stationfinder device*, (2025., ožujak). Poveznica: <https://sumo.dlr.de/docs/Simulation/Stationfinder.html>; pristupljeno 7. lipnja 2026.
- [26] Hagberg, A. A., Schult, D. A., Swart, P. J. *Exploring network structure, dynamics, and function using NetworkX*, Proceedings of the 7th Python in Science Conference (SciPy 2008), Pasadena, (2008.), str. 11-15
- [27] Boeing, G. *OSMnx: A Python package to work with graph-theoretic OpenStreetMap street networks*. Journal of Open Source Software, 2,12, (2017.), str. 1-8
- [28] Haklay, M., Weber, P. *OpenStreetMap: User-Generated Street Maps*. IEEE Pervasive Computing, 7,4 (2008.), str. 12-18
- [29] OpenStreetMap contributors. OpenStreetMap. Poveznica: <https://www.openstreetmap.org/>; pristupljeno 5. lipnja 2026.

Sažetak

U ovom radu proučena je optimalna raspodjela stanica za punjenje električnih vozila u kontekstu mješovitog autonomnog prometa u slučaju univerzalnih stanica i kompetitivne igre između više različitih vlasnika stanica. Definiran je koncept autonomnog prometa uz sebično i centralizirano raspoređivanje električnih vozila na stanice. Nabrojene su i opisane postojeće tehnike i pristupi rješavanja problema. Formaliziran je problem i model problema analiziran u radu. Opisana je implementacija modela i mikroskopska simulacija uz pomoć *SUMO* programskog paketa. Prezentirana su tri razvijena algoritma bazirana na različitim pristupima rješavanja problema uz njihove kompetitivne varijacije. Razvijeni algoritmi primjenjuju se na sintetičku mrežu i mrežu generiranu iz stvarnih podataka djela grada Zagreba. Uspoređuju se rezultati te obrađuju prednosti i nedostaci elementa kompetitivnosti u problemu optimalne raspodjele stanica.

Ključne riječi: električna vozila, autonomni promet, stanica za punjenje električnih vozila, optimalna raspodjela stanica, optimizacija, mješovito cjelobrojno linearno programiranje, teorija igara, graf neuronska mreža, *Eclipse SUMO*, *OpenStreetMap*

Summary

This paper studies the optimal electric vehicle charging station placement in the context of mixed autonomous traffic, considering universal stations and a competitive game between multiple station owners. The concept of autonomous traffic is defined under both selfish and centralized allocation of electric vehicles to charging stations. Existing methods and approaches for solving the problem are listed and described. The problem is formalized, with the model analysed in the paper. The implementation of the model and microscopic simulation using the SUMO simulation package are described. Three developed algorithms based on different approaches are described, along with their competitive variants. The developed algorithms are applied to a synthetic network and a network generated from real-world data from a part of the traffic network of the city of Zagreb. The results are compared, and the advantages and disadvantages of introducing competitiveness into the problem of optimal station placement are discussed.

Keywords: electric vehicles, autonomous traffic, electric vehicle charging station, optimal charging station placement, optimization, mixed-integer linear programming, game theory, graph neural network, Eclipse SUMO, OpenStreetMap

Privitak

Instalacija programske podrške

Potrebno je prvo preuzeti *Eclipse SUMO* programski paket sa službene poveznice (korištena je verzija 1.27.0):

<https://eclipse.dev/sumo/>

Programska podrška razvijena za izvođenje algoritama nalazi se na poveznici:

<https://github.com/IanM129/ChargingStationPlacementComparison>

Potrebno je zatim preuzeti datoteke sa poveznice i pokrenuti datoteku „run.py“. Pokrenuta skripta automatski će napraviti novu virtualnu okolinu u istom direktoriju sa potrebnim modulima.

Upute za korištenje programske podrške

Prilikom pokretanja „run.py“ datoteke i nakon instalacije virtualne okoline pojavit će se glavni izbornik. Glavni izbornik prvo prikazuje generalne opcije:

- 1) Generiranje novih podataka o vozilima, dodatno objašnjeno u poglavlju (4.3.1)
- 2) Usporedba rezultata prošlih simulacija
- 3) Učitavanje i vizualizacija rezultata jedne simulacije

Te su zatim prikazane opcije izvođenja:

- 4) Prazne simulacije mreže
- 5) Algoritama baziranog na pokrivenosti
- 6) Algoritma baziranog na teoriji igara
- 7) Učenje modela graf neuronske/ih mreže/a

Svaka od opcija izvršavanja algoritama (opcije od 4 do 7) može se pozvati uz broj nakon indeksa ili komande koji označava broj agenata. Umjesto broja može se navesti nula ili „_“ pri čemu će program učitati broj agenata iz konfiguracijske datoteke („config.xml“). Također se uz pomoć dva opcionalna argumenata mogu koristiti generirani podaci potražnje („vehicle data“) spremljeni u „/vehicle_data“ direktoriju ili direktno postaviti broj iteracija, dok se u suprotnom učitava iz konfiguracijske datoteke. Pozivanjem jedne od tih opcija izvršava se pripadni algoritam i simulacije u novom prozoru.

Odabirom opcije pod 1) prikazuje se ispis postojećih prometnih mreža. U repozitoriju postoje samo mreže korištene u radu; rešetkasta mreža, „Manhattan“, i mreža generirana iz dijela grada Zagreba, „Zagreb_small“. Moguće je dodati nove mreže stvaranjem novog direktorija unutar direktorija „networks“ i kreiranjem dvije datoteke: „base_net.net.xml“, koja definira strukturu mreže na način kompatibilan sa *Eclipse SUMO* simulacijskim paketom, i „<ime mreže>.sumocfg“ *SUMO* konfiguracijske datoteke koja definira pokretanje mreže. Datoteka mreže može se kreirati iz *OpenStreetMaps* datoteke („osm“ ekstenzija) koristeći „convert_osm_to_net.py“ skripte. Za pokretanje skripte potrebno je staviti „osm“ datoteku u direktorij nove mreže i zatim pokrenuti skriptu te upisati naziv direktorija nove mreže, nakon čega će skripta automatski generirati „base_net.net.xml“ datoteku mreže. *SUMO* konfiguracijska datoteka uvijek je ista pa se može samo kopirati i preimenovati postojeća iz neke druge mreže. Na primjer, za dodavanje mreže „Test“ stvorili bi direktorij „networks/test“ i u njemu kreirali datoteku „base_net.net.xml“ te kopirali konfiguracijsku datoteku „networks/manhattan/manhattan.sumocfg“ u direktorij i preimenovali je u „test.sumocfg“. Odabirom mreže nakon odabira opcije generiranja potražnje vozila program će generirati potražnju kompatibilnu sa odabranom mrežom.

Odabirom opcije pod 2) prikazuje se lista postojećih rezultata. Budući da ima uglavnom smisla uspoređivati rezultate nastale na istoj mreži, podacima o potražnji i postavkama, lista je grupirana prvo po direktorijima a zatim po postavkama. Na vrhu piše ime direktorija u kojem se nalaze odmah ispod ispisani rezultati, zatim pišu glavne postavke: mreža, broj stanica, način alokacije vozila na stanice. Zatim su ispisani rezultati uz informacije kada su pokrenuti, na kojoj mreži su pokrenuti, koji je algoritam korišten i s koliko agenata te postavke pod kojima su izvršeni. Upis indeksa prikazanog ispred grupe učitavaju se svi rezultati grupe, te upisom točke i zatim broja učitava se samo rezultat pod tim indeksom. Na primjer; ispis rezultata je sljedeći (Slika 0.1):

```

Choose sessions to compare:
=====
results\final\manhattan\centralized
=====
'manhattan' | k = 6 | centralized
=====
1 | manhattan_20260620_202333 (2) |100|
.1: Cover > 22.06.2026, 06:05:02, manhattan
[ Coverage | Solo agent | K: 6 | Centralized ]
.2: Cover2 > 23.06.2026, 00:40:23, manhattan
[ Coverage | Agents: 2 | K: 6 | Centralized ]
=====
results\final\manhattan\selfish
=====
'manhattan' | k = 6 | selfish
=====
2 | manhattan_20260620_202333 (2) |100|
.1: Cover > 21.06.2026, 19:27:41, manhattan
[ Coverage | Solo agent | K: 6 | Selfish ]
.2: Cover2 > 23.06.2026, 00:42:38, manhattan
[ Coverage | Agents: 2 | K: 6 | Selfish ]

```

Slika 0.1 Primjer ispisa rezultata grupiranih po direktorijima i postavkama

Upisom „2“ odabrat ćemo drugu grupu rezultata, što uključuje „Cover“ i „Cover2“ rezultate pod sebičnom alokacijom vozila. Upisom „2.2“ učitava se samo „Cover2“ rezultat, tj. drugi rezultat druge grupe umjesto cijele druge grupe. Također, upisom „1 2“ učitavaju se obje grupe zajedno, a sa „1 2.2“ učitava se prva grupa i drugi primjer druge grupe. Odabirom jedne od grupa ispisuje se pod-izbornik a tri opcije:

- 1) Usporedba ponovnim pokretanjem
- 2) Usporedba najboljim rezultatima
- 3) Usporedba trajanja izvršavanja

Nakon odabira opcije izvršiti će se pripadna usporedba i otvoriti grafovi sa prikazanim rezultatima.

Odabirom opcije pod 3) pojavljuje se popis rezultata, no ovaj put nisu grupirani. Željeni rezultat odabire se upisom indeksa pored ispisa njegovih informacija. Još jedna razlika u ovom ispisu sa prošlim je da su ispisani putevi do rezultata unutar glavnog direktorija svih rezultata umjesto samo imena samog direktorija rezultata. Odabirom rezultata otvara se novi pod-izbornik s dvije opcije:

- 1) Vizualizacija ponovnog pokretanja
- 2) Vizualizacija podataka o treniranju

Odabirom prve opcije pokreće se simulacija koja koristi stanice odabrane ponovnim pokretanjem rezultata i simulacija se vizualizira kroz *SUMO* grafičko sučelje. Odabirom

druge opcije otvaraju se grafovi koji prikazuju vrijednosti rezultata evaluacija pokrenutih kroz treniranje.